소스 모음

화면과 빌드 소스를 파일 순서대로 이어 두었습니다. 주석은 코드 옆에 그대로 있습니다. data/ 과제 코드, 압축된 라이브러리, 빌드가 만든 data.js, hero-images.js, sources JSON은 넣지 않았습니다.

cover_letter/css/styles.css

```
/* 자기소개 페이지의 밝은 배경, 청록 강조색, 고정 사이드바를 정한다. */

:root {
  --navy: #f8fafc;
  --navy-light: #ffffff;
  --slate: #475569;
  --lightest-slate: #0f172a;
  --white: #0f172a;
  --green: #0d9488;
  --green-dim: rgba(13, 148, 136, 0.12);
  --profile-ring-shadow:
    0 0 0 1px color-mix(in srgb, var(--green) 32%, transparent),
    0 10px 28px rgba(15, 23, 42, 0.12);
  --profile-ring-hover-shadow:
    0 0 0 1px color-mix(in srgb, var(--green) 50%, transparent),
    0 12px 32px rgba(15, 23, 42, 0.15);
  --font-sans: "Malgun Gothic", "Apple SD Gothic Neo", "Segoe UI", system-ui, sans-serif;
  --fz-xs: 0.75rem;
  --fz-sm: 0.875rem;
  --fz-md: 1rem;
  --fz-lg: 1.125rem;
  --space: 1rem;
  --sidebar-w: 300px;
  --header-offset: 6rem;
  --radius: 4px;
  --transition: 0.2s ease;
}

html[data-theme="dark"] {
  --navy: #0c0c0c;
  --navy-light: #161616;
  --slate: rgba(245, 245, 245, 0.66);
  --lightest-slate: #f5f5f5;
  --white: #f5f5f5;
  --green: #64ffda;
  --green-dim: rgba(100, 255, 218, 0.12);
  --profile-ring-shadow:
    0 0 0 1px color-mix(in srgb, var(--green) 22%, transparent),
    0 12px 40px rgba(0, 0, 0, 0.35);
  --profile-ring-hover-shadow:
    0 0 0 1px color-mix(in srgb, var(--green) 45%, transparent),
    0 16px 48px rgba(0, 0, 0, 0.4);
}

*,
*::before,
*::after {
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}

body {
  margin: 0;
  font-family: var(--font-sans);
```

```
  font-size: var(--fz-md);
  line-height: 1.7;
  color: var(--slate);
  background-color: var(--navy);
  -webkit-font-smoothing: antialiased;
  transition: background-color 0.25s ease, color 0.25s ease;
}

a {
  color: var(--lightest-slate);
  text-decoration: none;
  transition: color var(--transition);
}

a:hover {
  color: var(--green);
}

.visually-hidden {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}

.skip-link {
  position: absolute;
  left: -9999px;
  z-index: 2000;
  padding: 0.75rem 1rem;
  background: var(--green);
  color: var(--navy);
  font-weight: 600;
  border-radius: var(--radius);
}

.skip-link:focus {
  left: 1rem;
  top: 1rem;
}

/* Layout */
.layout {
  display: grid;
  grid-template-columns: minmax(200px, var(--sidebar-w)) minmax(0, 1fr);
  gap: 4rem;
  max-width: 1280px;
  margin: 0 auto;
  padding: var(--header-offset) 3rem 6rem 3rem;
  min-height: 100vh;
}

/* Sidebar */
.sidebar {
  position: sticky;
  top: var(--header-offset);
  align-self: start;
  height: fit-content;
  z-index: 50;
}

.sidebar-inner {
```

```
  display: flex;
  flex-direction: column;
  gap: 2.5rem;
}

.brand {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
}

/* 프로필 사진 — 틸 그라데이션 링 + 원형 크롭 */
.profile-figure {
  margin: 0 0 1.35rem;
  padding: 0;
}

.profile-photo-ring {
  width: 136px;
  height: 136px;
  padding: 3px;
  border-radius: 50%;
  background: linear-gradient(
    145deg,
    color-mix(in srgb, var(--green) 55%, transparent),
    color-mix(in srgb, var(--green) 18%, transparent) 50%,
    color-mix(in srgb, var(--green) 42%, transparent)
  );
  box-shadow: var(--profile-ring-shadow);
  transition: box-shadow 0.3s ease, transform 0.35s ease;
}

.profile-figure:hover .profile-photo-ring {
  box-shadow: var(--profile-ring-hover-shadow);
  transform: translateY(-2px);
}

.profile-photo {
  display: block;
  width: 100%;
  height: 100%;
  border-radius: 50%;
  object-fit: cover;
  object-position: center 18%;
  background: var(--navy-light);
  border: 2px solid var(--navy);
  box-sizing: border-box;
  transition: transform 0.35s ease;
}

.profile-figure:hover .profile-photo {
  transform: scale(1.03);
}

.brand-name {
  margin: 0;
  font-size: clamp(2rem, 4vw, 2.6rem);
  font-weight: 700;
  letter-spacing: -0.04em;
  color: var(--lightest-slate);
  line-height: 1.15;
}

.brand-role {
  margin: 0.35rem 0 0;
  font-size: var(--fz-lg);
  font-weight: 500;
```

```
  color: var(--lightest-slate);
  letter-spacing: -0.02em;
}

.brand-tagline {
  margin: 1rem 0 0;
  max-width: 320px;
  font-size: var(--fz-sm);
  line-height: 1.65;
  color: var(--slate);
}

/* Side nav — vertical, BC-style */
.side-nav-list {
  list-style: none;
  margin: 0;
  padding: 0;
}

.side-nav-list li {
  margin: 0;
}

.nav-link {
  display: inline-flex;
  align-items: center;
  gap: 0.75rem;
  padding: 0.35rem 0;
  font-size: var(--fz-xs);
  font-weight: 600;
  letter-spacing: 0.14em;
  text-transform: uppercase;
  color: var(--slate);
  transition: color var(--transition);
}

.nav-link:hover,
.nav-link:focus-visible {
  color: var(--lightest-slate);
  outline: none;
}

.nav-link.is-active {
  color: var(--lightest-slate);
}

.nav-marker {
  display: block;
  width: 2rem;
  height: 1px;
  background: var(--slate);
  transition: width var(--transition), background var(--transition);
  flex-shrink: 0;
}

.nav-link.is-active .nav-marker {
  width: 3.5rem;
  background: var(--lightest-slate);
}

.nav-link:hover .nav-marker {
  background: var(--lightest-slate);
  width: 3rem;
}

.social-list {
  list-style: none;
```

```
  margin: 0;
  padding: 0;
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}

.social-link {
  display: inline-flex;
  align-items: center;
  gap: 0.25rem;
  font-size: var(--fz-sm);
  font-weight: 500;
  color: var(--slate);
}

.social-link:hover {
  color: var(--green);
}

.social-link .external {
  font-size: 0.75em;
  opacity: 0.7;
  transition: transform var(--transition);
}

.social-link:hover .external {
  transform: translate(1px, -2px);
}

/* Main */
.main {
  min-width: 0;
}

.content-section {
  scroll-margin-top: calc(var(--header-offset) - 2rem);
  padding-bottom: 5rem;
}

.content-section:last-of-type {
  padding-bottom: 3rem;
}

.section-heading {
  position: sticky;
  top: 0;
  z-index: 10;
  margin: 0 0 1.25rem;
  padding: 0.5rem 0 1rem;
  font-size: var(--fz-xs);
  font-weight: 700;
  letter-spacing: 0.16em;
  text-transform: uppercase;
  color: var(--lightest-slate);
  background: linear-gradient(to bottom, var(--navy) 60%, transparent);
  -webkit-backdrop-filter: blur(8px);
  backdrop-filter: blur(8px);
}

.section-heading-text {
  display: inline-block;
  padding-bottom: 0.25rem;
  border-bottom: 1px solid color-mix(in srgb, var(--lightest-slate) 35%, transparent);
}

.lead {
```

```
  margin: 0 0 1rem;
  font-size: var(--fz-md);
  color: var(--slate);
}

.prose p {
  margin: 0 0 1.15rem;
  font-size: var(--fz-md);
  color: var(--slate);
}

.prose p:last-child {
  margin-bottom: 0;
}

.prose a {
  color: var(--green);
  text-decoration: underline;
  text-decoration-color: color-mix(in srgb, var(--green) 45%, transparent);
  text-underline-offset: 3px;
}

.prose a:hover {
  color: var(--lightest-slate);
}

/* Profile */
.profile-list {
  list-style: none;
  margin: 0;
  padding: 0;
  display: grid;
  gap: 0.65rem;
}

.profile-list li {
  display: grid;
  grid-template-columns: 7rem 1fr;
  gap: 1rem;
  font-size: var(--fz-sm);
  padding: 0.5rem 0;
  border-bottom: 1px solid color-mix(in srgb, var(--slate) 22%, transparent);
}

.profile-list li:last-child {
  border-bottom: none;
}

.profile-k {
  font-weight: 600;
  color: var(--lightest-slate);
  font-size: var(--fz-xs);
  letter-spacing: 0.06em;
  text-transform: uppercase;
}

.profile-v {
  color: var(--slate);
}

/* Tags */
.tag-cloud {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  flex-wrap: wrap;
```

```
  gap: 0.5rem;
}

.tag-cloud li {
  padding: 0.35rem 0.75rem;
  font-size: var(--fz-xs);
  font-weight: 600;
  letter-spacing: 0.04em;
  color: var(--green);
  background: var(--green-dim);
  border-radius: 999px;
}

/* Timeline — Experience style */
.timeline {
  list-style: none;
  margin: 0;
  padding: 0;
}

.timeline-item {
  position: relative;
  display: grid;
  grid-template-columns: minmax(100px, 140px) 1fr;
  gap: 1.25rem 1.5rem;
  padding: 1.5rem 0;
  border-bottom: 1px solid color-mix(in srgb, var(--slate) 18%, transparent);
}

.timeline-item:last-child {
  border-bottom: none;
  padding-bottom: 0;
}

.timeline-meta {
  font-size: var(--fz-xs);
  font-weight: 600;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  color: var(--slate);
  padding-top: 0.2rem;
}

.timeline-title {
  margin: 0 0 0.5rem;
  font-size: var(--fz-md);
  font-weight: 600;
  color: var(--lightest-slate);
  letter-spacing: -0.02em;
}

.timeline-desc {
  margin: 0 0 0.85rem;
  font-size: var(--fz-sm);
  line-height: 1.65;
  color: var(--slate);
}

.tech-tags {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  flex-wrap: wrap;
  gap: 0.4rem;
}

```

```
.tech-tags li {
  padding: 0.2rem 0.55rem;
  font-size: 0.7rem;
  font-weight: 500;
  color: var(--green);
  border: 1px solid color-mix(in srgb, var(--green) 35%, transparent);
  border-radius: var(--radius);
}

/* Motto */
.motto {
  margin: 0;
  padding: 0;
  border: none;
}

.motto-line {
  margin: 0 0 0.5rem;
  font-size: var(--fz-lg);
  font-weight: 700;
  color: var(--lightest-slate);
  letter-spacing: -0.02em;
}

.motto-sub {
  margin: 0;
  font-size: var(--fz-md);
  color: var(--slate);
  line-height: 1.75;
}

/* Footer */
.site-footer {
  padding-top: 2rem;
  border-top: 1px solid color-mix(in srgb, var(--slate) 18%, transparent);
}

.site-footer__inner {
  width: 100%;
}

.site-footer__note {
  margin: 0;
  font-size: 1.625rem;
  line-height: 1.6;
  text-align: center;
  color: color-mix(in srgb, var(--slate) 85%, transparent);
}

.footer-note {
  margin: 0;
  max-width: 42ch;
  font-size: var(--fz-xs);
  line-height: 1.6;
  color: color-mix(in srgb, var(--slate) 85%, transparent);
}

.to-top {
  flex-shrink: 0;
  width: 40px;
  height: 40px;
  font-size: 1rem;
  color: var(--slate);
  background: transparent;
  border: 1px solid color-mix(in srgb, var(--slate) 35%, transparent);
  border-radius: var(--radius);
  cursor: pointer;
```

```
  transition: color var(--transition), border-color var(--transition), background var(--transition);
}

.to-top:hover {
  color: var(--green);
  border-color: var(--green);
  background: var(--green-dim);
}

/* Responsive */
@media (max-width: 1024px) {
  .layout {
    grid-template-columns: 1fr;
    padding: 5rem 1.5rem 5rem;
    gap: 2rem;
  }

  .sidebar {
    position: relative;
    top: 0;
    padding-bottom: 1rem;
    border-bottom: 1px solid color-mix(in srgb, var(--slate) 15%, transparent);
  }

  .brand-tagline {
    max-width: none;
  }

  .profile-figure {
    align-self: center;
  }

  .side-nav-list {
    display: flex;
    flex-wrap: wrap;
    gap: 0.35rem 1rem;
    padding-top: 0.5rem;
  }

  .nav-link {
    padding: 0.4rem 0;
  }

  .nav-marker {
    display: none;
  }

  .timeline-item {
    grid-template-columns: 1fr;
    gap: 0.35rem;
  }

  .timeline-meta {
    padding-top: 0;
  }
}

@media (max-width: 480px) {
  .profile-list li {
    grid-template-columns: 1fr;
    gap: 0.25rem;
  }

  .layout {
    padding-left: 1.25rem;
    padding-right: 1.25rem;
```

```
593     }
594   }
```

cover_letter/js/script.js

```
/**
 * 자기소개서 페이지에서 현재 보고 있는 절에 맞춰 사이드 메뉴를 표시하고, 맨 위로 버튼을 연결한다.
 */
(function () {
  "use strict";

  /**
   * data-section이 id와 같은 메뉴 링크에만 활성 클래스를 붙인다.
   * @param {string} id 현재 화면에 들어온 절의 id
   */
  function setActiveNav(id) {
    document.querySelectorAll(".nav-link").forEach(function (link) {
      var sec = link.getAttribute("data-section");
      if (sec === id) {
        link.classList.add("is-active");
      } else {
        link.classList.remove("is-active");
      }
    });
  }

  /**
   * 본문 절이 화면의 일정 비율 이상 보이면 해당 메뉴를 활성으로 바꾼다.
   * 관찰이 시작되기 전에는 첫 절을 활성으로 둔다.
   */
  function initScrollSpy() {
    var sections = document.querySelectorAll(".content-section[id]");
    if (!sections.length) return;

    var observer = new IntersectionObserver(
      function (entries) {
        entries.forEach(function (entry) {
          if (entry.isIntersecting && entry.intersectionRatio >= 0.25) {
            setActiveNav(entry.target.id);
          }
        });
      },
      {
        root: null,
        rootMargin: "-20% 0px -55% 0px",
        threshold: [0, 0.25, 0.5, 0.75, 1]
      }
    );

    sections.forEach(function (section) {
      observer.observe(section);
    });

    /* 첫 화면 */
    var first = sections[0];
    if (first && first.id) setActiveNav(first.id);
  }

  /**
   * 맨 위로 버튼을 누르면 페이지 최상단으로 부드럽게 스크롤한다.
   */
  function initToTop() {
    var btn = document.getElementById("toTop");
    if (!btn) return;
    btn.addEventListener("click", function () {
      window.scrollTo({ top: 0, behavior: "smooth" });
    });
  }
```

```
64
65    document.addEventListener("DOMContentLoaded", function () {
66      initScrollSpy();
67      initToTop();
68    });
69  })();
```

cover_letter/profile.html

```
<!-- 자기소개 본문 화면. 절마다 사이드 메뉴 활성 표시와 맨 위로 이동을 쓴다. -->
<!DOCTYPE html>
<html lang="ko" data-default-theme="light">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>김신욱 · 자기소개</title>
  <script>
    (function () {
      try {
        var theme = localStorage.getItem("portfolio-theme-v2");
        if (theme === "dark" || theme === "light") {
          document.documentElement.setAttribute("data-theme", theme);
        }
      } catch (e) {
        /* 테마 저장소를 사용할 수 없는 환경에서는 기본 테마를 사용합니다. */
      }
    })();
  </script>
  <link rel="stylesheet" href="css/styles.css?v=theme-sync" />
  <link rel="stylesheet" href="../css/site-header.css?v=theme-sync" />
</head>
<body>
  <a class="skip-link" href="#main">본문으로 건너뛰기</a>

  <header class="site-header">
    <div class="site-header__inner">
      <a class="site-header__brand" href="../index.html">코탐록</a>
      <nav id="site-nav" class="site-nav" aria-label="자료 페이지">
        <ul class="site-nav__list">
          <li><a class="site-nav__link is-active" href="../cover_letter/profile.html" aria-current="page">자기소개 / 프로필</a></li>
          <li><a class="site-nav__link" href="../online_business_card/business-card.html">온라인 명함</a></li>
          <li><a class="site-nav__link" href="../cv/resume.html">이력서</a></li>
          <li><a class="site-nav__link" href="../study_note_record_organization/dashboard/index.html">학습록</a></li>
        </ul>
      </nav>
      <div class="site-header__actions">
        <button type="button" class="theme-toggle" data-theme-toggle aria-pressed="false" aria-label="다크 테마로 전환">
          <span class="theme-toggle__label">고대비</span>
          <span class="theme-toggle__state" data-theme-label>라이트</span>
        </button>
        <button
          type="button"
          class="nav-toggle"
          aria-expanded="false"
          aria-controls="site-nav"
          aria-label="메뉴 열기"
        >
          <span class="nav-toggle__bar" aria-hidden="true"></span>
          <span class="nav-toggle__bar" aria-hidden="true"></span>
        </button>
      </div>
    </div>
  </header>

  <div class="layout">
    <header class="sidebar" aria-label="소개 및 탐색">
      <div class="sidebar-inner">
        <div class="brand">
          <figure class="profile-figure">
            <div class="profile-photo-ring">
```

```
                <img
                  class="profile-photo"
                  src="P.png"
                  alt="김신욱 프로필 사진"
                  width="130"
                  height="130"
                  decoding="async"
                  fetchpriority="high"
                />
              </div>
            </figure>
            <h1 class="brand-name">김신욱</h1>
            <p class="brand-role">웹 · 모바일 · AI 서비스</p>
            <p class="brand-tagline">
              접근 가능하고 정돈된 디지털 경험을 지향하며, IT 분야 취업을 준비하고 있습니다.
            </p>
          </div>

          <nav class="side-nav" aria-label="섹션 이동">
            <ul class="side-nav-list" id="navList">
              <li>
                <a class="nav-link" href="#about" data-section="about"><span class="nav-marker" aria-hidden="true"></span>소개</a>
              </li>
              <li>
                <a class="nav-link" href="#profile" data-section="profile"><span class="nav-marker" aria-hidden="true"></span>프로필</a>
              </li>
              <li>
                <a class="nav-link" href="#interests" data-section="interests"><span class="nav-marker" aria-hidden="true"></span>관심 분야</a>
              </li>
              <li>
                <a class="nav-link" href="#learning" data-section="learning"><span class="nav-marker" aria-hidden="true"></span>학습 · 경험</a>
              </li>
              <li>
                <a class="nav-link" href="#motivation" data-section="motivation"><span class="nav-marker" aria-hidden="true"></span>지원 동기</a>
              </li>
              <li>
                <a class="nav-link" href="#motto" data-section="motto"><span class="nav-marker" aria-hidden="true"></span>생활신조</a>
              </li>
            </ul>
          </nav>

          <ul class="social-list" aria-label="외부 링크">
            <li>
              <a class="social-link" href="https://blog.naver.com/fifo23" target="_blank" rel="noopener noreferrer">
                Blog<span class="external" aria-hidden="true">↗</span>
              </a>
            </li>
          </ul>

        </div>
      </header>

      <main id="main" class="main">
        <section id="about" class="content-section" aria-labelledby="heading-about">
          <h2 id="heading-about" class="section-heading">
            <span class="section-heading-text">소개</span>
          </h2>
          <div class="prose">
            <p>
              4년제 응용통계학과(SW응용통계학)를 졸업한 뒤 IT 분야를 꾸준히 학습해 왔고,
              <a href="https://blog.naver.com/fifo23" target="_blank" rel="noopener noreferrer">코탐록 네이버 블로그</a>를
```

```
운영할 예정이며 취업 준비를 이어가고 있습니다.
              웹·모바일 개발과 AI·IT 전반에 관심이 많습니다.
            </p>
          </div>
        </section>

        <section id="profile" class="content-section" aria-labelledby="heading-profile">
          <h2 id="heading-profile" class="section-heading">
            <span class="section-heading-text">프로필</span>
          </h2>
          <ul class="profile-list">
            <li><span class="profile-k">성별</span><span class="profile-v">남성</span></li>
            <li><span class="profile-k">나이</span><span class="profile-v">30대 후반</span></li>
            <li><span class="profile-k">학력</span><span class="profile-v">대졸, 학점은행제(멀티미디어학) 도전 중</span></li>
            <li><span class="profile-k">목표</span><span class="profile-v">취업(이력서·자소서), AI 서비스를 제대로 활용하기</span></li>
            <li><span class="profile-k">취미</span><span class="profile-v">프로그래밍(주로 책 위주 학습)</span></li>
            <li><span class="profile-k">온라인 학습</span><span class="profile-v">패스트캠퍼스, 코드잇, 인프런, 유데미, KMOOC, 경기지식 등</span></li>
            <li><span class="profile-k">성장 과제</span><span class="profile-v">사교성, 도전정신 – 꾸준히 보완 중입니다.</span></li>
          </ul>
        </section>

        <section id="interests" class="content-section" aria-labelledby="heading-interests">
          <h2 id="heading-interests" class="section-heading">
            <span class="section-heading-text">관심 분야</span>
          </h2>
          <p class="lead">모바일, 웹, AI 서비스, IT/ICT 전반에 폭넓게 관심을 두고 있습니다.</p>
          <ul class="tag-cloud">
            <li>모바일</li>
            <li>웹</li>
            <li>AI 서비스</li>
            <li>IT / ICT</li>
            <li>멀티미디어</li>
          </ul>
        </section>

        <section id="learning" class="content-section" aria-labelledby="heading-learning">
          <h2 id="heading-learning" class="section-heading">
            <span class="section-heading-text">학습 · 경험</span>
          </h2>
          <ol class="timeline">
            <li class="timeline-item">
              <div class="timeline-meta">기초 · 통계</div>
              <div class="timeline-body">
                <h3 class="timeline-title">비주얼베이직, SAS · SPSS</h3>
                <p class="timeline-desc">통계·분석 도구와 프로그래밍 입문 경험.</p>
                <ul class="tech-tags">
                  <li>VB</li>
                  <li>SAS</li>
                  <li>SPSS</li>
                </ul>
              </div>
            </li>
            <li class="timeline-item">
              <div class="timeline-meta">앱 · 도구</div>
              <div class="timeline-body">
                <h3 class="timeline-title">자바 · 코틀린 · .NET</h3>
                <p class="timeline-desc">
                  자바(GUI, AWT, SWING) 로또 게임, 코틀린 XML 레이아웃 두더지 게임, SWING 스톱워치, 레인미터 정리, Visual .NET 텍스트 합치기 등.
                </p>
                <ul class="tech-tags">
                  <li>Java</li>
                  <li>Kotlin</li>
```

```
                <li>Android XML</li>
                <li>Visual .NET</li>
              </ul>
            </div>
          </li>
          <li class="timeline-item">
            <div class="timeline-meta">최근</div>
            <div class="timeline-body">
              <h3 class="timeline-title">AI · 생산성</h3>
              <p class="timeline-desc">
                생성형 AI로 소규모 프로젝트, 바이브 코딩·프롬프트 엔지니어링 학습, 펠로AI·젠스파크로 필기 슬라이드 정리 등.
              </p>
              <ul class="tech-tags">
                <li>Generative AI</li>
                <li>Prompt</li>
              </ul>
            </div>
          </li>
        </ol>
      </section>

      <section id="motivation" class="content-section" aria-labelledby="heading-motivation">
        <h2 id="heading-motivation" class="section-heading">
          <span class="section-heading-text">지원 동기 및 포부</span>
        </h2>
        <div class="prose">
          <p>
            IT 관련 개발직을 목표로 하며, 웹·모바일뿐 아니라 다른 IT 기술과 AI에도 관심이 많습니다.
            IT기술직·현장직, 웹·앱 개발자로 일하고 싶습니다.
          </p>
          <p>
            최근 AI 기술에 집중해 학습하고 있으며, 백업·정리된 학습 노트와 프로젝트를 바탕으로 이력서와 자소서를 다듬어 지원할 계획입니다.
            대학 졸업 이후 공백이 있었으나 하루 최소 약 3시간은 IT에 매진해 왔고, 일상에서도 IT 활용에 관심이 많습니다.
          </p>
          <p>
            개인만으로는 어려운 서비스를 팀원과 협업해 만들고, 올바른 가이드라인을 지키며 프로젝트에 참여하고 싶습니다. IT 분야 취업 후 기술 발전에 기여하고자 합니다.
          </p>
        </div>
      </section>

      <section id="motto" class="content-section" aria-labelledby="heading-motto">
        <h2 id="heading-motto" class="section-heading">
          <span class="section-heading-text">생활신조</span>
        </h2>
        <blockquote class="motto">
          <p class="motto-line">필사즉생</p>
          <p class="motto-sub">안 되는 것을 꾸준히 고민하며, 될 때까지 해내는 것이 제 신조입니다.</p>
        </blockquote>
      </section>

      <footer class="site-footer">
        <div class="site-footer__inner">
          <p class="site-footer__note">
            2026년 5월 25일 제작 · Cursor AI를 사용해 제작했습니다.
          </p>
        </div>
      </footer>
    </main>
  </div>

  <script src="js/script.js"></script>
  <script src="../js/main.js?v=theme-sync"></script>
</body>
</html>
```

css/site-header.css

```css
/* 여러 페이지가 공유하는 상단 헤더, 내비게이션, 자료 드롭다운, 모바일 메뉴의 배치와 색을 정한다. */
:root {
  --site-header-canvas: #0c0c0c;
  --site-header-text: #f5f5f5;
  --site-header-muted: rgba(245, 245, 245, 0.66);
  --site-header-border: rgba(255, 255, 255, 0.1);
  --site-header-border-hover: rgba(255, 255, 255, 0.16);
  --site-header-surface: #161616;
  --site-header-surface-hover: #1f1f1f;
  --site-header-bg: rgba(12, 12, 12, 0.92);
  --site-header-focus: 2px solid rgba(245, 245, 245, 0.85);
  --site-header-gutter: clamp(1.25rem, 5vw, 2.5rem);
  --site-header-height: 4rem;
  --site-header-motion: 180ms;
  --site-header-ease: cubic-bezier(0.22, 1, 0.36, 1);
}

html[data-theme="light"] {
  --site-header-canvas: #f8fafc;
  --site-header-text: #0f172a;
  --site-header-muted: rgba(15, 23, 42, 0.66);
  --site-header-border: rgba(15, 23, 42, 0.12);
  --site-header-border-hover: rgba(15, 23, 42, 0.22);
  --site-header-surface: #ffffff;
  --site-header-surface-hover: #f1f5f9;
  --site-header-bg: rgba(248, 250, 252, 0.92);
  --site-header-focus: 2px solid rgba(15, 23, 42, 0.85);
}

.site-header *,
.site-header *::before,
.site-header *::after {
  box-sizing: border-box;
}

.site-header {
  position: sticky;
  top: 0;
  z-index: 1000;
  color: var(--site-header-text);
  background: var(--site-header-bg);
  border-bottom: 1px solid var(--site-header-border);
  backdrop-filter: blur(10px);
  font-family: ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, "Noto Sans KR", "Apple SD Gothic Neo",
    "Malgun Gothic", sans-serif;
}

.site-header__inner {
  max-width: 1200px;
  min-height: var(--site-header-height);
  margin: 0 auto;
  padding: 0 var(--site-header-gutter);
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}

.site-header__brand {
  color: var(--site-header-text);
  font-size: 1.0625rem;
  font-weight: 600;
  line-height: 1.5;
```

```
  letter-spacing: -0.02em;
  text-decoration: none;
}

.site-header__brand:hover {
  text-decoration: underline;
  text-underline-offset: 0.2em;
}

.site-header__brand:focus-visible,
.site-nav__link:focus-visible,
.theme-toggle:focus-visible,
.nav-toggle:focus-visible {
  outline: var(--site-header-focus);
  outline-offset: 2px;
}

.site-header__actions {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.theme-toggle {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  min-height: 2rem;
  padding: 0 0.65rem;
  color: var(--site-header-text);
  background: var(--site-header-surface);
  border: 1px solid var(--site-header-border);
  border-radius: 999px;
  font: inherit;
  font-size: 0.8125rem;
  font-weight: 700;
  cursor: pointer;
  transition: border-color var(--site-header-motion) var(--site-header-ease), background-color var(--site-header-motio
n) var(--site-header-ease);
}

.theme-toggle:hover {
  border-color: var(--site-header-border-hover);
  background: var(--site-header-surface-hover);
}

.theme-toggle__label {
  color: var(--site-header-muted);
}

.theme-toggle__state {
  color: var(--site-header-text);
}

.nav-toggle {
  display: none;
  flex-direction: column;
  justify-content: center;
  gap: 6px;
  width: 2.5rem;
  height: 2.5rem;
  padding: 0;
  border: 1px solid var(--site-header-border);
  border-radius: 2px;
  background: var(--site-header-surface);
  cursor: pointer;
}
```

```
130
131 .nav-toggle:hover {
132   border-color: var(--site-header-border-hover);
133 }
134
135 .nav-toggle__bar {
136   display: block;
137   width: 1.125rem;
138   height: 1px;
139   margin: 0 auto;
140   background: var(--site-header-text);
141 }
142
143 .site-nav__list {
144   display: flex;
145   flex-wrap: wrap;
146   gap: 0.25rem 1.25rem;
147   list-style: none;
148   margin: 0;
149   padding: 0;
150 }
151
152 .site-nav__link {
153   display: inline-flex;
154   align-items: center;
155   min-height: 2rem;
156   color: var(--site-header-muted);
157   font-size: 0.9375rem;
158   line-height: 1.5;
159   text-decoration: none;
160   transition: color var(--site-header-motion) var(--site-header-ease);
161 }
162
163 .site-nav__link:hover,
164 .site-nav__link.is-active {
165   color: var(--site-header-text);
166 }
167
168 .site-nav__link:hover {
169   text-decoration: underline;
170   text-underline-offset: 0.2em;
171 }
172
173 @media (max-width: 768px) {
174   .nav-toggle {
175     display: flex;
176   }
177
178   .site-nav {
179     position: absolute;
180     left: 0;
181     right: 0;
182     top: 100%;
183     z-index: 1000;
184     max-height: 0;
185     overflow: hidden;
186     opacity: 0;
187     pointer-events: none;
188     background: var(--site-header-canvas);
189     border-bottom: 1px solid var(--site-header-border);
190     transition: max-height var(--site-header-motion) var(--site-header-ease), opacity var(--site-header-motion) var(--
site-header-ease);
191   }
192
193   .site-nav.is-open {
194     max-height: 18rem;
195     opacity: 1;
```

```
    pointer-events: auto;
  }

  .site-nav__list {
    flex-direction: column;
    padding: 1rem var(--site-header-gutter) 1.25rem;
    gap: 0.5rem;
  }

  .site-header__inner {
    position: relative;
  }
}

@media (min-width: 769px) {
  .site-nav {
    position: static;
    max-height: none !important;
    opacity: 1 !important;
    pointer-events: auto !important;
    overflow: visible !important;
  }
}
```

css/styles.css

```
/* 메인 페이지의 색, 글자 크기, 섹션, 라이트.다크 테마를 정한다. */
:root {
  --color-canvas: #0c0c0c;
  --color-surface: #161616;
  --color-surface-elevated: #1a1a1a;
  --color-text: #f5f5f5;
  --color-text-muted: rgba(245, 245, 245, 0.66);
  --color-border: rgba(255, 255, 255, 0.1);
  --color-border-hover: rgba(255, 255, 255, 0.14);
  --color-header-bg: rgba(12, 12, 12, 0.92);
  --focus-ring: 2px solid rgba(245, 245, 245, 0.85);
  --focus-ring-offset: 2px;

  --font-ui: ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, "Noto Sans KR", "Apple SD Gothic Neo",
    "Malgun Gothic", sans-serif;

  --text-hero: clamp(2rem, 4.5vw, 3rem);
  --lh-hero: 1.1;
  --text-section: clamp(1.25rem, 2vw, 1.5rem);
  --text-body: 1.0625rem;
  --lh-body: 1.6;
  --text-caption: 0.8125rem;
  --lh-caption: 1.5;

  --space-section: clamp(4rem, 10vw, 6.5rem);
  --space-gutter: clamp(1.25rem, 5vw, 2.5rem);
  --prose-max: 70ch;

  --header-h: 4rem;
  --motion-fast: 180ms;
  --ease-out-soft: cubic-bezier(0.22, 1, 0.36, 1);
}

html[data-theme="light"] {
  --color-canvas: #f8fafc;
  --color-surface: #ffffff;
  --color-surface-elevated: #f1f5f9;
  --color-text: #0f172a;
  --color-text-muted: rgba(15, 23, 42, 0.68);
  --color-border: rgba(15, 23, 42, 0.12);
  --color-border-hover: rgba(15, 23, 42, 0.2);
  --color-header-bg: rgba(248, 250, 252, 0.92);
  --focus-ring: 2px solid rgba(15, 23, 42, 0.85);
}

*,
*::before,
*::after {
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}

@media (prefers-reduced-motion: reduce) {
  html {
    scroll-behavior: auto;
  }
}

body {
  margin: 0;
```

```
  position: relative;
  font-family: var(--font-ui);
  font-size: var(--text-body);
  line-height: var(--lh-body);
  color: var(--color-text);
  background-color: var(--color-canvas);
}

@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}

img {
  max-width: 100%;
  height: auto;
}

a {
  color: inherit;
}

.skip-link {
  position: absolute;
  left: var(--space-gutter);
  top: 0.75rem;
  z-index: 100;
  padding: 0.5rem 0.875rem;
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: 2px;
  font-size: 0.875rem;
  text-decoration: none;
  transform: translateY(-120%);
  transition: transform var(--motion-fast) var(--ease-out-soft);
}

.skip-link:focus {
  outline: var(--focus-ring);
  outline-offset: var(--focus-ring-offset);
}

.skip-link:focus-visible {
  transform: translateY(0);
}

.site-header {
  position: sticky;
  top: 0;
  z-index: 50;
  border-bottom: 1px solid var(--color-border);
  background-color: var(--color-header-bg);
  backdrop-filter: blur(10px);
}

.site-header__inner {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 var(--space-gutter);
  min-height: var(--header-h);
  display: flex;
```

```
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}

.site-header__brand {
  font-weight: 600;
  letter-spacing: -0.02em;
  text-decoration: none;
}

.site-header__brand:hover {
  text-decoration: underline;
  text-underline-offset: 0.2em;
}

.site-header__brand:focus-visible {
  outline: var(--focus-ring);
  outline-offset: var(--focus-ring-offset);
}

.site-header__actions {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.theme-toggle {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  min-height: 2rem;
  padding: 0 0.65rem;
  color: var(--color-text);
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: 999px;
  font: inherit;
  font-size: 0.8125rem;
  font-weight: 700;
  cursor: pointer;
  transition: border-color var(--motion-fast) var(--ease-out-soft), background-color var(--motion-fast) var(--ease-out
-soft);
}

.theme-toggle:hover {
  border-color: var(--color-border-hover);
  background: var(--color-surface-elevated);
}

.theme-toggle:focus-visible {
  outline: var(--focus-ring);
  outline-offset: var(--focus-ring-offset);
}

.theme-toggle__label {
  color: var(--color-text-muted);
}

.theme-toggle__state {
  color: var(--color-text);
}

.nav-toggle {
  display: none;
  flex-direction: column;
  justify-content: center;
```

```
  gap: 6px;
  width: 2.5rem;
  height: 2.5rem;
  padding: 0;
  border: 1px solid var(--color-border);
  border-radius: 2px;
  background: var(--color-surface);
  cursor: pointer;
}

.nav-toggle:hover {
  border-color: var(--color-border-hover);
}

.nav-toggle:focus-visible {
  outline: var(--focus-ring);
  outline-offset: var(--focus-ring-offset);
}

.nav-toggle__bar {
  display: block;
  height: 1px;
  width: 1.125rem;
  margin: 0 auto;
  background: var(--color-text);
}

.site-nav__list {
  display: flex;
  flex-wrap: wrap;
  gap: 0.25rem 1.25rem;
  list-style: none;
  margin: 0;
  padding: 0;
}

.site-nav__item {
  position: relative;
}

.site-nav__link {
  font-size: 0.9375rem;
  color: var(--color-text-muted);
  text-decoration: none;
  transition: color var(--motion-fast) var(--ease-out-soft);
}

.site-nav__dropdown-toggle {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  padding: 0;
  border: 0;
  background: transparent;
  font: inherit;
  cursor: pointer;
}

.site-nav__dropdown-caret {
  width: 0.45rem;
  height: 0.45rem;
  border-right: 1px solid currentColor;
  border-bottom: 1px solid currentColor;
  transform: translateY(-0.12rem) rotate(45deg);
  transition: transform var(--motion-fast) var(--ease-out-soft);
}

```

```
.site-nav__dropdown-toggle[aria-expanded="true"] .site-nav__dropdown-caret {
  transform: translateY(0.08rem) rotate(225deg);
}

.site-nav__link:hover {
  color: var(--color-text);
  text-decoration: underline;
  text-underline-offset: 0.2em;
}

.site-nav__link:focus-visible {
  outline: var(--focus-ring);
  outline-offset: var(--focus-ring-offset);
  color: var(--color-text);
}

.site-nav__dropdown-menu {
  position: absolute;
  right: 0;
  top: calc(100% + 0.8rem);
  z-index: 60;
  min-width: 18rem;
  padding: 0.45rem;
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: 2px;
  box-shadow: 0 24px 60px rgba(0, 0, 0, 0.35);
  opacity: 0;
  pointer-events: none;
  transform: translateY(-0.4rem);
  transition: opacity var(--motion-fast) var(--ease-out-soft), transform var(--motion-fast) var(--ease-out-soft);
}

.site-nav__dropdown-menu::before {
  content: "";
  position: absolute;
  right: 1.1rem;
  top: -0.35rem;
  width: 0.7rem;
  height: 0.7rem;
  background: var(--color-surface);
  border-left: 1px solid var(--color-border);
  border-top: 1px solid var(--color-border);
  transform: rotate(45deg);
}

.site-nav__dropdown.is-open .site-nav__dropdown-menu {
  opacity: 1;
  pointer-events: auto;
  transform: translateY(0);
}

.site-nav__dropdown-link {
  position: relative;
  z-index: 1;
  display: block;
  padding: 0.8rem 0.85rem;
  color: var(--color-text);
  text-decoration: none;
  border-radius: 2px;
  transition: background-color var(--motion-fast) var(--ease-out-soft);
}

.site-nav__dropdown-link:hover,
.site-nav__dropdown-link:focus-visible {
  background: var(--color-surface-elevated);
  outline: none;
```

```
}

.site-nav__dropdown-title {
  display: block;
  font-size: 0.9375rem;
  font-weight: 600;
  line-height: 1.35;
}

.site-nav__dropdown-desc {
  display: block;
  margin-top: 0.15rem;
  color: var(--color-text-muted);
  font-size: 0.8125rem;
  line-height: 1.45;
}

@media (max-width: 768px) {
  .site-header {
    position: sticky;
  }

  .nav-toggle {
    display: flex;
  }

  .site-nav {
    position: absolute;
    left: 0;
    right: 0;
    top: 100%;
    z-index: 40;
    border-bottom: 1px solid var(--color-border);
    background: var(--color-canvas);
    max-height: 0;
    overflow: hidden;
    opacity: 0;
    pointer-events: none;
    transition: max-height var(--motion-fast) var(--ease-out-soft), opacity var(--motion-fast) var(--ease-out-soft);
  }

  .site-nav.is-open {
    max-height: 34rem;
    opacity: 1;
    pointer-events: auto;
  }

  .site-nav__list {
    flex-direction: column;
    padding: 1rem var(--space-gutter) 1.25rem;
    gap: 0.5rem;
  }

  .site-nav__dropdown-toggle {
    width: 100%;
    justify-content: space-between;
  }

  .site-nav__dropdown-menu {
    position: static;
    min-width: 0;
    margin-top: 0.55rem;
    padding: 0.35rem;
    box-shadow: none;
    opacity: 1;
    pointer-events: auto;
    transform: none;
```

```
    display: none;
  }

  .site-nav__dropdown-menu::before {
    display: none;
  }

  .site-nav__dropdown.is-open .site-nav__dropdown-menu {
    display: block;
  }

  .site-header__inner {
    position: relative;
  }
}

@media (min-width: 769px) {
  .site-nav {
    position: static;
    max-height: none !important;
    opacity: 1 !important;
    pointer-events: auto !important;
    border-bottom: none;
    background: transparent;
    overflow: visible !important;
  }
}

.hero {
  padding: calc(var(--space-section) * 0.85) var(--space-gutter) var(--space-section);
  border-bottom: 1px solid var(--color-border);
}

.hero__inner {
  max-width: 1200px;
  margin: 0 auto;
}

.hero__layout {
  display: grid;
  gap: clamp(2rem, 4vw, 3.25rem);
  align-items: end;
}

@media (max-width: 959px) {
  .hero__layout {
    align-items: start;
  }
}

@media (min-width: 960px) {
  .hero__layout {
    grid-template-columns: minmax(0, 1fr) minmax(0, 0.92fr);
  }
}

.hero__copy .hero__title {
  max-width: 22ch;
}

.hero__figure {
  margin: 0;
}

.media-frame {
  margin: 0;
}
```

```
.media-frame img {
  display: block;
  width: 100%;
  border: 1px solid var(--color-border);
  border-radius: 2px;
  object-fit: cover;
  aspect-ratio: 3 / 2;
  background: var(--color-surface);
}

.media-caption {
  margin: 0.55rem 0 0;
  font-size: var(--text-caption);
  line-height: var(--lh-caption);
  color: var(--color-text-muted);
}

.media-caption a {
  color: var(--color-text-muted);
  text-decoration: none;
}

.media-caption a:hover {
  color: var(--color-text);
  text-decoration: underline;
  text-underline-offset: 0.2em;
}

.media-caption a:focus-visible {
  outline: var(--focus-ring);
  outline-offset: var(--focus-ring-offset);
}

.section-figure {
  margin: 0 0 1.75rem;
}

.prose .section-figure {
  max-width: none;
}

.hero__eyebrow {
  margin: 0 0 1rem;
  font-size: var(--text-caption);
  line-height: var(--lh-caption);
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: var(--color-text-muted);
}

.hero__title {
  margin: 0 0 2rem;
  max-width: 18ch;
  font-size: var(--text-hero);
  line-height: var(--lh-hero);
  font-weight: 600;
  letter-spacing: -0.03em;
}

.hero__meta {
  margin: 0;
  max-width: var(--prose-max);
  padding: 1.25rem 0 0;
  border-top: 1px solid var(--color-border);
}

```

```
.hero__meta-row {
  display: grid;
  grid-template-columns: minmax(5rem, 8rem) 1fr;
  gap: 0.5rem 1.5rem;
  padding: 0.65rem 0;
  border-bottom: 1px solid var(--color-border);
}

.hero__meta-row:last-child {
  border-bottom: none;
}

.hero__meta dt {
  margin: 0;
  font-size: 0.875rem;
  color: var(--color-text-muted);
  font-weight: 500;
}

.hero__meta dd {
  margin: 0;
  font-size: 0.9375rem;
  color: var(--color-text);
}

@media (max-width: 420px) {
  .hero__meta-row {
    grid-template-columns: 1fr;
  }
}

.section:focus {
  outline: none;
}

.section-stack {
  background: var(--color-canvas);
}

.section {
  padding: var(--space-section) var(--space-gutter);
  border-bottom: 1px solid var(--color-border);
}

.section__inner {
  max-width: 1200px;
  margin: 0 auto;
}

.section__title {
  margin: 0 0 1.5rem;
  font-size: var(--text-section);
  line-height: 1.2;
  font-weight: 600;
  letter-spacing: -0.02em;
}

.section__title--flush {
  margin-bottom: 1.75rem;
}

.prose {
  max-width: var(--prose-max);
}

.prose p {
  margin: 0 0 1.25rem;
```

```
  color: var(--color-text);
}

.prose p:last-child {
  margin-bottom: 0;
}

.prose strong {
  font-weight: 600;
  color: var(--color-text);
}

.prose--after-cards {
  margin-top: 2rem;
}

.closing-thanks {
  margin-top: 1.5rem;
  font-weight: 500;
}

.skill-cards {
  list-style: none;
  margin: 0;
  padding: 0;
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 1rem 1.25rem;
}

@media (max-width: 640px) {
  .skill-cards {
    grid-template-columns: 1fr;
  }
}

.skill-card {
  margin: 0;
  padding: 1.25rem 1.35rem;
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: 2px;
  transition: border-color var(--motion-fast) var(--ease-out-soft), transform var(--motion-fast) var(--ease-out-soft);
}

.skill-card:hover {
  border-color: var(--color-border-hover);
  transform: translateY(-6px);
}

@media (prefers-reduced-motion: reduce) {
  .skill-card:hover {
    transform: none;
  }
}

.skill-card__label {
  margin: 0 0 0.5rem;
  font-size: 0.8125rem;
  font-weight: 600;
  letter-spacing: 0.04em;
  text-transform: uppercase;
  color: var(--color-text-muted);
}

.skill-card__body {
  margin: 0;
```

```
  font-size: 0.9375rem;
  line-height: var(--lh-body);
  color: var(--color-text);
}

.section-lead {
  margin: 0 0 1.75rem;
  max-width: var(--prose-max);
  color: var(--color-text-muted);
}

.resource-cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(16rem, 1fr));
  gap: 1rem 1.25rem;
}

@media (max-width: 860px) {
  .resource-cards {
    grid-template-columns: 1fr;
  }
}

.resource-card {
  display: flex;
  flex-direction: column;
  min-height: 17rem;
  padding: 1.35rem;
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: 2px;
  color: var(--color-text);
  text-decoration: none;
  transition: border-color var(--motion-fast) var(--ease-out-soft), transform var(--motion-fast) var(--ease-out-soft);
}

.resource-card:hover {
  border-color: var(--color-border-hover);
  transform: translateY(-6px);
}

.resource-card:focus-visible {
  outline: var(--focus-ring);
  outline-offset: var(--focus-ring-offset);
}

@media (prefers-reduced-motion: reduce) {
  .resource-card:hover {
    transform: none;
  }
}

.resource-card__eyebrow {
  color: var(--color-text-muted);
  font-size: 0.75rem;
  font-weight: 600;
  letter-spacing: 0.12em;
  text-transform: uppercase;
}

.resource-card__title {
  margin: 1rem 0 0;
  font-size: 1.125rem;
  line-height: 1.3;
  letter-spacing: -0.02em;
}

```

```
.resource-card__body {
  margin: 0.75rem 0 0;
  color: var(--color-text);
  font-size: 0.9375rem;
  line-height: var(--lh-body);
  word-break: keep-all;
}

.resource-card__path {
  margin-top: auto;
  padding-top: 1.25rem;
  color: var(--color-text-muted);
  font-family: ui-monospace, "Cascadia Code", "Segoe UI Mono", Consolas, monospace;
  font-size: 0.75rem;
  line-height: 1.45;
  word-break: break-all;
}

.section--footer-refs .prose ul {
  list-style: none;
  padding: 0;
  margin: 0 0 1.5rem;
}

.refs-list li {
  margin: 0 0 0.85rem;
  padding-bottom: 0.85rem;
  border-bottom: 1px solid var(--color-border);
  line-height: var(--lh-body);
}

.refs-list li:last-child {
  margin-bottom: 0;
  padding-bottom: 0;
  border-bottom: none;
}

.refs-list a {
  text-decoration: none;
}

.refs-list a:hover {
  text-decoration: underline;
  text-underline-offset: 0.2em;
}

.refs-list a:focus-visible {
  outline: var(--focus-ring);
  outline-offset: var(--focus-ring-offset);
}

.refs-list__term {
  display: inline-block;
  min-width: 7rem;
  color: var(--color-text-muted);
  font-weight: 500;
}

.caption {
  margin: 0;
  font-size: var(--text-caption);
  line-height: var(--lh-caption);
  color: var(--color-text-muted);
}

.site-footer {
  padding: 2.5rem var(--space-gutter) 3rem;
```

```
}

.site-footer__inner {
  max-width: 1200px;
  margin: 0 auto;
}

.site-footer__note {
  margin: 0;
  font-size: calc(var(--text-caption) * 2);
  line-height: var(--lh-body);
  text-align: center;
  color: var(--color-text-muted);
}

.site-footer__file {
  font-family: ui-monospace, "Cascadia Code", "Segoe UI Mono", Consolas, monospace;
  font-size: 0.8125rem;
}

.prose .refs-list + .caption {
  margin-top: 0;
}
```

cv/css/reset.css

```
/* 이력서 페이지에서 브라우저 기본 여백과 글자 스타일을 지우는 리셋이다. */
/* http://meyerweb.com/eric/tools/css/reset/
   v2.0 | 20110126
   License: none (public domain)
*/

html, body, div, span, applet, object, iframe,
h1, h2, h3, h4, h5, h6, p, blockquote, pre,
a, abbr, acronym, address, big, cite, code,
del, dfn, em, img, ins, kbd, q, s, samp,
small, strike, strong, sub, sup, tt, var,
b, u, i, center,
dl, dt, dd, ol, ul, li,
fieldset, form, label, legend,
table, caption, tbody, tfoot, thead, tr, th, td,
article, aside, canvas, details, embed,
figure, figcaption, footer, header, hgroup,
menu, nav, output, ruby, section, summary,
time, mark, audio, video {
    margin: 0;
    padding: 0;
    border: 0;
    font-size: 100%;
    font: inherit;
    vertical-align: baseline;
}
/* HTML5 display-role reset for older browsers */
article, aside, details, figcaption, figure,
footer, header, hgroup, menu, nav, section {
    display: block;
}
body {
    line-height: 1;
}
ol, ul {
    list-style: none;
}
blockquote, q {
    quotes: none;
}
blockquote:before, blockquote:after,
q:before, q:after {
    content: '';
    content: none;
}
table {
    border-collapse: collapse;
    border-spacing: 0;
}
```

cv/css/style.css

```
@charset "UTF-8";

/* 이력서 본문의 배경, 글꼴, 경력 항목 여백과 구분선을 정한다. */

@import url('http://fonts.googleapis.com/earlyaccess/nanumgothic.css');
@import url('http://fonts.googleapis.com/earlyaccess/nanummyeongjo.css');

:root {
   --resume-bg: #ffffff;
   --resume-text: #222222;
   --resume-muted: #999999;
   --resume-border-strong: #222222;
   --resume-border-soft: #e9e9e9;
}

html[data-theme="dark"] {
   --resume-bg: #0c0c0c;
   --resume-text: #f5f5f5;
   --resume-muted: rgba(245, 245, 245, 0.62);
   --resume-border-strong: rgba(245, 245, 245, 0.82);
   --resume-border-soft: rgba(245, 245, 245, 0.16);
}

body {
   background: var(--resume-bg);
   color: var(--resume-text);
}

.wrap {
   padding: 24px 24px 50px;
   font-family: 'NanumGothic', 'Nanum Gothic', 'sans-serif';
   font-size: 14px;
   line-height: 1.2;
   color: var(--resume-text);
}
.header {
   position: relative;
   margin-bottom: 24px;
}
.header-title {
   margin-bottom: 24px;
   font-family: 'NanumMyeongjo', 'Nanum Myeongjo', 'sans-serif';
   font-size: 24px;
}
.header-picture-image {
   width: 100%;
   max-width: 150px;
   vertical-align: top;
}
.content {
   position: relative;
}
.section {
   position: relative;
   margin-top: 60px;
}
.section-title {
   font-size: 16px;
   font-weight: bold;
}
.article {
   margin-top: 24px;
}
```

```
.section-title + .article {
   margin-top: 6px;
   padding-top: 24px;
   border-top: 3px solid var(--resume-border-strong);
}
.article-title {
   position: relative;
   font-weight: bold;
}
.article-list {
   list-style: disc;
   padding-left: 18px;
}
.article-item {
   margin-top: 6px;
}
.article-text {
   margin-top: 6px;
}
.site-footer {
   margin-top: 96px;
   padding-top: 28px;
   border-top: 1px solid var(--resume-border-soft);
}
.site-footer__inner {
   width: 100%;
}
.site-footer__note {
   margin: 0;
   font-size: 1.625rem;
   line-height: 1.6;
   color: var(--resume-muted);
   text-align: center;
}

@media (min-width: 600px) {

    .wrap {
        max-width: 700px;
        margin: 0 auto;
        padding: 50px 50px 100px;
    }
    /* .header {
        overflow: hidden;
    } */
    .header::after {
        content: '';
        display: block;
        clear: both;
    }
    .header-title {
        float: left;
        width: 70%;
    }
    .header-picture {
        float: right;
        width: 28%;
    }
    .header-picture-image {
       max-width: 100%;
    }
    .section::after {
        content: '';
        display: block;
        clear: both;
    }
    .section-title {
```

```
        position: sticky;
        top: 10px;
        float: left;
        width: 28%;
        padding-top: 12px;
        border-top: 3px solid var(--resume-border-strong);
    }
    .article {
        float: right;
        width: 70%;
    }
    .section-title + .article {
        margin-top: 0;
        padding-top: 12px;
    }
} /* @media (min-width: 600px) */
```

cv/js/ui.js

```
/**
 * 이력서의 각 경력 항목에서 시작일·종료일을 읽어 재직 기간을 계산한다.
 * 종료일이 today이면 오늘 날짜로 바꾸고, 연·개월을 periodResult에 괄호로 표시한다.
 */
function fillCareerPeriods() {
    $('.article').each(function () {

        const startDate = $(this).find('.startDate').text().trim();

        if (startDate !== '') {

            const startDateArr = startDate.split('.').map(str => str.trim()).filter(Boolean);
            let endDate = $(this).find('.endDate').text().trim();
            let endDateArr;

            if (endDate === 'today') {

                const now = new Date();
                const year = now.getFullYear();
                const month = String(now.getMonth() + 1).padStart(2, '0');
                const day = String(now.getDate()).padStart(2, '0');

                endDateArr = [year, month, day];
                $(this).find('.endDate').text(`${year}. ${month}. ${day}.`);

            } else {

                endDateArr = endDate.split('.').map(str => str.trim()).filter(Boolean);

                const year = endDateArr[0];
                const month = String(parseInt(endDateArr[1], 10)).padStart(2, '0');
                const day = String(parseInt(endDateArr[2], 10)).padStart(2, '0');

                endDateArr = [year, month, day];
                $(this).find('.endDate').text(`${year}. ${month}. ${day}.`);

            }

            const startDateCalc = new Date(startDateArr[0], startDateArr[1], startDateArr[2]);
            const endDateCalc = new Date(endDateArr[0], endDateArr[1], endDateArr[2]);
            const dateCalc = endDateCalc.getTime() - startDateCalc.getTime();
            const dateCalcResult = dateCalc / (1000 * 60 * 60 * 24);

            let dateResultText;
            const dateResultYear = Math.floor(dateCalcResult / 365);
            const dateResultMonth = Math.round((dateCalcResult - (dateResultYear * 365)) / 30);

            if (dateResultYear === 0 && dateResultMonth === 0) {
                dateResultText = '-';
            } else if (dateResultYear === 0) {
                dateResultText = `${dateResultMonth}개월`;
            } else if (dateResultMonth === 0) {
                dateResultText = `${dateResultYear}년`;
            } else {
                dateResultText = `${dateResultYear}년 ${dateResultMonth}개월`;
            }

            $(this).find('.periodResult').text(`(${dateResultText})`);
        }

    });
}
```

```
63
64  $(document).ready(fillCareerPeriods);
```

cv/resume.html

```
<!-- 이력서 화면. 학력과 경력의 시작·종료 날짜를 보여주고, 기간 계산 스크립트와 공통 헤더를 불러온다. -->
<!DOCTYPE html>
<html lang="ko" data-default-theme="light">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>이력서</title>
    <script>
        (function () {
            try {
                var theme = localStorage.getItem("portfolio-theme-v2");
                if (theme === "dark" || theme === "light") {
                    document.documentElement.setAttribute("data-theme", theme);
                }
            } catch (e) {
                /* 테마 저장소를 사용할 수 없는 환경에서는 기본 테마를 사용합니다. */
            }
        })();
    </script>
    <link rel="stylesheet" href="css/reset.css">
    <link rel="stylesheet" href="css/style.css?v=theme-sync">
    <link rel="stylesheet" href="../css/site-header.css?v=theme-sync">
</head>
<body>
    <header class="site-header">
        <div class="site-header__inner">
            <a class="site-header__brand" href="../index.html">코탐록</a>
            <nav id="site-nav" class="site-nav" aria-label="자료 페이지">
                <ul class="site-nav__list">
                    <li><a class="site-nav__link" href="../cover_letter/profile.html">자기소개 / 프로필</a></li>
                    <li><a class="site-nav__link" href="../online_business_card/business-card.html">온라인 명함</a></li
>
                    <li><a class="site-nav__link is-active" href="../cv/resume.html" aria-current="page">이력서</a></li
>
                    <li><a class="site-nav__link" href="../study_note_record_organization/dashboard/index.html">학습록
</a></li>
                </ul>
            </nav>
            <div class="site-header__actions">
                <button type="button" class="theme-toggle" data-theme-toggle aria-pressed="false" aria-label="다크 테마
로 전환">
                    <span class="theme-toggle__label">고대비</span>
                    <span class="theme-toggle__state" data-theme-label>라이트</span>
                </button>
                <button
                    type="button"
                    class="nav-toggle"
                    aria-expanded="false"
                    aria-controls="site-nav"
                    aria-label="메뉴 열기"
                >
                    <span class="nav-toggle__bar" aria-hidden="true"></span>
                    <span class="nav-toggle__bar" aria-hidden="true"></span>
                </button>
            </div>
        </div>
    </header>

    <!-- wrap-->
    <div class="wrap">
        <!-- header -->
        <header class="header">
            <h1 class="header-title">
```

```
                이력서
            </h1>
            <figure class="header-picture">
                <img src="img/profile.png" class="header-picture-image" alt="사진">
            </figure>
        </header>
        <!-- //header -->

        <!-- content -->
        <main class="content">
            <!-- section -->
            <section class="section">

                <h2 class="section-title">경력</h2>

                <article class="article">
                    <h3 class="article-title">추가 예정</h3>
                    <p class="article-text">아직 등록된 구직 이력이 없어, 관련 경력 사항은 추후 추가 예정입니다.</p>
                </article>

            </section>
            <!-- //section -->

            <!-- section -->
            <section class="section">

                <h2 class="section-title">학력</h2>

                <article class="article">
                    <h3 class="article-title">대학 졸업</h3>
                </article>

                <article class="article">
                    <h3 class="article-title">학점은행제 도전 중</h3>
                </article>

            </section>
            <!-- //section -->

            <!-- section -->
            <section class="section">

                <h2 class="section-title">자격증</h2>

                <article class="article">
                    <h3 class="article-title">워드프로세서 1급</h3>
                </article>

            </section>
            <!-- //section -->

            <footer class="site-footer">
                <div class="site-footer__inner">
                    <p class="site-footer__note">
                        2026년 5월 25일 제작 · Cursor AI를 사용해 제작했습니다.
                    </p>
                </div>
            </footer>

        </main>
        <!-- //content -->

    </div>
    <!-- //wrap-->

    <script src="js/jquery-4.0.0.min.js"></script>
    <script src="js/ui.js"></script>
```

```
    <script src="../js/main.js?v=theme-sync"></script>

</body>
</html>
```

docs/build_code_reference.py

```python
# -*- coding: utf-8 -*-
"""소스 주석을 읽어 코드 설명 마크다운과 PDF를 만든다."""
from __future__ import annotations

import ast
import html
import os
import re
import shutil
import subprocess
import tempfile
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
DOCS = Path(__file__).resolve().parent
MD_PATH = DOCS / "code-reference.md"
HTML_PATH = DOCS / "code-reference.html"
PDF_PATH = DOCS / "code-reference.pdf"

ID_RE = re.compile(r"\d{7,8}")
NAME_RE = re.compile("김신욱")


def mask_privacy(text: str) -> str:
    """파일 목록에 섞인 학번과 이름을 가린다."""
    text = ID_RE.sub("OOOOOOOO", text)
    return NAME_RE.sub("OOO", text)


def rel(path: Path) -> str:
    return path.relative_to(ROOT).as_posix()


class Doc:
    def __init__(self) -> None:
        self.blocks: list[tuple] = []

    def h1(self, text: str) -> None:
        self.blocks.append(("h1", text))

    def h2(self, text: str) -> None:
        self.blocks.append(("h2", text))

    def h3(self, text: str) -> None:
        self.blocks.append(("h3", text))

    def h4(self, text: str) -> None:
        self.blocks.append(("h4", text))

    def p(self, text: str) -> None:
        self.blocks.append(("p", text))

    def ul(self, items: list[str]) -> None:
        self.blocks.append(("ul", items))

    def code(self, text: str) -> None:
        self.blocks.append(("code", text))

    def render_md(self) -> str:
        out: list[str] = []
        for kind, payload in self.blocks:
            if kind == "h1":
                out.append(f"# {payload}\n")
```

```
            elif kind == "h2":
                out.append(f"\n## {payload}\n")
            elif kind == "h3":
                out.append(f"\n### {payload}\n")
            elif kind == "h4":
                out.append(f"\n#### {payload}\n")
            elif kind == "p":
                out.append(payload + "\n")
            elif kind == "ul":
                out.extend(f"- {item}" for item in payload)
                out.append("")
            elif kind == "code":
                out.append("```")
                out.append(payload)
                out.append("```\n")
        return "\n".join(out).strip() + "\n"

    def render_html(self) -> str:
        parts = [
            "<!DOCTYPE html>",
            '<html lang="ko">',
            "<head>",
            '<meta charset="utf-8" />',
            "<title>코드 설명</title>",
            "<style>",
            HTML_CSS,
            "</style>",
            "</head>",
            "<body>",
        ]
        first_h2 = True
        for kind, payload in self.blocks:
            if kind == "h1":
                parts.append(f"<h1>{html.escape(payload)}</h1>")
            elif kind == "h2":
                cls = "first" if first_h2 else ""
                first_h2 = False
                parts.append(f'<h2 class="{cls}">{html.escape(payload)}</h2>')
            elif kind == "h3":
                parts.append(f"<h3>{html.escape(payload)}</h3>")
            elif kind == "h4":
                parts.append(f"<h4>{html.escape(payload)}</h4>")
            elif kind == "p":
                parts.append(f"<p>{inline_html(payload)}</p>")
            elif kind == "ul":
                items = "".join(f"<li>{inline_html(item)}</li>" for item in payload)
                parts.append(f"<ul>{items}</ul>")
            elif kind == "code":
                parts.append(f"<pre>{html.escape(payload)}</pre>")
        parts.append("</body></html>")
        return "\n".join(parts)


HTML_CSS = """
@page { margin: 18mm 16mm; }
body { font-family: "Malgun Gothic", sans-serif; font-size: 11pt; line-height: 1.55; color: #1c1c1c; }
h1 { font-size: 22pt; margin: 0 0 0.6em; }
h2 { font-size: 16pt; margin: 1.4em 0 0.4em; page-break-before: always; }
h2.first { page-break-before: auto; }
h3 { font-size: 13pt; margin: 1.1em 0 0.3em; page-break-after: avoid; }
h4 { font-size: 11.5pt; margin: 0.9em 0 0.25em; page-break-after: avoid; }
p, li { orphans: 3; widows: 3; }
ul { margin: 0.2em 0 0.8em; padding-left: 1.2em; }
code { font-family: Consolas, "Malgun Gothic", monospace; font-size: 9.5pt; }
pre { font-family: Consolas, "Malgun Gothic", monospace; font-size: 9.5pt; background: #f3f3f3;
      padding: 8px 10px; white-space: pre-wrap; page-break-inside: avoid; }
"""
```

```

INLINE_CODE = re.compile(r"`([^`]+)`")
BOLD = re.compile(r"\*\*(.+?)\*\*")


def inline_html(text: str) -> str:
    escaped = html.escape(text)
    escaped = INLINE_CODE.sub(lambda m: f"<code>{m.group(1)}</code>", escaped)
    escaped = BOLD.sub(lambda m: f"<strong>{m.group(1)}</strong>", escaped)
    return escaped


def read_text(path: Path) -> str:
    return path.read_text(encoding="utf-8")


def jsdoc_before(lines: list[str], index: int) -> str:
    if index == 0 or lines[index - 1].strip() != "*/":
        return ""
    start = index - 1
    while start >= 0 and "/**" not in lines[start]:
        start -= 1
    if start < 0:
        return ""
    chunk = "\n".join(lines[start:index])
    body_lines = []
    for line in chunk.splitlines():
        line = re.sub(r"^\s*/?\*+/?\s?", "", line).strip()
        if line in ("/**", "*/", "*"):
            continue
        if line:
            body_lines.append(line)
    return "\n".join(body_lines).strip()


def split_jsdoc(doc: str) -> tuple[str, list[str], str]:
    desc: list[str] = []
    params: list[str] = []
    returns = ""
    for line in doc.splitlines():
        param = re.match(r"@param\s+\{([^}]+)\}\s+(\S+)\s*(.*)", line)
        ret = re.match(r"@returns?\s+\{([^}]+)\}\s*(.*)", line)
        if param:
            extra = f" {param.group(3)}".rstrip()
            params.append(f"{param.group(2)}: {param.group(1)}{extra}")
        elif ret:
            extra = f" {ret.group(2)}".rstrip()
            returns = f"{ret.group(1)}{extra}"
        elif not line.startswith("@"):
            desc.append(line)
    return " ".join(desc).strip(), params, returns


def clip_expr(text: str, limit: int = 72) -> str:
    text = " ".join(text.split())
    if len(text) <= limit:
        return text
    return text[: limit - 1] + "…"


def py_expr(node: ast.AST | None) -> str:
    if node is None:
        return ""
    try:
        return clip_expr(ast.unparse(node))
    except Exception:
        return "식"
```

```


def append_py_stmt(stmt: ast.AST, depth: int, lines: list[str], limit: int) -> None:
    if len(lines) >= limit:
        return
    pad = "  " * depth
    if isinstance(stmt, ast.Expr) and isinstance(stmt.value, ast.Constant) and isinstance(stmt.value.value, str):
        return
    if isinstance(stmt, ast.Pass):
        return
    if isinstance(stmt, (ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef)):
        lines.append(f"{pad}{stmt.name} 을 정의한다")
        return
    if isinstance(stmt, ast.Return):
        value = py_expr(stmt.value)
        lines.append(f"{pad}반환 {value}" if value else f"{pad}반환")
        return
    if isinstance(stmt, ast.Assign):
        targets = ", ".join(py_expr(target) for target in stmt.targets)
        lines.append(f"{pad}{targets} ← {py_expr(stmt.value)}")
        return
    if isinstance(stmt, ast.AnnAssign):
        if stmt.value is not None:
            lines.append(f"{pad}{py_expr(stmt.target)} ← {py_expr(stmt.value)}")
        else:
            lines.append(f"{pad}필드 {py_expr(stmt.target)}")
        return
    if isinstance(stmt, ast.AugAssign):
        lines.append(f"{pad}{py_expr(stmt.target)} ← {py_expr(stmt.target)} 연산 {py_expr(stmt.value)}")
        return
    if isinstance(stmt, ast.If):
        lines.append(f"{pad}만약 {py_expr(stmt.test)}")
        for child in stmt.body:
            append_py_stmt(child, depth + 1, lines, limit)
        if stmt.orelse:
            lines.append(f"{pad}아니면")
            for child in stmt.orelse:
                append_py_stmt(child, depth + 1, lines, limit)
        return
    if isinstance(stmt, ast.For):
        lines.append(f"{pad}반복 {py_expr(stmt.target)} ← {py_expr(stmt.iter)}")
        for child in stmt.body:
            append_py_stmt(child, depth + 1, lines, limit)
        return
    if isinstance(stmt, ast.While):
        lines.append(f"{pad}동안 {py_expr(stmt.test)}")
        for child in stmt.body:
            append_py_stmt(child, depth + 1, lines, limit)
        return
    if isinstance(stmt, ast.With):
        heads = ", ".join(py_expr(item.context_expr) for item in stmt.items)
        lines.append(f"{pad}범위 {heads}")
        for child in stmt.body:
            append_py_stmt(child, depth + 1, lines, limit)
        return
    if isinstance(stmt, ast.Try):
        lines.append(f"{pad}시도")
        for child in stmt.body:
            append_py_stmt(child, depth + 1, lines, limit)
        for handler in stmt.handlers:
            label = py_expr(handler.type) if handler.type else "예외"
            lines.append(f"{pad}실패 {label}")
            for child in handler.body:
                append_py_stmt(child, depth + 1, lines, limit)
        return
    if isinstance(stmt, ast.Raise):
        lines.append(f"{pad}중단 {py_expr(stmt.exc)}".rstrip())
```

```
        return
    if isinstance(stmt, ast.Continue):
        lines.append(f"{pad}다음 반복")
        return
    if isinstance(stmt, ast.Break):
        lines.append(f"{pad}반복 종료")
        return
    if isinstance(stmt, ast.Delete):
        lines.append(f"{pad}삭제 {', '.join(py_expr(target) for target in stmt.targets)}")
        return
    if isinstance(stmt, (ast.Import, ast.ImportFrom)):
        lines.append(f"{pad}모듈을 불러온다")
        return
    try:
        lines.append(f"{pad}{clip_expr(ast.unparse(stmt))}")
    except Exception:
        lines.append(f"{pad}문장을 실행한다")


def py_pseudocode(node: ast.AST) -> str:
    lines: list[str] = []
    if isinstance(node, ast.ClassDef):
        lines.append(f"클래스 {node.name}")
        for stmt in node.body:
            if isinstance(stmt, ast.Expr) and isinstance(stmt.value, ast.Constant):
                continue
            if isinstance(stmt, (ast.FunctionDef, ast.AsyncFunctionDef)):
                lines.append(f"  메서드 {stmt.name}")
            elif isinstance(stmt, ast.AnnAssign):
                lines.append(f"  필드 {py_expr(stmt.target)}")
            elif isinstance(stmt, ast.Assign):
                targets = ", ".join(py_expr(target) for target in stmt.targets)
                lines.append(f"  {targets} ← {py_expr(stmt.value)}")
            else:
                append_py_stmt(stmt, 1, lines, 40)
        return "\n".join(lines)
    for stmt in getattr(node, "body", []):
        append_py_stmt(stmt, 0, lines, 40)
    if len(lines) >= 40:
        lines.append("…")
    return "\n".join(lines) if lines else "수행할 문장이 없다"


def js_body_lines(lines: list[str], index: int) -> list[str]:
    blob = "\n".join(lines[index:])
    start = blob.find("{")
    if start < 0:
        return []
    depth = 0
    end = start
    for pos in range(start, len(blob)):
        char = blob[pos]
        if char == "{":
            depth += 1
        elif char == "}":
            depth -= 1
            if depth == 0:
                end = pos
                break
    body = blob[start + 1 : end]
    return body.splitlines()


def js_pseudocode(lines: list[str], index: int) -> str:
    body = js_body_lines(lines, index)
    out: list[str] = []
    depth = 0
```

```
    skipping = False
    skip_depth = 0
    for raw in body:
        if len(out) >= 40:
            break
        line = raw.strip()
        if not line or line.startswith("//") or line.startswith("/*") or line.startswith("*"):
            if "{" in line or "}" in line:
                pass
            else:
                continue
        if skipping:
            skip_depth += line.count("{") - line.count("}")
            if skip_depth <= 0:
                skipping = False
            continue
        nested = re.match(r"function\s+(\w+)\s*\(", line)
        if nested:
            out.append(f"{'  ' * depth}{nested.group(1)} 을 정의한다")
            skip_depth = line.count("{") - line.count("}")
            if skip_depth > 0:
                skipping = True
            continue
        open_count = line.count("{")
        close_count = line.count("}")
        code = re.sub(r"\s*\{\s*$", "", line).rstrip(";").strip()
        if re.match(r"\}?\s*else\s+if\b", code):
            code = re.sub(r"^\}?\s*", "", code)
        elif re.match(r"\}?\s*else\b", code):
            code = "else"
        pad = "  " * depth
        if code.startswith("if ("):
            out.append(f"{pad}만약 {clip_expr(code[3:].strip(' ()'))}")
        elif code.startswith("} else if (") or code.startswith("else if ("):
            cond = code.split("(", 1)[1].rsplit(")", 1)[0]
            out.append(f"{pad}아니면 만약 {clip_expr(cond)}")
        elif code in ("else", "} else"):
            out.append(f"{pad}아니면")
        elif code.startswith("for ") or code.startswith("for("):
            out.append(f"{pad}반복 {clip_expr(code)}")
        elif code.startswith("while"):
            out.append(f"{pad}동안 {clip_expr(code)}")
        elif code.startswith("return"):
            value = code[6:].strip()
            out.append(f"{pad}반환 {clip_expr(value)}" if value else f"{pad}반환")
        elif re.match(r"(?:var|let|const)\s+", code):
            out.append(f"{pad}{clip_expr(re.sub(r'^(?:var|let|const)\s+', '', code).replace(' = ', ' ← ', 1))}")
        elif code.startswith("try"):
            out.append(f"{pad}시도")
        elif code.startswith("catch"):
            out.append(f"{pad}실패")
        elif code.startswith("throw"):
            out.append(f"{pad}중단 {clip_expr(code[5:].strip())}")
        elif code and not re.fullmatch(r"[\s{}\[\]();,]+", code):
            out.append(f"{pad}{clip_expr(code)}")
        depth += open_count - close_count
        if depth < 0:
            depth = 0
    if len(out) >= 40:
        out.append("…")
    return "\n".join(out) if out else "수행할 문장이 없다"


def parse_js(path: Path) -> tuple[str, list[dict]]:
    lines = read_text(path).splitlines()
    file_doc = ""
    if lines and lines[0].strip().startswith("/**"):
```

```
        end = 0
        while end < len(lines) and "*/" not in lines[end]:
            end += 1
        file_doc = jsdoc_before(lines, end + 1)
    symbols = []
    for index, line in enumerate(lines):
        match = re.match(r"\s*function\s+(\w+)\s*\(([^)]*)\)", line)
        if not match:
            continue
        doc = jsdoc_before(lines, index)
        desc, params, returns = split_jsdoc(doc)
        if not params and match.group(2).strip():
            params = [part.strip() for part in match.group(2).split(",") if part.strip()]
        symbols.append(
            {
                "name": match.group(1),
                "signature": f"function {match.group(1)}({match.group(2).strip()})",
                "line": index + 1,
                "desc": desc,
                "params": params,
                "returns": returns,
                "pseudo": js_pseudocode(lines, index),
            }
        )
    return file_doc, symbols


def annotation(node: ast.AST | None) -> str:
    if node is None:
        return ""
    return ast.unparse(node)


def py_params(node: ast.FunctionDef | ast.AsyncFunctionDef) -> list[str]:
    args = node.args
    names: list[ast.arg] = list(args.posonlyargs) + list(args.args)
    default_start = len(names) - len(args.defaults)
    parts: list[str] = []
    for index, arg in enumerate(names):
        if arg.arg in ("self", "cls"):
            continue
        label = arg.arg
        ann = annotation(arg.annotation)
        if ann:
            label += f": {ann}"
        if index >= default_start:
            default = ast.unparse(args.defaults[index - default_start])
            label += f" = {default}"
        parts.append(label)
    if args.vararg:
        label = "*" + args.vararg.arg
        ann = annotation(args.vararg.annotation)
        parts.append(f"{label}: {ann}" if ann else label)
    for arg, default in zip(args.kwonlyargs, args.kw_defaults):
        label = arg.arg
        ann = annotation(arg.annotation)
        if ann:
            label += f": {ann}"
        if default is not None:
            label += f" = {ast.unparse(default)}"
        parts.append(label)
    if args.kwarg:
        label = "**" + args.kwarg.arg
        ann = annotation(args.kwarg.annotation)
        parts.append(f"{label}: {ann}" if ann else label)
    return parts

```

```

def py_signature(node: ast.AST) -> str:
    if isinstance(node, ast.ClassDef):
        bases = ", ".join(ast.unparse(base) for base in node.bases)
        return f"class {node.name}({bases})" if bases else f"class {node.name}"
    assert isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef))
    prefix = "async def" if isinstance(node, ast.AsyncFunctionDef) else "def"
    ret = annotation(node.returns)
    tail = f" -> {ret}" if ret else ""
    return f"{prefix} {node.name}({ast.unparse(node.args)}){tail}"


def iter_py_symbols(node: ast.AST, prefix: str = ""):
    for child in getattr(node, "body", []):
        if isinstance(child, ast.ClassDef):
            name = f"{prefix}{child.name}"
            yield name, child
            yield from iter_py_symbols(child, name + ".")
        elif isinstance(child, (ast.FunctionDef, ast.AsyncFunctionDef)):
            name = f"{prefix}{child.name}"
            yield name, child
            yield from iter_py_symbols(child, name + ".")


def parse_py(path: Path) -> tuple[str, list[dict]]:
    tree = ast.parse(read_text(path))
    file_doc = ast.get_docstring(tree) or ""
    symbols = []
    for name, node in iter_py_symbols(tree):
        desc = ast.get_docstring(node) or ""
        params: list[str] = []
        returns = ""
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            params = py_params(node)
            returns = annotation(node.returns)
        symbols.append(
            {
                "name": name,
                "signature": py_signature(node),
                "line": node.lineno,
                "desc": " ".join(desc.split()),
                "params": params,
                "returns": returns,
                "pseudo": py_pseudocode(node),
            }
        )
    return file_doc, symbols


CODE_SUFFIXES = {
    ".vb",
    ".html",
    ".htm",
    ".js",
    ".css",
    ".py",
    ".cs",
    ".java",
    ".c",
    ".cpp",
    ".h",
    ".config",
    ".vbproj",
    ".sln",
    ".resx",
    ".settings",
    ".suo",
```

```
    ".user",
}


def file_comment(path: Path) -> str:
    text = read_text(path)
    if path.suffix.lower() == ".html":
        match = re.match(r"<!--\s*(.*?)\s*-->", text, re.S)
        return " ".join(match.group(1).split()) if match else ""
    if path.suffix.lower() == ".css":
        if text.startswith("@charset"):
            text = text.split("\n", 1)[1]
        match = re.search(r"/\*\s*(.*?)\s*\*/", text, re.S)
        return " ".join(match.group(1).split()) if match else ""
    return ""


def add_symbol(doc: Doc, path: Path, symbol: dict) -> None:
    doc.h4(symbol["name"])
    doc.p(f"`{rel(path)}:{symbol['line']}`")
    doc.code(symbol["signature"])
    doc.p(symbol["desc"] or "설명이 없습니다.")
    params = symbol["params"] or ["없음"]
    doc.ul([f"인자: {', '.join(params)}", f"반환: {symbol['returns'] or '없음'}"])
    pseudo = symbol.get("pseudo") or ""
    if pseudo:
        doc.p("유사코드")
        doc.code(pseudo)


def add_code_file(doc: Doc, path: Path, file_doc: str, symbols: list[dict]) -> None:
    doc.h3(rel(path))
    if file_doc:
        doc.p(file_doc)
    for symbol in symbols:
        add_symbol(doc, path, symbol)


def data_role(path: Path) -> str:
    name = path.name
    suffix = path.suffix.lower()
    if suffix in {".png", ".jpg", ".jpeg", ".gif", ".webp", ".bmp"}:
        return "학습·과제 화면 캡처 또는 도표"
    if "과제분류하기" in path.parts:
        return "과제 제출 원본"
    if "독서" in name:
        return "독서 노트"
    if "수강" in name or "영상" in name:
        return "수강 노트"
    if suffix == ".hwpx":
        return "한글 문서 원본. 요약 빌드에서는 제외한다"
    return "학습 기록 원본"


def is_generated_data(path: Path) -> bool:
    parts = set(path.parts)
    if parts & {"obj", "bin", "My Project"}:
        return True
    if path.name.endswith(".Designer.vb"):
        return True
    if path.suffix.lower() in {".exe", ".dll", ".pdb", ".cache", ".suo", ".user"}:
        return True
    return False


def collect_originals() -> dict[str, list[str]]:
    data = ROOT / "study_note_record_organization" / "data"
```

```
    grouped: dict[str, list[str]] = {}
    if not data.is_dir():
        return grouped
    for path in sorted(data.rglob("*")):
        if not path.is_file() or is_generated_data(path):
            continue
        if ".vs" in path.parts or path.suffix.lower() in CODE_SUFFIXES:
            continue
        label = mask_privacy(f"{rel(path)} — {data_role(path)}")
        grouped.setdefault(path.suffix.lower() or "(확장자 없음)", []).append(label)
    return grouped


def build() -> Doc:
    doc = Doc()
    doc.h1("코드 설명")
    doc.p(
        "이력서, 자기소개, 온라인 명함, 학습 기록 대시보드와 그 빌드 스크립트를 함수·클래스 단위로 정리한 문서다. "
        "설명은 소스 주석에서 뽑았고, 함수가 없는 파일은 파일 역할만 적었다."
    )

    doc.h2("프로젝트 구성")
    doc.p(
        "루트는 취업 지원용 포트폴리오 페이지다. 이력서, 자기소개, 명함은 각각 폴더로 나뉘고, "
        "학습 노트와 과제 원본은 study_note_record_organization 안에 있다. "
        "그 폴더의 파이썬 스크립트가 원본을 요약 마크다운으로 바꾸고, 대시보드가 읽는 data.js와 원본 JSON을 만든다."
    )
    doc.ul(
        [
            "index.html, css/, js/main.js — 메인 포트폴리오",
            "cv/ — 이력서",
            "cover_letter/ — 자기소개",
            "online_business_card/ — 스크롤 명함",
            "study_note_record_organization/dashboard/ — 학습·과제 카드 화면",
            "study_note_record_organization/scripts/ — 요약과 정적 데이터 빌드",
            "study_note_record_organization/data/ — 노트, 과제, 이미지 원본",
        ]
    )

    doc.h2("디렉터리 지도")
    doc.ul(
        [
            "css/ — 메인 페이지와 공통 헤더 스타일",
            "js/ — 메인 페이지 내비게이션과 테마",
            "cv/css, cv/js — 이력서 스타일과 재직 기간 계산",
            "cover_letter/css, cover_letter/js — 자기소개 스타일과 스크롤 메뉴",
            "online_business_card/css, online_business_card/js — 명함 애니메이션",
            "study_note_record_organization/dashboard/js — 카드, 검색, 히어로. data.js와 sources는 생성물",
            "study_note_record_organization/scripts — build_output.py, build_static_site.py, extract_sources.py",
            "study_note_record_organization/output — 빌드가 쓰는 요약 마크다운과 catalog.json",
            "study_note_record_organization/data/과제분류하기 — 과제 문서 원본. 코드 파일은 이 문서에서 빼 둔다",
        ]
    )

    sections = [
        (
            "메인 사이트",
            [
                ROOT / "index.html",
                ROOT / "css" / "styles.css",
                ROOT / "css" / "site-header.css",
                ROOT / "js" / "main.js",
            ],
        ),
        (
            "이력서",
            [
```

```
                ROOT / "cv" / "resume.html",
                ROOT / "cv" / "css" / "reset.css",
                ROOT / "cv" / "css" / "style.css",
                ROOT / "cv" / "js" / "ui.js",
            ],
        ),
        (
            "자기소개서",
            [
                ROOT / "cover_letter" / "profile.html",
                ROOT / "cover_letter" / "css" / "styles.css",
                ROOT / "cover_letter" / "js" / "script.js",
            ],
        ),
        (
            "명함",
            [
                ROOT / "online_business_card" / "business-card.html",
                ROOT / "online_business_card" / "css" / "style.css",
                ROOT / "online_business_card" / "js" / "script.js",
            ],
        ),
        (
            "학습 기록 대시보드",
            [
                ROOT / "study_note_record_organization" / "dashboard" / "index.html",
                ROOT / "study_note_record_organization" / "dashboard" / "css" / "styles.css",
                ROOT / "study_note_record_organization" / "dashboard" / "js" / "app.js",
                ROOT / "study_note_record_organization" / "dashboard" / "js" / "hero.js",
            ],
        ),
    ]

    for title, paths in sections:
        doc.h2(title)
        for path in paths:
            suffix = path.suffix.lower()
            if suffix == ".js":
                file_doc, symbols = parse_js(path)
                add_code_file(doc, path, file_doc, symbols)
            else:
                doc.h3(rel(path))
                doc.p(file_comment(path) or "파일 역할 주석이 없습니다.")

    doc.h2("빌드 스크립트")
    doc.p(
        "scripts 폴더의 파이썬이 학습 원본을 요약하고 대시보드 데이터를 갱신한다. "
        "아래 설명은 각 함수와 클래스 독스트링이다."
    )
    for path in [
        ROOT / "study_note_record_organization" / "scripts" / "extract_sources.py",
        ROOT / "study_note_record_organization" / "scripts" / "build_output.py",
        ROOT / "study_note_record_organization" / "scripts" / "build_static_site.py",
    ]:
        file_doc, symbols = parse_py(path)
        add_code_file(doc, path, file_doc, symbols)

    doc.h2("생성 산출물")
    doc.p(
        "다음 파일은 사람이 함수를 적어 둔 소스가 아니다. "
        "build_static_site.py를 다시 실행하면 data.js, hero-images.js, sources JSON의 본문이 바뀌므로 그 안에 설명 주석을
넣지 않았다."
    )
    sources = ROOT / "study_note_record_organization" / "dashboard" / "js" / "sources"
    source_count = len(list(sources.glob("*.json"))) if sources.is_dir() else 0
    doc.ul(
        [
```

```
            "study_note_record_organization/dashboard/js/data.js — 카드 목록, 요약 HTML, 원본 파일 지도를 window.SITE_DATA로 넣는다.",
            "study_note_record_organization/dashboard/js/hero-images.js — data/image 경로를 window.HERO_IMAGES 배열로 넣는다.",
            f"study_note_record_organization/dashboard/js/sources/*.json — 원본 텍스트 {source_count}개. 키와 본문만 담는다.",
        ]
    )

    doc.h2("학습 원본 파일 목록")
    doc.p(
        "data 폴더의 노트, 과제 문서, 이미지다. 함수가 없어 파일마다 한 줄만 적는다. "
        "코드 파일은 넣지 않았고, 원문 전문도 넣지 않았다. 학번과 이름은 가렸다."
    )
    grouped = collect_originals()
    if not grouped:
        doc.p("목록에 넣을 원본 파일이 없습니다.")
    for suffix in sorted(grouped):
        doc.h3(suffix)
        doc.ul(grouped[suffix])
    return doc


def find_browser() -> Path | None:
    local = os.environ.get("LOCALAPPDATA", "")
    candidates = [
        Path(r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe"),
        Path(r"C:\Program Files\Microsoft\Edge\Application\msedge.exe"),
        Path(r"C:\Program Files\Google\Chrome\Application\chrome.exe"),
        Path(local) / "Google" / "Chrome" / "Application" / "chrome.exe",
    ]
    for candidate in candidates:
        if candidate.is_file():
            return candidate
    return None


def write_pdf(html_path: Path, pdf_path: Path) -> None:
    browser = find_browser()
    if browser is None:
        raise SystemExit("PDF로 인쇄할 Edge 또는 Chrome을 찾지 못했습니다.")
    if pdf_path.exists():
        pdf_path.unlink()
    profile = Path(tempfile.mkdtemp(prefix="code-ref-pdf-"))
    url = html_path.resolve().as_uri()
    command = [
        str(browser),
        "--headless=new",
        "--disable-gpu",
        "--no-first-run",
        "--no-pdf-header-footer",
        f"--user-data-dir={profile}",
        f"--print-to-pdf={pdf_path.resolve()}",
        url,
    ]
    try:
        subprocess.run(command, check=True, timeout=120)
    finally:
        shutil.rmtree(profile, ignore_errors=True)
    if not pdf_path.is_file() or pdf_path.stat().st_size < 1000:
        raise SystemExit(f"PDF가 만들어지지 않았습니다: {pdf_path}")


def main() -> None:
    doc = build()
    MD_PATH.write_text(doc.render_md(), encoding="utf-8")
    HTML_PATH.write_text(doc.render_html(), encoding="utf-8")
```

```
    write_pdf(HTML_PATH, PDF_PATH)
    print(f"Wrote {MD_PATH}")
    print(f"Wrote {HTML_PATH}")
    print(f"Wrote {PDF_PATH} ({PDF_PATH.stat().st_size // 1024} KB)")


if __name__ == "__main__":
    main()
```

docs/build_codebook.py

```python
# -*- coding: utf-8 -*-
"""코드북 마크다운을 이어 한글 PDF로 만든다."""
from __future__ import annotations

import html
import os
import re
import shutil
import subprocess
import tempfile
from pathlib import Path

ROOT = Path(__file__).resolve().parent
CHAPTERS = ROOT / "codebook"
MD_PATH = ROOT / "codebook.md"
HTML_PATH = ROOT / "codebook.html"
PDF_PATH = ROOT / "codebook.pdf"

INLINE_CODE = re.compile(r"`([^`]+)`")
BOLD = re.compile(r"\*\*(.+?)\*\*")


def inline(text: str) -> str:
    escaped = html.escape(text)
    escaped = INLINE_CODE.sub(lambda match: f"<code>{match.group(1)}</code>", escaped)
    escaped = BOLD.sub(lambda match: f"<strong>{match.group(1)}</strong>", escaped)
    return escaped


def markdown_to_html(text: str) -> str:
    lines = text.splitlines()
    parts: list[str] = []
    index = 0
    while index < len(lines):
        line = lines[index]
        if line.startswith("```"):
            body: list[str] = []
            index += 1
            while index < len(lines) and not lines[index].startswith("```"):
                body.append(lines[index])
                index += 1
            index += 1
            parts.append(f'<pre class="code">{html.escape(chr(10).join(body))}</pre>')
            continue
        if not line.strip():
            index += 1
            continue
        if line.startswith("#"):
            level = len(line) - len(line.lstrip("#"))
            level = min(max(level, 1), 4)
            title = line[level:].strip()
            parts.append(f"<h{level}>{inline(title)}</h{level}>")
            index += 1
            continue
        if line.startswith("- "):
            items: list[str] = []
            while index < len(lines) and lines[index].startswith("- "):
                items.append(f"<li>{inline(lines[index][2:].strip())}</li>")
                index += 1
            parts.append("<ul>" + "".join(items) + "</ul>")
            continue
        if re.match(r"\d+\.\s+", line):
            items = []
```

```
            while index < len(lines) and re.match(r"\d+\.\s+", lines[index]):
                item_text = re.sub(r"^\d+\.\s+", "", lines[index]).strip()
                items.append(f"<li>{inline(item_text)}</li>")
                index += 1
            parts.append("<ol>" + "".join(items) + "</ol>")
            continue
        paragraph = [line.strip()]
        index += 1
        while (
            index < len(lines)
            and lines[index].strip()
            and not lines[index].startswith(("#", "- ", "```"))
            and not re.match(r"\d+\.\s+", lines[index])
        ):
            paragraph.append(lines[index].strip())
            index += 1
        parts.append(f"<p>{inline(' '.join(paragraph))}</p>")
    return "\n".join(parts)


def combined_markdown() -> str:
    files = sorted(CHAPTERS.glob("*.md"))
    if not files:
        raise SystemExit(f"코드북 마크다운이 없습니다: {CHAPTERS}")
    chunks = [path.read_text(encoding="utf-8").strip() for path in files]
    return "\n\n".join(chunks) + "\n"


HTML_CSS = """
@page { margin: 18mm 16mm; }
body { font-family: "Malgun Gothic", sans-serif; font-size: 11pt; line-height: 1.55; color: #1c1c1c; }
h1 { font-size: 20pt; margin: 0 0 0.6em; page-break-before: always; }
h1:first-of-type { page-break-before: auto; }
h2 { font-size: 15pt; margin: 1.2em 0 0.4em; page-break-after: avoid; }
h3 { font-size: 12.5pt; margin: 1em 0 0.3em; page-break-after: avoid; }
p, li { orphans: 3; widows: 3; }
ul, ol { margin: 0.2em 0 0.8em; padding-left: 1.2em; }
code { font-family: Consolas, "Malgun Gothic", monospace; font-size: 9.5pt; }
pre { font-family: Consolas, "Malgun Gothic", monospace; font-size: 9pt; background: #f3f3f3;
      padding: 8px 10px; white-space: pre-wrap; }
"""


def render_html(body: str) -> str:
    return "\n".join(
        [
            "<!DOCTYPE html>",
            '<html lang="ko">',
            "<head>",
            '<meta charset="utf-8" />',
            "<title>코드북</title>",
            "<style>",
            HTML_CSS,
            "</style>",
            "</head>",
            "<body>",
            body,
            "</body>",
            "</html>",
        ]
    )


def find_browser() -> Path | None:
    local = os.environ.get("LOCALAPPDATA", "")
    candidates = [
        Path(r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe"),
```

```
        Path(r"C:\Program Files\Microsoft\Edge\Application\msedge.exe"),
        Path(r"C:\Program Files\Google\Chrome\Application\chrome.exe"),
        Path(local) / "Google" / "Chrome" / "Application" / "chrome.exe",
    ]
    for candidate in candidates:
        if candidate.is_file():
            return candidate
    return None


def write_pdf(html_path: Path, pdf_path: Path) -> None:
    browser = find_browser()
    if browser is None:
        raise SystemExit("PDF로 인쇄할 Edge 또는 Chrome을 찾지 못했습니다.")
    if pdf_path.exists():
        pdf_path.unlink()
    profile = Path(tempfile.mkdtemp(prefix="codebook-pdf-"))
    command = [
        str(browser),
        "--headless=new",
        "--disable-gpu",
        "--no-first-run",
        "--no-pdf-header-footer",
        f"--user-data-dir={profile}",
        f"--print-to-pdf={pdf_path.resolve()}",
        html_path.resolve().as_uri(),
    ]
    try:
        subprocess.run(command, check=True, timeout=120)
    finally:
        shutil.rmtree(profile, ignore_errors=True)
    if not pdf_path.is_file() or pdf_path.stat().st_size < 1000:
        raise SystemExit(f"PDF가 만들어지지 않았습니다: {pdf_path}")


def main() -> None:
    markdown = combined_markdown()
    MD_PATH.write_text(markdown, encoding="utf-8")
    HTML_PATH.write_text(render_html(markdown_to_html(markdown)), encoding="utf-8")
    write_pdf(HTML_PATH, PDF_PATH)
    print(f"Wrote {MD_PATH}")
    print(f"Wrote {HTML_PATH}")
    print(f"Wrote {PDF_PATH} ({PDF_PATH.stat().st_size // 1024} KB)")


if __name__ == "__main__":
    main()
```

docs/build_source_bundle.py

```python
# -*- coding: utf-8 -*-
"""주석이 붙은 소스를 파일 단위로 이어 하나의 PDF로 만든다."""
from __future__ import annotations

import html
import os
import shutil
import subprocess
import tempfile
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
DOCS = Path(__file__).resolve().parent
HTML_PATH = DOCS / "source-bundle.html"
PDF_PATH = DOCS / "source-bundle.pdf"

SUFFIXES = {".html", ".css", ".js", ".py"}
SKIP_DIRS = {
    "data",
    "output",
    "sources",
    "__pycache__",
    ".vs",
    "obj",
    "bin",
    "node_modules",
}
SKIP_FILES = {"data.js", "hero-images.js"}


def include(path: Path) -> bool:
    if path.suffix.lower() not in SUFFIXES:
        return False
    if path.name in SKIP_FILES or path.name.endswith(".min.js"):
        return False
    if any(part in SKIP_DIRS for part in path.parts):
        return False
    if DOCS in path.parents and path.suffix.lower() == ".html":
        return False
    return True


def source_files() -> list[Path]:
    files = [path for path in ROOT.rglob("*") if path.is_file() and include(path)]
    return sorted(files, key=lambda path: path.relative_to(ROOT).as_posix())


def file_block(path: Path) -> str:
    rel = path.relative_to(ROOT).as_posix()
    text = path.read_text(encoding="utf-8")
    lines = text.splitlines()
    width = max(len(str(len(lines))), 2)
    numbered = "\n".join(f"{index:>{width}}  {line}" for index, line in enumerate(lines, start=1))
    return f"<h2>{html.escape(rel)}</h2>\n<pre>{html.escape(numbered)}</pre>"


def render_html(blocks: list[str]) -> str:
    intro = (
        "<h1>소스 모음</h1>"
        "<p>화면과 빌드 소스를 파일 순서대로 이어 두었습니다. "
        "주석은 코드 옆에 그대로 있습니다. "
        "data/ 과제 코드, 압축된 라이브러리, 빌드가 만든 data.js, hero-images.js, sources JSON은 넣지 않았습니다.</p>"
    )
```

```
    css = """
@page { margin: 12mm 10mm; }
body { font-family: "Malgun Gothic", sans-serif; color: #1c1c1c; }
h1 { font-size: 18pt; margin: 0 0 0.4em; }
h2 { font-size: 12pt; margin: 1.2em 0 0.3em; page-break-before: always; page-break-after: avoid; }
h2:first-of-type { page-break-before: auto; }
p { font-size: 10.5pt; line-height: 1.5; }
pre { font-family: Consolas, "Malgun Gothic", monospace; font-size: 8pt; line-height: 1.35;
      white-space: pre-wrap; word-break: break-all; margin: 0; }
"""
    return "\n".join(
        [
            "<!DOCTYPE html>",
            '<html lang="ko">',
            "<head>",
            '<meta charset="utf-8" />',
            "<title>소스 모음</title>",
            "<style>",
            css,
            "</style>",
            "</head>",
            "<body>",
            intro,
            "\n".join(blocks),
            "</body>",
            "</html>",
        ]
    )


def find_browser() -> Path | None:
    local = os.environ.get("LOCALAPPDATA", "")
    candidates = [
        Path(r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe"),
        Path(r"C:\Program Files\Microsoft\Edge\Application\msedge.exe"),
        Path(r"C:\Program Files\Google\Chrome\Application\chrome.exe"),
        Path(local) / "Google" / "Chrome" / "Application" / "chrome.exe",
    ]
    for candidate in candidates:
        if candidate.is_file():
            return candidate
    return None


def write_pdf(html_path: Path, pdf_path: Path) -> None:
    browser = find_browser()
    if browser is None:
        raise SystemExit("PDF로 인쇄할 Edge 또는 Chrome을 찾지 못했습니다.")
    if pdf_path.exists():
        pdf_path.unlink()
    profile = Path(tempfile.mkdtemp(prefix="source-bundle-"))
    command = [
        str(browser),
        "--headless=new",
        "--disable-gpu",
        "--no-first-run",
        "--no-pdf-header-footer",
        f"--user-data-dir={profile}",
        f"--print-to-pdf={pdf_path.resolve()}",
        html_path.resolve().as_uri(),
    ]
    try:
        subprocess.run(command, check=True, timeout=180)
    finally:
        shutil.rmtree(profile, ignore_errors=True)
    if not pdf_path.is_file() or pdf_path.stat().st_size < 1000:
        raise SystemExit(f"PDF가 만들어지지 않았습니다: {pdf_path}")
```

```


def main() -> None:
    files = source_files()
    if not files:
        raise SystemExit("묶을 소스 파일이 없습니다.")
    HTML_PATH.write_text(render_html([file_block(path) for path in files]), encoding="utf-8")
    write_pdf(HTML_PATH, PDF_PATH)
    print(f"Files {len(files)}")
    print(f"Wrote {PDF_PATH} ({PDF_PATH.stat().st_size // 1024} KB)")


if __name__ == "__main__":
    main()
```

docs/build_specs.py

```python
# -*- coding: utf-8 -*-
"""기술명세 마크다운을 이어 한글 PDF로 만든다.

Mermaid 차트는 Edge로 그린 뒤 잘라서 PNG로 넣는다.
"""
from __future__ import annotations

import hashlib
import html
import os
import re
import shutil
import subprocess
import tempfile
import time
from pathlib import Path

from PIL import Image, ImageChops

ROOT = Path(__file__).resolve().parent
SPECS = ROOT / "specs"
DIAGRAMS = SPECS / "diagrams"
MD_PATH = ROOT / "technical-specs.md"
HTML_PATH = ROOT / "technical-specs.html"
PDF_PATH = ROOT / "technical-specs.pdf"
USED_DIAGRAMS: set[str] = set()

INLINE_CODE = re.compile(r"`([^`]+)`")
BOLD = re.compile(r"\*\*(.+?)\*\*")


def inline(text: str) -> str:
    escaped = html.escape(text)
    escaped = INLINE_CODE.sub(lambda match: f"<code>{match.group(1)}</code>", escaped)
    escaped = BOLD.sub(lambda match: f"<strong>{match.group(1)}</strong>", escaped)
    return escaped


def markdown_to_html(text: str) -> str:
    lines = text.splitlines()
    parts: list[str] = []
    index = 0
    while index < len(lines):
        line = lines[index]
        if line.startswith("```"):
            language = line[3:].strip()
            body: list[str] = []
            index += 1
            while index < len(lines) and not lines[index].startswith("```"):
                body.append(lines[index])
                index += 1
            index += 1
            source = "\n".join(body).strip() + "\n"
            if language == "mermaid":
                parts.append(render_diagram(source))
            else:
                parts.append(f'<pre class="code">{html.escape(source)}</pre>')
            continue
        if not line.strip():
            index += 1
            continue
        if line.startswith("#"):
            level = len(line) - len(line.lstrip("#"))
```

```
            level = min(max(level, 1), 4)
            title = line[level:].strip()
            parts.append(f"<h{level}>{inline(title)}</h{level}>")
            index += 1
            continue
        if line.startswith("- "):
            items: list[str] = []
            while index < len(lines) and lines[index].startswith("- "):
                items.append(f"<li>{inline(lines[index][2:].strip())}</li>")
                index += 1
            parts.append("<ul>" + "".join(items) + "</ul>")
            continue
        ordered = re.match(r"\d+\.\s+", line)
        if ordered:
            items = []
            while index < len(lines) and re.match(r"\d+\.\s+", lines[index]):
                item_text = re.sub(r"^\d+\.\s+", "", lines[index]).strip()
                items.append(f"<li>{inline(item_text)}</li>")
                index += 1
            parts.append("<ol>" + "".join(items) + "</ol>")
            continue
        paragraph = [line.strip()]
        index += 1
        while (
            index < len(lines)
            and lines[index].strip()
            and not lines[index].startswith(("#", "- ", "```"))
            and not re.match(r"\d+\.\s+", lines[index])
        ):
            paragraph.append(lines[index].strip())
            index += 1
        parts.append(f"<p>{inline(' '.join(paragraph))}</p>")
    return "\n".join(parts)


def combined_markdown() -> str:
    files = sorted(SPECS.glob("*.md"))
    if not files:
        raise SystemExit(f"명세 마크다운이 없습니다: {SPECS}")
    chunks = [path.read_text(encoding="utf-8").strip() for path in files]
    return "\n\n".join(chunks) + "\n"


HTML_CSS = """
@page { margin: 18mm 16mm; }
body { font-family: "Malgun Gothic", sans-serif; font-size: 11pt; line-height: 1.55; color: #1c1c1c; }
h1 { font-size: 20pt; margin: 0 0 0.6em; page-break-before: always; }
h1:first-of-type { page-break-before: auto; }
h2 { font-size: 15pt; margin: 1.2em 0 0.4em; page-break-after: avoid; }
h3 { font-size: 12.5pt; margin: 1em 0 0.3em; page-break-after: avoid; }
p, li { orphans: 3; widows: 3; }
ul { margin: 0.2em 0 0.8em; padding-left: 1.2em; }
code { font-family: Consolas, "Malgun Gothic", monospace; font-size: 9.5pt; }
pre { font-family: Consolas, "Malgun Gothic", monospace; font-size: 9pt; background: #f3f3f3;
      padding: 8px 10px; white-space: pre-wrap; }
figure.chart { margin: 0.4em 0 1em; text-align: center; page-break-inside: avoid; break-inside: avoid; }
figure.chart img { max-width: 100%; max-height: 230mm; width: auto; height: auto; }
"""


def render_html(body: str) -> str:
    return "\n".join(
        [
            "<!DOCTYPE html>",
            '<html lang="ko">',
            "<head>",
            '<meta charset="utf-8" />',
```

```
            "<title>기술명세</title>",
            "<style>",
            HTML_CSS,
            "</style>",
            "</head>",
            "<body>",
            body,
            "</body>",
            "</html>",
        ]
    )


def find_browser() -> Path | None:
    local = os.environ.get("LOCALAPPDATA", "")
    candidates = [
        Path(r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe"),
        Path(r"C:\Program Files\Microsoft\Edge\Application\msedge.exe"),
        Path(r"C:\Program Files\Google\Chrome\Application\chrome.exe"),
        Path(local) / "Google" / "Chrome" / "Application" / "chrome.exe",
    ]
    for candidate in candidates:
        if candidate.is_file():
            return candidate
    return None


ACTOR_LINE = re.compile(r"^(\w+)\[(.+)\]$")
CASE_LINE = re.compile(r"^(\w+)\(\[(.+)\]\)$")
BOUNDARY_LINE = re.compile(r"^subgraph\s+\w+\[(.+)\]$")
ASSOC_LINE = re.compile(r"^(\w+)\s+---\s+(\w+)$")
EXTEND_LINE = re.compile(r"^(\w+)\s+-\.->\|([^|]+)\|\s+(\w+)$")
BREAK_LINE = re.compile(r"<br\s*/?>")


def parse_usecase(source: str) -> tuple[str, list, list, list, list]:
    actors: list[tuple[str, str]] = []
    cases: list[tuple[str, str]] = []
    assocs: list[tuple[str, str]] = []
    extends: list[tuple[str, str, str]] = []
    boundary = "시스템"
    for raw in source.splitlines():
        line = raw.strip()
        if not line or line.startswith(("%%", "flowchart", "direction")) or line == "end":
            continue
        found = BOUNDARY_LINE.match(line)
        if found:
            boundary = found.group(1)
            continue
        found = CASE_LINE.match(line)
        if found:
            cases.append((found.group(1), found.group(2)))
            continue
        found = ACTOR_LINE.match(line)
        if found:
            actors.append((found.group(1), found.group(2)))
            continue
        found = ASSOC_LINE.match(line)
        if found:
            assocs.append((found.group(1), found.group(2)))
            continue
        found = EXTEND_LINE.match(line)
        if found:
            extends.append((found.group(1), found.group(2), found.group(3)))
    if not actors or not cases:
        raise SystemExit("사람 그림으로 그릴 행위자나 유스케이스가 없습니다.")
    return boundary, actors, cases, assocs, extends
```

```

 def text_width(text: str) -> int:
     return sum(17 if ord(char) > 127 else 9 for char in text)


 def usecase_svg(source: str) -> str:
     boundary, actors, cases, assocs, extends = parse_usecase(source)
     parsed = [(case_id, BREAK_LINE.split(label)) for case_id, label in cases]
     ellipse_w = max(max(text_width(line) for line in lines) for _, lines in parsed) + 52
     row_h = 78
     ellipse_h = 50
     pad_top = 40
     pad_x = 48
     boundary_x = 188
     boundary_y = 10
     extend_room = 28
     boundary_w = pad_x + ellipse_w + pad_x + extend_room
     boundary_h = pad_top + row_h * len(parsed) + 8
     ellipse_x = boundary_x + pad_x
     centers: dict[str, float] = {}
     for index, (case_id, _) in enumerate(parsed):
         centers[case_id] = boundary_y + pad_top + index * row_h + ellipse_h / 2
     linked: dict[str, list[str]] = {}
     for actor_id, case_id in assocs:
         linked.setdefault(actor_id, []).append(case_id)

     def actor_anchor(actor_id: str) -> float:
         targets = linked.get(actor_id, [])
         if not targets:
             return boundary_y + boundary_h / 2
         return sum(centers[case_id] for case_id in targets) / len(targets)

     width = boundary_x + boundary_w + 20
     height = boundary_y + boundary_h + 28
     parts = [
         f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}" viewBox="0 0 {width} {height}">',
         "<defs><marker id='extend-arrow' markerWidth='8' markerHeight='8' refX='7' refY='4' orient='auto'>",
         "<path d='M0,0 L8,4 L0,8 Z' fill='#555'/></marker></defs>",
         f"<rect x='{boundary_x}' y='{boundary_y}' width='{boundary_w}' height='{boundary_h}' fill='none' stroke='#9a9a
9a' stroke-width='1.4'/>",
         f"<text x='{boundary_x + boundary_w / 2}' y='{boundary_y + 24}' text-anchor='middle' font-size='15' fill='#33
3'>{html.escape(boundary)}</text>",
     ]
     hands: dict[str, tuple[float, float]] = {}
     for index, (actor_id, name) in enumerate(actors):
         center_x = 78
         center_y = actor_anchor(actor_id)
         radius = 13
         head_y = center_y - 34
         neck = head_y + radius
         hip = neck + 30
         arm_y = neck + 12
         foot = hip + 24
         arm = 20
         leg = 15
         parts.append(
             f"<g fill='none' stroke='#222' stroke-width='1.7' stroke-linecap='round'>"
             f"<circle cx='{center_x}' cy='{head_y}' r='{radius}'/>"
             f"<line x1='{center_x}' y1='{neck}' x2='{center_x}' y2='{hip}'/>"
             f"<line x1='{center_x - arm}' y1='{arm_y}' x2='{center_x + arm}' y2='{arm_y}'/>"
             f"<line x1='{center_x}' y1='{hip}' x2='{center_x - leg}' y2='{foot}'/>"
             f"<line x1='{center_x}' y1='{hip}' x2='{center_x + leg}' y2='{foot}'/>"
             f"</g>"
             f"<text x='{center_x}' y='{foot + 20}' text-anchor='middle' font-size='15' fill='#222'>{html.escape(name)}
</text>"
         )
```

```
        hands[actor_id] = (center_x + arm, arm_y)
    for actor_id, case_id in assocs:
        start_x, start_y = hands[actor_id]
        end_x = ellipse_x
        end_y = centers[case_id]
        bend = (start_x + end_x) / 2
        parts.append(
            f"<path d='M {start_x} {start_y} C {bend} {start_y}, {bend} {end_y}, {end_x} {end_y}' fill='none' stroke
='#777' stroke-width='1.3'/>"
        )
    right = ellipse_x + ellipse_w
    for source_id, label, target_id in extends:
        start_y = centers[source_id]
        end_y = centers[target_id]
        lane = right + 16
        mid_y = (start_y + end_y) / 2
        parts.append(
            f"<path d='M {right} {start_y} C {lane} {start_y}, {lane} {end_y}, {right} {end_y}' fill='none' stroke='#5
55' stroke-width='1.3' stroke-dasharray='4 3' marker-end='url(#extend-arrow)'/>"
            f"<text x='{lane + 8}' y='{mid_y + 4}' font-size='13' fill='#333'>{html.escape(label)}</text>"
        )
    for case_id, lines in parsed:
        center_y = centers[case_id]
        parts.append(
            f"<ellipse cx='{ellipse_x + ellipse_w / 2}' cy='{center_y}' rx='{ellipse_w / 2}' ry='{ellipse_h / 2}' fill
='#f4f4f4' stroke='#666' stroke-width='1.3'/>"
        )
        if len(lines) == 1:
            parts.append(
                f"<text x='{ellipse_x + ellipse_w / 2}' y='{center_y + 5}' text-anchor='middle' font-size='15' fill='#
222'>{html.escape(lines[0])}</text>"
            )
        else:
            first = center_y - 6
            spans = "".join(
                f"<tspan x='{ellipse_x + ellipse_w / 2}' y='{first + offset * 18}'>{html.escape(line)}</tspan>"
                for offset, line in enumerate(lines)
            )
            parts.append(f"<text text-anchor='middle' font-size='15' fill='#222'>{spans}</text>")
    parts.append("</svg>")
    return "".join(parts)


def svg_page(svg: str) -> str:
    return f"""<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="utf-8" />
<style>
  html, body {{ margin: 0; background: #fff; }}
  body {{ padding: 20px; font-family: "Malgun Gothic", sans-serif; }}
  svg {{ font-family: "Malgun Gothic", sans-serif; }}
</style>
</head>
<body>
{svg}
</body>
</html>
"""


def diagram_page(source: str) -> str:
    if "actor-style: person" in source:
        return svg_page(usecase_svg(source))
    escaped = html.escape(source)
    return f"""<!DOCTYPE html>
<html lang="ko">
```

```
<head>
<meta charset="utf-8" />
<style>
  html, body {{ margin: 0; background: #fff; }}
  body {{ padding: 24px; }}
  .mermaid {{ font-family: "Malgun Gothic", sans-serif; }}
</style>
</head>
<body>
<pre class="mermaid">{escaped}</pre>
<script type="module">
  import mermaid from "https://cdn.jsdelivr.net/npm/mermaid@11/dist/mermaid.esm.min.mjs";
  mermaid.initialize({{ startOnLoad: true, theme: "neutral", fontFamily: "Malgun Gothic, sans-serif" }});
</script>
</body>
</html>
"""


def crop_whitespace(path: Path) -> None:
    image = Image.open(path).convert("RGB")
    background = Image.new("RGB", image.size, (255, 255, 255))
    box = ImageChops.difference(image, background).getbbox()
    if box is None:
        raise SystemExit(f"차트가 비어 있습니다: {path.name}")
    pad = 16
    left = max(box[0] - pad, 0)
    top = max(box[1] - pad, 0)
    right = min(box[2] + pad, image.width)
    bottom = min(box[3] + pad, image.height)
    cropped = image.crop((left, top, right, bottom))
    if cropped.width < 40 or cropped.height < 40:
        raise SystemExit(f"차트가 너무 작습니다: {path.name}")
    cropped.save(path)


def render_diagram(source: str) -> str:
    digest = hashlib.sha256(source.encode("utf-8")).hexdigest()[:12]
    USED_DIAGRAMS.add(digest)
    DIAGRAMS.mkdir(parents=True, exist_ok=True)
    png = DIAGRAMS / f"{digest}.png"
    if not png.is_file():
        browser = find_browser()
        if browser is None:
            raise SystemExit("차트를 그릴 Edge 또는 Chrome을 찾지 못했습니다.")
        html_path = DIAGRAMS / f"{digest}.html"
        html_path.write_text(diagram_page(source), encoding="utf-8")
        profile = Path(tempfile.mkdtemp(prefix="specs-chart-"))
        command = [
            str(browser),
            "--headless=new",
            "--disable-gpu",
            "--no-first-run",
            "--hide-scrollbars",
            "--force-device-scale-factor=2",
            "--virtual-time-budget=12000",
            "--window-size=1400,2200",
            f"--user-data-dir={profile}",
            f"--screenshot={png.resolve()}",
            html_path.resolve().as_uri(),
        ]
        try:
            subprocess.run(command, check=True, timeout=60)
            deadline = time.time() + 8
            while not png.is_file() and time.time() < deadline:
                time.sleep(0.2)
        finally:
```

```
            html_path.unlink(missing_ok=True)
            shutil.rmtree(profile, ignore_errors=True)
        if not png.is_file():
            raise SystemExit(f"차트 이미지가 만들어지지 않았습니다: {png.name}")
        crop_whitespace(png)
    relative = png.relative_to(ROOT).as_posix()
    return f'<figure class="chart"><img src="{html.escape(relative)}" alt="차트" /></figure>'


def cleanup_diagrams() -> None:
    if not DIAGRAMS.is_dir():
        return
    for path in DIAGRAMS.iterdir():
        if path.is_dir():
            shutil.rmtree(path, ignore_errors=True)
        elif path.stem not in USED_DIAGRAMS:
            path.unlink()


def write_pdf(html_path: Path, pdf_path: Path) -> None:
    browser = find_browser()
    if browser is None:
        raise SystemExit("PDF로 인쇄할 Edge 또는 Chrome을 찾지 못했습니다.")
    if pdf_path.exists():
        pdf_path.unlink()
    profile = Path(tempfile.mkdtemp(prefix="specs-pdf-"))
    command = [
        str(browser),
        "--headless=new",
        "--disable-gpu",
        "--no-first-run",
        "--no-pdf-header-footer",
        f"--user-data-dir={profile}",
        f"--print-to-pdf={pdf_path.resolve()}",
        html_path.resolve().as_uri(),
    ]
    try:
        subprocess.run(command, check=True, timeout=120)
    finally:
        shutil.rmtree(profile, ignore_errors=True)
    if not pdf_path.is_file() or pdf_path.stat().st_size < 1000:
        raise SystemExit(f"PDF가 만들어지지 않았습니다: {pdf_path}")


def main() -> None:
    markdown = combined_markdown()
    MD_PATH.write_text(markdown, encoding="utf-8")
    HTML_PATH.write_text(render_html(markdown_to_html(markdown)), encoding="utf-8")
    cleanup_diagrams()
    write_pdf(HTML_PATH, PDF_PATH)
    print(f"Wrote {MD_PATH}")
    print(f"Wrote {HTML_PATH}")
    print(f"Wrote {PDF_PATH} ({PDF_PATH.stat().st_size // 1024} KB)")


if __name__ == "__main__":
    main()
```

index.html

```
<!-- 포트폴리오 메인 화면. 지원 동기, 성장·학습, 역량 절과 자료 메뉴를 담고, 저장된 라이트·다크 테마를 그리기 전에 적용한다. -->
<!DOCTYPE html>
<html lang="ko" data-default-theme="light" data-theme="light">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="description" content="IT·웹·모바일·AI 서비스 분야 취업 지원 자기소개" />
    <title>자기소개서 – IT 취업 지원</title>
    <script>
      (function () {
        try {
          var theme = localStorage.getItem("portfolio-theme-v2");
          if (theme === "dark" || theme === "light") {
            document.documentElement.setAttribute("data-theme", theme);
          }
        } catch (e) {
          /* 테마 저장소를 사용할 수 없는 환경에서는 기본 테마를 사용합니다. */
        }
      })();
    </script>
    <link rel="preconnect" href="https://images.unsplash.com" crossorigin />
    <link rel="dns-prefetch" href="https://images.unsplash.com" />
    <link rel="stylesheet" href="./css/styles.css?v=theme-sync" />
  </head>
  <body>
    <a class="skip-link" href="#main">본문으로 건너뛰기</a>

    <header class="site-header">
      <div class="site-header__inner">
        <a class="site-header__brand" href="./">코탐록</a>
        <nav id="site-nav" class="site-nav" aria-label="페이지 섹션">
          <ul class="site-nav__list">
            <li><a class="site-nav__link" href="#section-motivation">지원 동기</a></li>
            <li><a class="site-nav__link" href="#section-growth">성장·학습</a></li>
            <li><a class="site-nav__link" href="#section-skills">역량</a></li>
            <li class="site-nav__item site-nav__dropdown">
              <button
                type="button"
                class="site-nav__link site-nav__dropdown-toggle"
                aria-expanded="false"
                aria-controls="resource-dropdown"
              >
                자료
                <span class="site-nav__dropdown-caret" aria-hidden="true"></span>
              </button>
              <div id="resource-dropdown" class="site-nav__dropdown-menu" aria-label="자료 카테고리" hidden>
                <a class="site-nav__dropdown-link" href="#section-links">
                  <span class="site-nav__dropdown-title">자료 보기</span>
                  <span class="site-nav__dropdown-desc">현재 페이지의 자료 카테고리로 이동</span>
                </a>
                <a class="site-nav__dropdown-link" href="cover_letter/profile.html">
                  <span class="site-nav__dropdown-title">자기소개 / 프로필</span>
                  <span class="site-nav__dropdown-desc">기본 소개와 학습 경험</span>
                </a>
                <a class="site-nav__dropdown-link" href="online_business_card/business-card.html">
                  <span class="site-nav__dropdown-title">온라인 명함</span>
                  <span class="site-nav__dropdown-desc">3D 스크롤 명함 페이지</span>
                </a>
                <a class="site-nav__dropdown-link" href="cv/resume.html">
                  <span class="site-nav__dropdown-title">이력서</span>
                  <span class="site-nav__dropdown-desc">문서형 이력서 페이지</span>
                </a>
```

```
                <a class="site-nav__dropdown-link" href="study_note_record_organization/dashboard/index.html">
                  <span class="site-nav__dropdown-title">학습·과제 기록</span>
                  <span class="site-nav__dropdown-desc">학습록 아카이브 대시보드</span>
                </a>
              </div>
            </li>
            <li><a class="site-nav__link" href="#section-strengths">강점</a></li>
            <li><a class="site-nav__link" href="#section-closing">마치며</a></li>
            <li><a class="site-nav__link" href="#section-refs">참고</a></li>
          </ul>
        </nav>
        <div class="site-header__actions">
          <button type="button" class="theme-toggle" data-theme-toggle aria-pressed="false" aria-label="다크 테마로 전
환">
            <span class="theme-toggle__label">고대비</span>
            <span class="theme-toggle__state" data-theme-label>라이트</span>
          </button>
          <button
            type="button"
            class="nav-toggle"
            aria-expanded="false"
            aria-controls="site-nav"
            aria-label="메뉴 열기"
          >
            <span class="nav-toggle__bar" aria-hidden="true"></span>
            <span class="nav-toggle__bar" aria-hidden="true"></span>
          </button>
        </div>
      </div>
    </header>

    <main id="main">
      <section class="hero" aria-labelledby="hero-title">
        <div class="hero__inner hero__layout">
          <div class="hero__copy">
            <p class="hero__eyebrow">자기소개서</p>
            <h1 id="hero-title" class="hero__title">IT·웹·모바일·AI 분야,<br />현장에 맞추어 성장하고 있습니다.</h1>
            <dl class="hero__meta">
              <div class="hero__meta-row">
                <dt>작성 목적</dt>
                <dd>IT·웹·모바일·AI 서비스 분야 취업 지원</dd>
              </div>
              <div class="hero__meta-row">
                <dt>작성일</dt>
                <dd>2026년 5월</dd>
              </div>
            </dl>
          </div>
          <figure class="hero__figure media-frame">
            <img
              src="https://images.unsplash.com/photo-1624020399372-ab7d8686b29f?auto=format&amp;fit=crop&amp;w=1400&am
p;q=75&amp;ixlib=rb-4.0.3"
              srcset="
                https://images.unsplash.com/photo-1624020399372-ab7d8686b29f?auto=format&amp;fit=crop&amp;w=800&amp;q=
75&amp;ixlib=rb-4.0.3 800w,
                https://images.unsplash.com/photo-1624020399372-ab7d8686b29f?auto=format&amp;fit=crop&amp;w=1400&amp;q
=75&amp;ixlib=rb-4.0.3 1400w
              "
              sizes="(max-width: 959px) 100vw, 42vw"
              width="1400"
              height="934"
              alt="어두운 공간에서 켜진 노트북 화면 — 개발·학습 장면을 상징하는 배경 이미지"
              decoding="async"
              fetchpriority="high"
              loading="eager"
            />
            <figcaption class="media-caption">
```

```
126                <a href="https://unsplash.com/photos/a-laptop-computer-in-a-dark-room-vSfk-xRtdcM" rel="noopener norefer
rer">Unsplash</a>
127                · David Schultz
128              </figcaption>
129            </figure>
130          </div>
131        </section>
132
133        <div class="section-stack">
134          <section id="section-motivation" class="section" aria-labelledby="heading-motivation">
135            <div class="section__inner prose">
136              <h2 id="heading-motivation" class="section__title">1. 지원 동기 및 입사 후 포부</h2>
137              <p>
138                저는 웹·모바일·AI 서비스를 아우르는 IT·ICT 분야에서
139                <strong>현장에서 바로 쓰이는 기술</strong>을 익히고, 팀의 일원으로서 제 역할을 다하고 싶은
140                구직자입니다.<br />
141                대학 이후 비주얼베이직·통계 도구(SAS, SPSS)로 데이터와 도구에 익숙해진 뒤, 오프라인 교육을
142                통해 컴퓨터공학 기초를 다지고
143                <strong>자바(GUI·AWT·SWING)·코틀린(안드로이드)</strong> 로 실제 동작하는 프로그램을 만들어
144                보았습니다. 추첨부터 당첨 확인까지 이어지는 로또 시뮬레이션, XML 레이아웃 기반의 간단한 두더지
145                게임, 스톱워치 등은 “코드가 사용자 화면과 어떻게 맞닿는지”를 직접 체감한 경험입니다.
146              </p>
147              <p>
148                최근에는 <strong>생성형 AI와 바이브 코딩, 프롬프트 엔지니어링</strong>을 학습하며 개발 속도와
149                아이디어 검증 방식이 바뀌는 흐름에 발을 맞추고 있습니다. 펠로 AI·젠스파크 등으로 강의·필기를
150                정리하고 슬라이드로 재구성해 보며,
151                <strong>AI를 보조 도구로 두고도 결과의 책임과 품질은 사람이 관리해야 한다</strong>는 점을
152                중요하게 생각합니다.
153              </p>
154              <p>
155                입사 후에는 귀사의 개발 프로세스와 코딩 컨벤션에 빠르게 적응하고, 배운 것을 문서화·블로그(네이버
156                코탐록) 운영과 연계해
157                <strong>스스로의 성장 기록을 팀 내 지식 공유에도 도움이 되도록</strong> 확장하고 싶습니다.
158                장기적으로는 웹·모바일·AI 서비스 경계에서
159                <strong>사용자 가치가 드러나는 제품</strong>을 만드는 데 기여하는 실무자가 되고자 합니다.
160              </p>
161            </div>
162          </section>
163
164          <section id="section-growth" class="section" aria-labelledby="heading-growth">
165            <div class="section__inner prose">
166              <h2 id="heading-growth" class="section__title">2. 성장 과정과 학습 태도</h2>
167              <figure class="section-figure media-frame">
168                <img
169                  src="https://images.unsplash.com/photo-1489875347897-49f64b51c1f8?auto=format&amp;fit=crop&amp;w=1200&
amp;q=75&amp;ixlib=rb-4.0.3"
170                  srcset="
171                    https://images.unsplash.com/photo-1489875347897-49f64b51c1f8?auto=format&amp;fit=crop&amp;w=720&amp;
q=75&amp;ixlib=rb-4.0.3 720w,
172                    https://images.unsplash.com/photo-1489875347897-49f64b51c1f8?auto=format&amp;fit=crop&amp;w=1200&am
p;q=75&amp;ixlib=rb-4.0.3 1200w
173                  "
174                  sizes="(max-width: 70ch) 100vw, 70ch"
175                  width="1200"
176                  height="800"
177                  alt="어두운 톤의 노트북 화면에 소스 코드가 표시된 모습"
178                  decoding="async"
179                  loading="lazy"
180                />
181                <figcaption class="media-caption">
182                  <a href="https://unsplash.com/photos/macbook-pro-with-images-of-computer-language-codes-fPkvU7RDmCo" r
el="noopener noreferrer">Unsplash</a>
183                  · Caspar Camille Rubin
184                </figcaption>
185              </figure>
186              <p>
187                저의 학습은 “한 번에 완벽하게”보다
```

```
            <strong>책·강의·작은 프로젝트를 반복하며 층을 쌓는 방식</strong>에 가깝습니다. 페스트캠퍼스,
            코드잇, 인프런, 유데미, KMOOC, 경기지식 등 여러 플랫폼을 병행하며 같은 주제라도 설명 방식이 다른
            자료를 비교하는 습관이 있습니다. 학점은행제를 통해 학력 체계를 정리하고 있으며,
            <strong>지금까지의 비정규 경로의 학습도 앞으로의 정규 학습과 연결해 일관된 스토리</strong>로
            만들고자 합니다.
          </p>
          <p>
            취미로도 프로그래밍을 이어 왔습니다. 레인미터로 바탕화면을 정리하거나, Visual .NET으로 텍스트
            파일 병합 도구를 만들어 보는 등 <strong>일상의 불편을 작은 도구로 줄이는 것</strong>에 흥미가
            있습니다. 멀티미디어·디지털 콘텐츠에도 관심이 있어 블로그를 체계적으로 운영할 예정이며 기술·독서·프로젝트
            백업을 정리하고 있습니다.
          </p>
        </div>
      </section>

      <section id="section-skills" class="section" aria-labelledby="heading-skills">
        <div class="section__inner">
          <h2 id="heading-skills" class="section__title section__title--flush">3. 보유 역량 및 실습 경험 (요약)</h2>
          <ul class="skill-cards" aria-label="역량 요약 카드">
            <li class="skill-card">
              <h3 class="skill-card__label">언어·환경</h3>
              <p class="skill-card__body">Java (AWT, Swing, GUI), Kotlin (Android, XML 레이아웃)</p>
            </li>
            <li class="skill-card">
              <h3 class="skill-card__label">프로젝트 성격</h3>
              <p class="skill-card__body">로또 추첨·당첨 로직, 미니 게임, 스톱워치, 파일 처리 유틸 등</p>
            </li>
            <li class="skill-card">
              <h3 class="skill-card__label">도구·흐름</h3>
              <p class="skill-card__body">생성형 AI 활용 개발, 프롬프트 설계, 필기·녹화 자료의 슬라이드화</p>
            </li>
            <li class="skill-card">
              <h3 class="skill-card__label">기타</h3>
              <p class="skill-card__body">통계·데이터 도구(SAS, SPSS) 경험, CS 기초 교육 이수</p>
            </li>
          </ul>
          <div class="prose prose--after-cards">
            <p>
              현재는 <strong>이력서·자소서에 담을 수 있도록</strong> 백업된 프로젝트와 독서 노트를
              재정리하며, 웹·모바일·AI 서비스로 역량 스택을 넓히는 데 집중하고 있습니다.
            </p>
          </div>
        </div>
      </section>

      <section id="section-links" class="section" aria-labelledby="heading-links">
        <div class="section__inner">
          <h2 id="heading-links" class="section__title section__title--flush">자료 카테고리</h2>
          <p class="section-lead">
            자기소개 페이지를 기준으로, 목적별 지원 자료 네 가지를 하위 페이지로 분류했습니다.
          </p>
          <div class="resource-cards" aria-label="연결 자료 카테고리">
            <a class="resource-card" href="cover_letter/profile.html">
              <span class="resource-card__eyebrow">Profile</span>
              <h3 class="resource-card__title">자기소개 / 프로필</h3>
              <p class="resource-card__body">
                기본 소개, 관심 분야, 학습 경험, 지원 동기와 생활신조를 포트폴리오 형식으로 정리했습니다.
              </p>
              <span class="resource-card__path">cover_letter/profile.html</span>
            </a>
            <a class="resource-card" href="online_business_card/business-card.html">
              <span class="resource-card__eyebrow">Business Card</span>
              <h3 class="resource-card__title">온라인 명함</h3>
              <p class="resource-card__body">
                코탐록 콘셉트의 3D 스크롤 명함과 자기소개 흐름을 결합한 인터랙티브 페이지입니다.
              </p>
              <span class="resource-card__path">online_business_card/business-card.html</span>
```

```
            </a>
            <a class="resource-card" href="cv/resume.html">
              <span class="resource-card__eyebrow">Resume</span>
              <h3 class="resource-card__title">이력서</h3>
              <p class="resource-card__body">
                경력, 학력, 기타 항목을 문서형 레이아웃으로 확인할 수 있는 이력서 페이지입니다.
              </p>
              <span class="resource-card__path">cv/resume.html</span>
            </a>
            <a class="resource-card" href="study_note_record_organization/dashboard/index.html">
              <span class="resource-card__eyebrow">Study Archive</span>
              <h3 class="resource-card__title">학습·과제 기록 아카이브</h3>
              <p class="resource-card__body">
                독서·강의·과제 노트를 요약해 검색하고, 원본 텍스트까지 이어서 볼 수 있는 학습록 대시보드입니다.
              </p>
              <span class="resource-card__path">study_note_record_organization/dashboard/index.html</span>
            </a>
          </div>
        </div>
      </section>

      <section id="section-strengths" class="section" aria-labelledby="heading-strengths">
        <div class="section__inner prose">
          <h2 id="heading-strengths" class="section__title">4. 강점과 보완하려는 점</h2>
          <p>
            <strong>강점</strong>으로는 (1) 혼자서도
            <strong>장기간 학습·실습을 이어가는 꾸준함</strong>, (2)
            <strong>도구와 언어를 바꿔 가며</strong> 문제를 구현해 본 경험, (3) AI 시대에 맞춰
            <strong>학습·정리 방식을 스스로 업데이트</strong>하려는 태도를 꼽을 수 있습니다.
          </p>
          <p>
            한편 <strong>사교성</strong>과 <strong>새로운 도전을 먼저 제안하는 성향</strong>은 약점으로
            인지하고 있습니다. 이에 따라 스터디·오픈소스 이슈 참여 등
            <strong>작은 단위의 대외 활동</strong>부터 범위를 넓히고, 팀 내에서는
            <strong>문서화·코드 리뷰 의견</strong>처럼 비대면으로도 기여할 수 있는 채널부터 소통량을 늘리려
            합니다. 귀사에서도 초기에는 맡은 역할에서 <strong>신뢰를 쌓는 것</strong>을 최우선으로 하고,
            점진적으로 협업 범위를 넓히고 싶습니다.
          </p>
        </div>
      </section>

      <section id="section-closing" class="section" aria-labelledby="heading-closing">
        <div class="section__inner prose">
          <h2 id="heading-closing" class="section__title">5. 마치며</h2>
          <p>
            37세라는 나이는 늦은 출발처럼 보일 수 있지만, 저에게는
            <strong>다양한 도구와 자기주도 학습으로 기초와 응용을 직접 연결해 온 시간</strong>입니다. 귀사에서
            웹·모바일·AI·IT 현장 중 어느 한 축이라도 맡겨 주신다면, 겸손한 자세로 프로세스를 배우고
            <strong>재사용 가능한 코드와 기록 가능한 학습</strong>으로 보답하겠습니다.
          </p>
          <p class="closing-thanks">감사합니다.</p>
        </div>
      </section>

      <section id="section-refs" class="section section--footer-refs" aria-labelledby="heading-refs">
        <div class="section__inner prose">
          <h2 id="heading-refs" class="section__title">첨부·참고 (선택 기재용)</h2>
          <ul class="refs-list">
            <li><span class="refs-list__term">블로그</span> <a href="https://blog.naver.com/fifo23">네이버 블로그 코탐록</a></li>
            <li><span class="refs-list__term">관심 분야</span> 모바일, 웹, AI 서비스, IT/ICT 일반</li>
            <li><span class="refs-list__term">학력</span> 대졸, 학점은행제 진행 중</li>
          </ul>
          <p class="caption">
            본 문서는 지원 기업·직무에 맞게 문단 순서와 강조점을 조정해 사용하시기 바랍니다.
          </p>
        </div>
```

```
        </section>
      </div>
    </main>

    <footer class="site-footer">
      <div class="site-footer__inner">
        <p class="site-footer__note">
          2026년 5월 25일 제작 · Cursor AI를 사용해 제작했습니다.
        </p>
      </div>
    </footer>

    <script src="./js/main.js?v=theme-sync" defer></script>
  </body>
</html>
```

js/main.js

```
/**
 * 포트폴리오 메인 페이지의 내비게이션, 자료 드롭다운, 테마 전환을 초기화한다.
 * DOM이 준비되면 부드러운 앵커 이동, 모바일 메뉴, 섹션 포커스를 연결한다.
 */
(function () {
  "use strict";

  var mqReduceMotion =
    typeof window.matchMedia === "function" ? window.matchMedia("(prefers-reduced-motion: reduce)") : null;
  var THEME_KEY = "portfolio-theme-v2";

  /**
   * 사용자가 동작 줄이기를 켰는지 확인한다.
   * 켜져 있으면 스크롤을 즉시 이동하고, 아니면 부드럽게 이동한다.
   * @returns {boolean}
   */
  function prefersReducedMotion() {
    return mqReduceMotion ? mqReduceMotion.matches : false;
  }

  /**
   * 문서 전체에서 선택자와 맞는 요소를 배열로 돌려준다.
   * @param {string} selector CSS 선택자
   * @returns {Element[]}
   */
  function queryAll(selector) {
    return Array.prototype.slice.call(document.querySelectorAll(selector));
  }

  /**
   * 지정한 요소 안에서만 선택자와 맞는 요소를 배열로 돌려준다.
   * @param {Element} root 검색을 시작할 요소
   * @param {string} selector CSS 선택자
   * @returns {Element[]}
   */
  function queryAllIn(root, selector) {
    return Array.prototype.slice.call(root.querySelectorAll(selector));
  }

  /**
   * 상단 메뉴의 페이지 안 링크를 부드러운 스크롤로 연결한다.
   * 햄버거 버튼으로 모바일 메뉴를 열고, Escape와 넓은 화면에서는 메뉴를 닫는다.
   */
  function initSmoothNav() {
    var nav = document.getElementById("site-nav");
    var toggle = document.querySelector(".nav-toggle");

    queryAll(".site-nav__link").forEach(function (link) {
      var href = link.getAttribute("href") || "";
      if (!href.startsWith("#") || href.length < 2) return;

      link.addEventListener("click", function (e) {
        var target = document.getElementById(href.slice(1));
        if (!target) return;
        e.preventDefault();
        target.scrollIntoView({ behavior: prefersReducedMotion() ? "auto" : "smooth", block: "start" });
        closeMobileNav(nav, toggle);
        target.focus({ preventScroll: true });
      });
    });

    if (toggle && nav) {
      toggle.addEventListener("click", function () {
```

```
        var open = !nav.classList.contains("is-open");
        setMobileNav(nav, toggle, open);
      });
    }

    document.addEventListener("keydown", function (e) {
      if (e.key === "Escape" && nav && nav.classList.contains("is-open")) {
        closeMobileNav(nav, toggle);
        if (toggle) toggle.focus();
      }
    });

    window.addEventListener(
      "resize",
      debounce(function () {
        if (window.innerWidth > 768 && nav && toggle) {
          closeMobileNav(nav, toggle);
        }
      }, 150),
    );
  }

  /**
   * 자료 메뉴 드롭다운의 열기, 항목 이동, 바깥 클릭·Escape 닫기를 연결한다.
   * 항목이 페이지 안 링크이면 해당 구역으로 스크롤한 뒤 메뉴를 닫는다.
   */
  function initResourceDropdown() {
    var nav = document.getElementById("site-nav");
    var navToggle = document.querySelector(".nav-toggle");

    queryAll(".site-nav__dropdown").forEach(function (dropdown) {
      var dropdownToggle = dropdown.querySelector(".site-nav__dropdown-toggle");

      if (!dropdownToggle) return;

      dropdownToggle.addEventListener("click", function (e) {
        e.preventDefault();
        var open = dropdownToggle.getAttribute("aria-expanded") !== "true";
        closeResourceDropdowns(dropdown);
        setResourceDropdown(dropdown, open);
      });

      queryAllIn(dropdown, ".site-nav__dropdown-link").forEach(function (link) {
        link.addEventListener("click", function (e) {
          var href = link.getAttribute("href") || "";

          if (href.startsWith("#") && href.length > 1) {
            var target = document.getElementById(href.slice(1));
            if (target) {
              e.preventDefault();
              target.scrollIntoView({ behavior: prefersReducedMotion() ? "auto" : "smooth", block: "start" });
              target.focus({ preventScroll: true });
            }
          }

          closeResourceDropdowns();
          closeMobileNav(nav, navToggle);
        });
      });
    });

    document.addEventListener("click", function (e) {
      var target = e.target;
      if (target instanceof Element && target.closest(".site-nav__dropdown")) return;
      closeResourceDropdowns();
    });

```

```
    document.addEventListener("keydown", function (e) {
      if (e.key === "Escape") {
        closeResourceDropdowns();
      }
    });
  }

  /**
   * 저장된 테마를 적용하고, 버튼과 다른 탭의 저장소 변경으로 라이트·다크를 바꾼다.
   */
  function initThemeToggle() {
    var toggle = document.querySelector("[data-theme-toggle]");
    if (!toggle) return;

    var storedTheme = getStoredTheme();
    var theme = storedTheme || document.documentElement.getAttribute("data-default-theme") || "dark";

    applyTheme(theme, toggle);

    toggle.addEventListener("click", function () {
      var current = document.documentElement.getAttribute("data-theme") === "light" ? "light" : "dark";
      var next = current === "light" ? "dark" : "light";
      setStoredTheme(next);
      applyTheme(next, toggle);
    });

    window.addEventListener("storage", function (e) {
      if (e.key !== THEME_KEY) return;
      var syncedTheme = e.newValue || document.documentElement.getAttribute("data-default-theme") || "dark";
      applyTheme(syncedTheme, toggle);
    });
  }

  /**
   * html 요소와 토글 버튼에 라이트 또는 다크 테마 상태와 안내 문구를 반영한다.
   * @param {"dark" | "light" | string} theme 적용할 테마. light가 아니면 dark로 둔다.
   * @param {Element} toggle 테마 전환 버튼
   */
  function applyTheme(theme, toggle) {
    var normalizedTheme = theme === "light" ? "light" : "dark";
    var label = toggle.querySelector("[data-theme-label]");
    document.documentElement.setAttribute("data-theme", normalizedTheme);
    toggle.setAttribute("aria-pressed", normalizedTheme === "dark" ? "true" : "false");
    toggle.setAttribute("aria-label", normalizedTheme === "dark" ? "라이트 테마로 전환" : "다크 테마로 전환");

    if (label) {
      label.textContent = normalizedTheme === "dark" ? "다크" : "라이트";
    }
  }

  /**
   * 브라우저에 저장된 테마 값을 읽는다. 저장소를 쓸 수 없으면 null을 돌려준다.
   * @returns {string|null}
   */
  function getStoredTheme() {
    try {
      return localStorage.getItem(THEME_KEY);
    } catch (e) {
      return null;
    }
  }

  /**
   * 선택한 테마를 브라우저 저장소에 남긴다. 저장이 막히면 현재 페이지만 유지한다.
   * @param {string} theme 저장할 테마 이름
   */
  function setStoredTheme(theme) {
```

```
    try {
      localStorage.setItem(THEME_KEY, theme);
    } catch (e) {
      /* localStorage가 제한된 환경에서는 현재 페이지에만 적용합니다. */
    }
  }

  /**
   * 자료 드롭다운 하나의 열림 상태, 버튼 표시, 메뉴 숨김을 맞춘다.
   * @param {Element} dropdown 드롭다운 루트
   * @param {boolean} open 열지 여부
   */
  function setResourceDropdown(dropdown, open) {
    var toggle = dropdown.querySelector(".site-nav__dropdown-toggle");
    var menu = dropdown.querySelector(".site-nav__dropdown-menu");
    dropdown.classList.toggle("is-open", open);
    if (toggle) toggle.setAttribute("aria-expanded", open ? "true" : "false");
    if (menu) menu.hidden = !open;
  }

  /**
   * 지정한 하나를 제외한 자료 드롭다운을 모두 닫는다.
   * @param {Element=} except 닫지 않을 드롭다운
   */
  function closeResourceDropdowns(except) {
    queryAll(".site-nav__dropdown").forEach(function (dropdown) {
      if (except && dropdown === except) return;
      setResourceDropdown(dropdown, false);
    });
  }

  /**
   * 모바일 내비게이션을 닫고, 함께 열려 있던 자료 드롭다운도 닫는다.
   * @param {HTMLElement | null} nav 내비게이션 요소
   * @param {HTMLElement | null} toggle 햄버거 버튼
   */
  function closeMobileNav(nav, toggle) {
    setMobileNav(nav, toggle, false);
  }

  /**
   * 모바일 내비게이션의 열림 클래스와 버튼 안내 문구를 설정한다.
   * 닫을 때는 자료 드롭다운도 함께 닫는다.
   * @param {HTMLElement | null} nav 내비게이션 요소
   * @param {HTMLElement | null} toggle 햄버거 버튼
   * @param {boolean} open 열지 여부
   */
  function setMobileNav(nav, toggle, open) {
    if (!nav || !toggle) return;
    nav.classList.toggle("is-open", open);
    toggle.setAttribute("aria-expanded", open ? "true" : "false");
    toggle.setAttribute("aria-label", open ? "메뉴 닫기" : "메뉴 열기");
    if (!open) closeResourceDropdowns();
  }

  /**
   * 연속 호출을 모아 마지막 호출만 wait 밀리초 뒤에 실행하는 함수를 만든다.
   * 창 크기 변경처럼 자주 일어나는 이벤트에서 메뉴를 한 번만 닫기 위해 쓴다.
   * @param {() => void} fn 지연 실행할 함수
   * @param {number} wait 대기 시간(밀리초)
   * @returns {Function}
   */
  function debounce(fn, wait) {
    var t;
    return function () {
      clearTimeout(t);
      t = window.setTimeout(fn, wait);
```

```
    };
  }

  /**
   * id가 있는 섹션에 tabindex -1을 넣어, 메뉴 이동 뒤 키보드 포커스를 받을 수 있게 한다.
   */
  function initSectionTargets() {
    queryAll(".section[id]").forEach(function (section) {
      if (!section.hasAttribute("tabindex")) {
        section.setAttribute("tabindex", "-1");
      }
    });
  }

  if (document.readyState === "loading") {
    document.addEventListener("DOMContentLoaded", function () {
      initSmoothNav();
      initResourceDropdown();
      initThemeToggle();
      initSectionTargets();
    });
  } else {
    initSmoothNav();
    initResourceDropdown();
    initThemeToggle();
    initSectionTargets();
  }
})();
```

online_business_card/business-card.html

```
<!-- 온라인 명함 화면. 스크롤에 따라 명함이 뒤집히고 커진 뒤, 이어서 이력 절이 나타난다. -->
<!DOCTYPE html>
<html lang="ko" data-default-theme="light" data-theme="light">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>코탐록 - 코딩 탐험가 이력서</title>
    <script>
        (function () {
            try {
                var theme = localStorage.getItem("portfolio-theme-v2");
                if (theme === "dark" || theme === "light") {
                    document.documentElement.setAttribute("data-theme", theme);
                }
            } catch (e) {
                /* 테마 저장소를 사용할 수 없는 환경에서는 라이트(흰색) 기본값을 사용합니다. */
            }
        })();
    </script>
    <link rel="stylesheet" href="css/style.css?v=scroll-hint">
    <link rel="stylesheet" href="../css/site-header.css?v=theme-sync">
</head>
<body>
    <header class="site-header">
        <div class="site-header__inner">
            <a class="site-header__brand" href="../index.html">코탐록</a>
            <nav id="site-nav" class="site-nav" aria-label="자료 페이지">
                <ul class="site-nav__list">
                    <li><a class="site-nav__link" href="../cover_letter/profile.html">자기소개 / 프로필</a></li>
                    <li><a class="site-nav__link is-active" href="../online_business_card/business-card.html" aria-cur
rent="page">온라인 명함</a></li>
                    <li><a class="site-nav__link" href="../cv/resume.html">이력서</a></li>
                    <li><a class="site-nav__link" href="../study_note_record_organization/dashboard/index.html">학습록
</a></li>
                </ul>
            </nav>
            <div class="site-header__actions">
                <button type="button" class="theme-toggle" data-theme-toggle aria-pressed="false" aria-label="다크 테마
로 전환">
                    <span class="theme-toggle__label">고대비</span>
                    <span class="theme-toggle__state" data-theme-label>라이트</span>
                </button>
                <button
                    type="button"
                    class="nav-toggle"
                    aria-expanded="false"
                    aria-controls="site-nav"
                    aria-label="메뉴 열기"
                >
                    <span class="nav-toggle__bar" aria-hidden="true"></span>
                    <span class="nav-toggle__bar" aria-hidden="true"></span>
                </button>
            </div>
        </div>
    </header>

    <div class="scroll-hint" id="scrollHint" aria-hidden="true">
        <div class="scroll-hint__inner">
            <span class="scroll-hint__text">아래로 스크롤하세요</span>
            <span class="scroll-hint__line"></span>
        </div>
    </div>

```

```
    <!-- 애니메이션을 위한 스크롤 영역 -->
    <div class="scroll-container">
        <!-- 화면에 고정되는 3D 씬 영역 -->
        <div class="sticky-container">
            <!-- 3D 효과를 위한 원근감 컨테이너 -->
            <div class="scene">
                <!-- 명함 객체 -->
                <div class="card" id="card">
                    <!-- 명함 앞면 -->
                    <div class="card-face card-front">
                        <div class="card-content">
                            <h1>코탐록</h1>
                            <p class="title">웹 / 모바일 / AI 코딩 탐험가</p>
                            <p class="blog">blog.naver.com/fifo23</p>
                        </div>
                    </div>
                    <!-- 명함 뒷면 -->
                    <div class="card-face card-back">
                        <div class="card-content">
                            <h2>Resume</h2>
                            <p>Scroll down to explore</p>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>

    <!-- 명함 확대 후 나타나는 이력서 상세 영역 (일반 스크롤) -->
    <div class="resume-container" id="resume-container">
        <div class="resume-content">
                    <section class="intro">
                        <h2>기록으로 지도를 그리는 코딩 탐험가, '코탐록'을 쓰다</h2>
                        <p>저는 정해진 길만 걷는 코더가 아니라, 웹과 모바일, 그리고 AI라는 미지의 영역을 꾸준히 학습하고 지도로 만들어가는 '코딩 탐험가'입니다. 네이버 블로그 '코탐록(blog.naver.com/fifo23)'을 운영할 예정이며, 단순한 호기심을 넘어 IT 기술 전반에 대한 깊이 있는 탐구를 기록하려고 합니다.</p>
                        <p>화려한 사교성이나 무모한 도전정신보다는, 매일 꾸준히 책을 읽고 학습하며 묵묵히 저만의 기술 스택을 쌓아가는 성실함이 저의 가장 큰 무기입니다. 프론트엔드와 백엔드, 웹과 앱이라는 경계에 갇히지 않고 전체적인 서비스의 숲을 보기 위해 다양한 기술 생태계를 탐험해 왔으며, 이러한 꼼꼼한 기록과 학습 습관은 IT 현장에서 마주할 어떤 낯선 업무라도 차분하게 분석하고 적응할 수 있는 든든한 밑거름이 될 것입니다.</p>
                    </section>

                    <section class="skills">
                        <h2>탄탄한 기본기 위에 AI 에이전트를 더해 학습의 한계를 넓히다</h2>
                        <p>새로운 기술을 마주할 때, 저는 패스트캠퍼스, 인프런, 코드잇 등 다양한 교육 플랫폼과 전공 서적을 통해 기본기부터 단단하게 다집니다. 대학 시절 Visual Basic, SAS, SPSS로 시작한 저의 탐험은 오프라인 기관의 CS 지식을 거쳐 Java(GUI, AWT, SWING)와 Kotlin(모바일 앱)으로 이어졌습니다.</p>
                        <p>특히 최근에는 생성형 AI와 프롬프트 엔지니어링, 바이브 코딩(Vibe Coding)을 적극적으로 학습하며 저만의 개발 생산성을 극대화하고 있습니다. 낯선 기술이나 복잡한 개념을 마주하더라도 AI 에이전트를 나침반 삼아 원리를 파악하고, 펠로AI나 젠스파크를 활용해 학습한 필기를 슬라이드와 디지털 콘텐츠로 구조화하여 블로그에 기록할 예정입니다. 이러한 저만의 체계적인 학습 방식은 끊임없이 신기술이 쏟아지는 IT 분야에서 남들보다 빠르고 정확하게 지식을 내재화하는 원동력입니다.</p>
                    </section>

                    <section class="projects">
                        <h2>무모한 도전보다는 묵묵한 끈기로: 일상을 바꾸는 실용적 개발</h2>
                        <p>저는 거창한 프로젝트보다 일상의 문제를 해결하고 동작하는 결과물을 직접 만들어내는 과정에서 큰 보람을 느낍니다. Java와 Kotlin을 활용해 로또 추첨부터 당첨 확인까지 가능한 게임 앱과 XML 레이아웃 기반의 두더지 잡기 게임, SWING 스톱워치 등을 직접 구현하며 모바일과 웹 애플리케이션의 구조를 체득했습니다. 또한, 레인미터로 바탕화면을 정리하거나 Visual .NET을 활용해 텍스트 파일 병합 프로그램을 만드는 등 실생활에 필요한 툴을 직접 개발해 왔습니다.</p>
                        <p>이 과정에서 마주치는 수많은 오류와 막막함 앞에서도 저는 포기하지 않았습니다. 문제를 적당히 우회하기보다는, 묵묵히 자리에 앉아 공식 문서와 책을 뒤적이며 끝까지 파고드는 집요함으로 에러를 해결했습니다. 최근에는 생성형 AI에 코드를 요청하고 수정하며 자질구레한 프로그램들을 완성해 보는 등, 문제 해결의 도구를 넓혀가며 실무에 즉시 투입 가능한 실용적인 개발 역량을 증명해 오고 있습니다.</p>
                    </section>

                    <section class="conclusion">
                        <h2>IT 현장이라는 새로운 목적지를 향해 꾸준히 걷는 개발자</h2>
```

```
                        <p>지금까지 저의 '코탐록'이 웹과 모바일, AI를 넘나들며 책과 강의를 통해 저만의 기술적 역량과 적성을 찾아가는 치열한 준비 과정이었다면, 이제는 새로운 비즈니스 목표를 달성하기 위한 실전 탐험을 시작하고자 합니다.</p>
                        <p>30대 후반이라는 나이는 IT 업계에서 결코 적지 않은 나이일 수 있습니다. 하지만 그만큼 흔들리지 않는 끈기와 성실함, 그리고 기술을 대하는 진중한 태도를 갖추고 있습니다. 입사 후 웹 개발, 앱 개발, 혹은 IT 현장직 등 어떤 역할이 주어지더라도, 저만의 '코탐록'을 써 내려가던 묵묵함과 AI를 활용한 스마트한 문제 해결 능력으로 팀과 회사를 위해 끝까지 책임지는 든든한 인재가 되겠습니다.</p>
                    </section>
                </div>
            </div>
    <script src="js/script.js?v=scroll-hint"></script>
    <script src="../js/main.js?v=theme-sync"></script>
</body>
</html>
```

online_business_card/css/style.css

```
/* 명함의 3D 변환, 스크롤 안내, 이력 절이 보일 때의 스타일을 정한다. */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    /* 깔끔하고 모던한 폰트 설정 */
    font-family: 'Pretendard', -apple-system, BlinkMacSystemFont, system-ui, Roboto, 'Helvetica Neue', 'Segoe UI', 'Apple SD Gothic Neo', 'Noto Sans KR', 'Malgun Gothic', sans-serif;
    background-color: #f5f5f7;
    color: #333;
    overflow-x: hidden;
}

.scroll-hint {
    position: fixed;
    right: clamp(18px, 4vw, 42px);
    top: 50%;
    z-index: 40;
    pointer-events: none;
    opacity: 1;
    transform: translateY(-50%);
    transition: opacity 0.35s ease, visibility 0.35s ease;
}

.scroll-hint.is-hidden {
    opacity: 0;
    visibility: hidden;
}

.scroll-hint__inner {
    display: flex;
    align-items: center;
    gap: 0.8rem;
    color: #666;
    font-size: 0.78rem;
    font-weight: 700;
    letter-spacing: 0.16em;
    writing-mode: vertical-rl;
    text-orientation: mixed;
    animation: scrollHintFloat 1.6s ease-in-out infinite;
}

.scroll-hint__line {
    width: 1px;
    height: 42px;
    background: linear-gradient(to bottom, transparent, #777, transparent);
}

@keyframes scrollHintFloat {
    0%,
    100% {
        transform: translateY(-6px);
    }

    50% {
        transform: translateY(8px);
    }
}

/* 스크롤을 발생시키기 위한 전체 컨테이너 높이 (250vh) */
```

```css
.scroll-container {
    height: 250vh;
}

/* 화면에 고정되어 애니메이션을 보여줄 영역 */
.sticky-container {
    position: sticky;
    top: 0;
    width: 100vw;
    height: 100vh;
    overflow: hidden;
    display: flex;
    justify-content: center;
    align-items: center;
    background-color: #f5f5f7;
}

/* 3D 원근감을 위한 씬 */
.scene {
    width: 100vw;
    height: 100vh;
    perspective: 1000px; /* 원근감 깊이 */
    display: flex;
    justify-content: center;
    align-items: center;
    position: absolute;
    z-index: 10;
}

/* 명함 본체 */
.card {
    width: 400px;
    height: 240px;
    position: relative;
    transform-style: preserve-3d; /* 자식 요소의 3D 공간 유지 */
    /* 초기 2.5D 비스듬한 상태 */
    transform: rotateX(10deg) rotateY(-15deg);
    will-change: transform; /* 성능 최적화 */
}

/* 명함의 앞/뒷면 공통 스타일 */
.card-face {
    position: absolute;
    width: 100%;
    height: 100%;
    backface-visibility: hidden; /* 뒷면일 때 안 보이게 처리 */
    border-radius: 12px;
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 30px;
    background-color: #ffffff;
}

/* 명함 앞면 */
.card-front {
    border: 1px solid #eaeaea;
}

/* 명함 뒷면 */
.card-back {
    transform: rotateY(180deg); /* 기본적으로 뒤집혀 있음 */
    background-color: #1a1a1a; /* 모던한 다크 그레이 */
    color: #ffffff;
}

```

```css
.card-content {
    text-align: center;
}

.card-front h1 {
    font-size: 32px;
    font-weight: 800;
    margin-bottom: 12px;
    letter-spacing: -0.5px;
    color: #111;
}

.card-front .title {
    font-size: 16px;
    font-weight: 500;
    color: #666;
    margin-bottom: 24px;
}

.card-front .blog {
    font-size: 14px;
    color: #888;
    background: #f0f0f0;
    padding: 6px 12px;
    border-radius: 20px;
    display: inline-block;
}

.card-back h2 {
    font-size: 28px;
    font-weight: 700;
    margin-bottom: 12px;
    letter-spacing: 2px;
}

.card-back p {
    font-size: 14px;
    color: #aaa;
    font-weight: 300;
}

/* 이력서 상세 영역 (일반적인 문서 흐름) */
.resume-container {
    position: relative;
    width: 100vw;
    background-color: #1a1a1a; /* 명함 뒷면 색상과 일치시켜 자연스러운 전환 */
    color: #f5f5f7;
    z-index: 20;
}

/* 이력서 내부 1단 레이아웃 콘텐츠 */
.resume-content {
    max-width: 800px;
    margin: 0 auto;
    padding: 100px 40px;
}

.resume-content section {
    margin-bottom: 80px;
    opacity: 0;
    transform: translateY(40px);
    transition: all 0.8s cubic-bezier(0.16, 1, 0.3, 1);
}

/* 스크롤에 따라 나타나는 애니메이션 클래스 */
.resume-content section.visible {
    opacity: 1;
```

```
    transform: translateY(0);
}

.resume-content h2 {
    font-size: 26px;
    font-weight: 700;
    margin-bottom: 24px;
    color: #ffffff;
    border-bottom: 1px solid #333;
    padding-bottom: 16px;
    line-height: 1.4;
}

.resume-content p {
    font-size: 17px;
    line-height: 1.8;
    color: #cccccc;
    margin-bottom: 20px;
    word-break: keep-all;
}

@media (prefers-reduced-motion: reduce) {
    .scroll-hint__inner {
        animation: none;
    }
}

@media (max-width: 640px) {
    .scroll-hint {
        right: 14px;
    }

    .scroll-hint__inner {
        font-size: 0.7rem;
    }
}
```

online_business_card/js/script.js

```
/**
 * 스크롤에 따라 명함을 뒤집고 확대하며, 이력서 절이 보이면 나타나게 한다.
 */
document.addEventListener('DOMContentLoaded', () => {
    const card = document.getElementById('card');
    const scrollContainer = document.querySelector('.scroll-container');
    const resumeSections = document.querySelectorAll('.resume-content section');
    const scrollHint = document.getElementById('scrollHint');

    let ticking = false;

    /**
     * 스크롤 진행률로 명함 기울기·뒤집기·확대 transform을 다시 계산해 카드에 적용한다.
     * 앞 절반은 뒤집기, 뒤 절반은 화면을 덮도록 확대하고, 조금 내리면 스크롤 안내를 숨긴다.
     */
    function updateAnimation() {
        // scroll-container의 높이를 기준으로 애니메이션 진행률 계산
        const scrollY = window.scrollY;
        // scroll-container가 끝나는 지점 (sticky가 끝나는 지점)
        const maxScroll = scrollContainer.offsetHeight - window.innerHeight;

        // progress는 0에서 1까지만 제한
        const progress = Math.min(Math.max(scrollY / maxScroll, 0), 1);

        // Phase 1 (0.0 ~ 0.5): 명함 뒤집기
        // Phase 2 (0.5 ~ 1.0): 명함 확대

        let rotateX = 10;
        let rotateY = -15;
        let scale = 1;

        if (progress <= 0.5) {
            const flipProgress = progress / 0.5;
            rotateX = 10 * (1 - flipProgress);
            rotateY = -15 + (195 * flipProgress);
            scale = 1;
            card.style.opacity = 1;
        } else {
            const scaleProgress = (progress - 0.5) / 0.5;
            rotateX = 0;
            rotateY = 180;

            // 확대 효과 (가속도 곡선 적용하여 점점 빠르게 커지도록)
            const easeScale = scaleProgress * scaleProgress * scaleProgress;
            scale = 1 + (40 * easeScale); // 화면을 덮도록 크게 확대

            card.style.opacity = 1;
        }

        // 계산된 속성값을 CSS transform에 적용
        card.style.transform = `rotateX(${rotateX}deg) rotateY(${rotateY}deg) scale(${scale})`;

        if (scrollHint) {
            scrollHint.classList.toggle('is-hidden', scrollY > 24);
        }

        ticking = false;
    }

    // 스크롤 이벤트 리스너 (requestAnimationFrame으로 성능 최적화)
    window.addEventListener('scroll', () => {
        if (!ticking) {
            window.requestAnimationFrame(updateAnimation);
```

```
            ticking = true;
        }
    });

    // 초기 로드 시 한 번 실행하여 초기 상태 세팅
    updateAnimation();

    // 이력서 섹션 스크롤 애니메이션 (Intersection Observer 활용)
    // 화면에 요소가 나타날 때 애니메이션을 트리거하여 스크롤 양에 구애받지 않음
    const observerOptions = {
        root: null,
        rootMargin: '0px',
        threshold: 0.2 // 섹션이 20% 보일 때 애니메이션 트리거
    };

    const observer = new IntersectionObserver((entries, observer) => {
        entries.forEach(entry => {
            if (entry.isIntersecting) {
                entry.target.classList.add('visible');
                // 한 번 나타난 후에는 계속 보이게 하려면 아래 주석 해제
                // observer.unobserve(entry.target);
            } else {
                // 스크롤을 위로 올렸을 때 다시 애니메이션을 보고 싶다면
                entry.target.classList.remove('visible');
            }
        });
    }, observerOptions);

    resumeSections.forEach(section => {
        observer.observe(section);
    });
});
```

study_note_record_organization/dashboard/css/styles.css

```
/* 학습 기록 대시보드의 카드, 검색, 모달, 히어로 갤러리 색과 배치를 정한다. */
/* 레이아웃 참고: https://ackdark23358.github.io/ (코탐록) */

:root {
  --navy: #b8d4ff;
  --navy-soft: #8abaff;
  --text: #f5f5f5;
  --text-muted: rgba(245, 245, 245, 0.66);
  --text-light: rgba(245, 245, 245, 0.5);
  --bg: #0c0c0c;
  --bg-subtle: #161616;
  --border: rgba(255, 255, 255, 0.12);
  --study: #6ba8f5;
  --assignment: #52c896;
  --shadow: 0 2px 12px rgba(0, 0, 0, 0.4);
  --overlay: rgba(0, 0, 0, 0.62);
  --footer-note: rgba(245, 245, 245, 0.55);
  --container: min(90rem, 88vw);
  --font: "Malgun Gothic", "Apple SD Gothic Neo", "Noto Sans KR", system-ui, sans-serif;
  --line: 1.75;
}

html[data-theme="light"] {
  --navy: #003366;
  --navy-soft: #1a4d7a;
  --text: #222222;
  --text-muted: #555555;
  --text-light: #777777;
  --bg: #ffffff;
  --bg-subtle: #f5f6f8;
  --border: #dde2e8;
  --study: #1a56a8;
  --assignment: #0d6e4f;
  --shadow: 0 2px 12px rgba(0, 51, 102, 0.08);
  --overlay: rgba(0, 30, 60, 0.4);
  --footer-note: #8b9cb3;
}

html {
  color-scheme: dark;
  scroll-behavior: smooth;
  -webkit-text-size-adjust: 100%;
}

html[data-theme="light"] {
  color-scheme: light;
}

*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: var(--font);
  font-size: 1rem;
  line-height: var(--line);
  color: var(--text);
  background: var(--bg);
}

```

```
.container {
  width: var(--container);
  max-width: 100%;
  margin-left: auto;
  margin-right: auto;
  padding-left: clamp(0.75rem, 2vw, 1.25rem);
  padding-right: clamp(0.75rem, 2vw, 1.25rem);
}

.visually-hidden {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  border: 0;
}

.hidden {
  display: none !important;
}

.skip-link {
  position: absolute;
  left: -9999px;
  top: 0;
  z-index: 2000;
  padding: 0.5rem 1rem;
  background: var(--navy);
  color: #fff;
  text-decoration: none;
}

.skip-link:focus {
  left: 0;
}

/* 포트폴리오 공통 상단 카테고리(site-header)와 본문 레이아웃 */
body:has(.site-header) .page {
  padding-top: 0.5rem;
}

/* ── Hero ── */
.hero {
  width: 100%;
  margin-bottom: clamp(1.25rem, 3vw, 2rem);
}

.hero__stage {
  display: block;
  position: relative;
  width: min(90rem, 88vw);
  max-width: 100%;
  height: clamp(12rem, 32vh, 22rem);
  margin: clamp(1rem, 3vw, 1.75rem) auto 0;
  padding: 0;
  border: 1px solid var(--border);
  border-radius: 8px;
  overflow: hidden;
  cursor: pointer;
  background: var(--bg-subtle);
  font: inherit;
  color: inherit;
  box-shadow: var(--shadow);
  transition: border-color 0.28s ease, box-shadow 0.28s ease;
```

```
}

.hero__stage:hover {
  border-color: var(--navy-soft);
  box-shadow: 0 4px 20px rgba(0, 51, 102, 0.12);
}

.hero__stage:focus-visible {
  outline: 2px solid var(--navy);
  outline-offset: 3px;
}

.hero__slides {
  position: absolute;
  inset: 0;
}

.hero__img {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  object-fit: cover;
  opacity: 0;
  transition: opacity 1.2s ease-in-out;
  pointer-events: none;
  user-select: none;
}

.hero__img.is-visible {
  opacity: 1;
}

.hero__hint {
  position: absolute;
  left: 50%;
  bottom: 1rem;
  transform: translateX(-50%);
  z-index: 2;
  padding: 0.4rem 1rem;
  font-size: 0.85rem;
  font-weight: 600;
  letter-spacing: 0.04em;
  color: #ffffff;
  background: rgba(0, 30, 60, 0.55);
  border: 1px solid rgba(255, 255, 255, 0.35);
  border-radius: 999px;
  backdrop-filter: blur(4px);
  animation: hero-hint-pulse 2.4s ease-in-out infinite;
  pointer-events: none;
}

@keyframes hero-hint-pulse {
  0%,
  100% {
    opacity: 0.55;
    transform: translateX(-50%) translateY(0);
  }
  50% {
    opacity: 1;
    transform: translateX(-50%) translateY(-4px);
  }
}

@media (prefers-reduced-motion: reduce) {
  .hero__img {
    transition: opacity 0.35s ease;
```

```
  }

  .hero__hint {
    animation: none;
    opacity: 0.9;
  }
}

/* ── Page sections ── */
.page {
  padding-top: clamp(1rem, 2.5vw, 1.5rem);
  padding-bottom: 3rem;
}

.section {
  margin-bottom: 2.5rem;
}

.section__title {
  margin: 0 0 1rem;
  font-size: clamp(1.25rem, 3vw, 1.5rem);
  font-weight: 700;
  color: var(--navy);
  line-height: 1.35;
}

.section__lead,
.section__desc,
.section__note {
  margin: 0;
  color: var(--text);
  max-width: 52em;
}

.section__desc {
  margin-bottom: 1.25rem;
  color: var(--text-muted);
  font-size: 0.95rem;
}

.section__note {
  margin-bottom: 2.5rem;
  color: var(--text-muted);
  font-size: 0.95rem;
}

/* 2-column info (코탐록 역량 요약 블록) */
.info-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 2rem 3rem;
  margin-bottom: 1.75rem;
}

.info-col__title {
  margin: 0 0 0.5rem;
  font-size: 1rem;
  font-weight: 700;
  color: var(--navy);
}

.info-col__text {
  margin: 0;
  font-size: 0.95rem;
  color: var(--text);
}

```

```
.info-col__text strong {
  color: var(--navy);
}

/* Toolbar & search */
.toolbar {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 0.75rem 1rem;
  margin-bottom: 1.5rem;
}

.search-field__input {
  width: 100%;
  max-width: 24rem;
  padding: 0.55rem 0.75rem;
  font: inherit;
  font-size: 0.95rem;
  border: 1px solid var(--border);
  border-radius: 3px;
  background: var(--bg);
}

.search-field__input:focus {
  outline: 2px solid var(--navy);
  outline-offset: 1px;
}

/* Card grid — 코탐록 자료 카테고리 카드 스타일 */
.card-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1.5rem 2rem;
  align-items: start;
}

.card {
  display: block;
  width: 100%;
  margin: 0;
  padding: 1.2rem 1.35rem 1.35rem;
  border: 1px solid var(--border);
  border-radius: 6px;
  background: var(--bg);
  text-align: left;
  font-family: inherit;
  color: inherit;
  cursor: pointer;
  box-shadow: none;
  transition:
    border-color 0.28s ease,
    background-color 0.28s ease,
    box-shadow 0.28s ease,
    transform 0.28s ease;
}

.card:hover {
  border-color: var(--navy-soft);
  background: var(--bg-subtle);
  box-shadow: var(--shadow);
  transform: translateY(-2px);
}

.card:active {
  transform: translateY(0);
  box-shadow: none;
```

```
  transition-duration: 0.12s;
}

.card:focus-visible {
  outline: 2px solid var(--navy);
  outline-offset: 2px;
  border-color: var(--navy);
}

.card--study {
  border-left-width: 3px;
  border-left-color: var(--study);
}

.card--study:hover,
.card--study:focus-visible {
  border-left-color: var(--study);
}

.card--assignment {
  border-left-width: 3px;
  border-left-color: var(--assignment);
}

.card--assignment:hover,
.card--assignment:focus-visible {
  border-left-color: var(--assignment);
}

.card-label {
  margin: 0 0 0.35rem;
  font-size: 0.72rem;
  font-weight: 600;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  color: var(--text-light);
}

.card--study .card-label {
  color: var(--study);
}

.card--assignment .card-label {
  color: var(--assignment);
}

.card__title {
  margin: 0 0 0.5rem;
  font-size: 1.05rem;
  font-weight: 700;
  color: var(--navy);
  line-height: 1.4;
}

.card__summary {
  margin: 0;
  font-size: 0.9rem;
  color: var(--text-muted);
  line-height: 1.6;
  display: -webkit-box;
  -webkit-line-clamp: 4;
  -webkit-box-orient: vertical;
  overflow: hidden;
  transition: color 0.28s ease;
}

.card:hover .card__title {
```

```
  color: var(--navy-soft);
}

@media (prefers-reduced-motion: reduce) {
  .card {
    transition: border-color 0.2s ease, background-color 0.2s ease;
  }

  .card:hover,
  .card:active {
    transform: none;
  }
}

.empty,
.noscript {
  text-align: center;
  color: var(--text-muted);
  padding: 2rem 0;
}

.loading {
  color: var(--text-muted);
  padding: 1.5rem 0;
}

.error {
  color: #b91c1c;
}

.error code {
  font-size: 0.88em;
  background: var(--bg-subtle);
  padding: 0.1rem 0.3rem;
}

.site-footer {
  margin-top: 2rem;
  padding: 1.5rem 0 2.5rem;
  border-top: 1px solid var(--border);
}

.site-footer__inner {
  text-align: center;
}

.site-footer__meta {
  margin: 0 0 1.25rem;
  font-size: 0.85rem;
  line-height: 1.6;
  color: var(--text-muted);
}

.site-footer__note {
  margin: 0 auto;
  max-width: 22rem;
  font-size: 1rem;
  font-weight: 300;
  line-height: 1.65;
  text-align: center;
  color: var(--footer-note);
}

.site-footer code {
  font-size: 0.82em;
}

```

```
.site-footer a {
  color: var(--study);
}

/* ── Modal ── */
.modal-overlay {
  position: fixed;
  inset: 0;
  z-index: 1000;
  background: var(--overlay);
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 1rem;
}

.modal {
  width: 100%;
  max-width: 40rem;
  max-height: 90vh;
  background: var(--bg);
  border: 1px solid var(--border);
  box-shadow: var(--shadow);
  display: flex;
  flex-direction: column;
}

.modal--wide {
  max-width: 44rem;
}

#source-overlay {
  z-index: 1100;
}

.modal__header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  gap: 1rem;
  padding: 1.15rem 1.35rem;
  border-bottom: 1px solid var(--border);
  background: var(--bg-subtle);
}

.modal__meta {
  min-width: 0;
  flex: 1;
}

.modal__title {
  margin: 0;
  font-size: 1.15rem;
  font-weight: 700;
  color: var(--navy);
}

.modal__summary {
  margin: 0.35rem 0 0;
  font-size: 0.9rem;
  color: var(--text-muted);
}

.modal__close {
  flex-shrink: 0;
  width: 2.25rem;
  height: 2.25rem;
```

```
  border: 1px solid var(--border);
  background: var(--bg);
  color: var(--navy);
  font-size: 1.35rem;
  line-height: 1;
  cursor: pointer;
  border-radius: 3px;
}

.modal__body {
  padding: 1.25rem 1.35rem 1.5rem;
  overflow-y: auto;
  flex: 1;
}

.md-content h1 {
  font-size: 1.2rem;
  margin: 0 0 1rem;
  color: var(--navy);
  border-bottom: 1px solid var(--border);
  padding-bottom: 0.4rem;
}

.md-content h2 {
  font-size: 1.05rem;
  margin: 1.4rem 0 0.6rem;
  color: var(--navy-soft);
  font-weight: 700;
}

.md-content h3 {
  font-size: 1rem;
  margin: 1rem 0 0.4rem;
  font-weight: 600;
}

.md-content p,
.md-content li {
  font-size: 0.95rem;
}

.md-content ul {
  padding-left: 1.25rem;
}

.md-content code {
  background: var(--bg-subtle);
  padding: 0.1rem 0.35rem;
  font-size: 0.88em;
  border: 1px solid var(--border);
}

.md-content pre {
  background: var(--bg-subtle);
  border: 1px solid var(--border);
  padding: 1rem;
  overflow-x: auto;
  font-size: 0.85rem;
}

.md-content img {
  display: block;
  width: 100%;
  max-width: 22rem;
  height: auto;
  margin: 0.75rem auto;
  border: 1px solid var(--border);
```

```
  border-radius: 4px;
  background: var(--bg-subtle);
}

.md-content em {
  display: block;
  margin-top: 0.35rem;
  font-size: 0.88rem;
  color: var(--text-muted);
  font-style: normal;
}

.md-content a,
.md-content a.source-link {
  color: var(--study);
}

.source-plain {
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
}

.source-plain__text {
  white-space: pre-wrap;
  word-break: break-word;
  font-size: 0.95rem;
  line-height: var(--line);
}

.source-plain__figure {
  margin: 0;
  padding: 0;
}

.source-plain__figure img {
  display: block;
  width: 100%;
  max-width: 100%;
  height: auto;
  border: 1px solid var(--border);
  border-radius: 4px;
  background: #000000;
}

.source-plain__figure figcaption {
  margin-top: 0.5rem;
  font-size: 0.85rem;
  color: var(--text-muted);
  text-align: center;
}

/* ── Responsive ── */
@media (max-width: 900px) {
  .card-grid {
    grid-template-columns: repeat(2, 1fr);
    gap: 1.25rem 1.5rem;
  }
}

@media (max-width: 768px) {
  .info-grid {
    grid-template-columns: 1fr;
    gap: 1.5rem;
  }
}

```

```
@media (max-width: 520px) {
  .card-grid {
    grid-template-columns: 1fr;
  }

  .modal-overlay {
    padding: 0;
    align-items: stretch;
  }

  .modal {
    max-height: 100dvh;
    border-radius: 0;
  }
}

@media (prefers-reduced-motion: reduce) {
  html {
    scroll-behavior: auto;
  }
}
```

study_note_record_organization/dashboard/index.html

```
<!-- 학습 기록 대시보드. 카드 목록, 검색, 요약 모달, 원본 보기를 담는다. -->
<!DOCTYPE html>
<html lang="ko" data-default-theme="light" data-theme="light">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="학습·과제 기록 요약 아카이브" />
    <title>학습록 — 학습·과제 기록</title>
    <script>
      (function () {
        try {
          var theme = localStorage.getItem("portfolio-theme-v2");
          if (theme === "dark" || theme === "light") {
            document.documentElement.setAttribute("data-theme", theme);
          }
        } catch (e) {
          /* 테마 저장소를 사용할 수 없는 환경에서는 기본 테마를 사용합니다. */
        }
      })();
    </script>
    <link rel="stylesheet" href="css/styles.css" />
    <link rel="stylesheet" href="../../css/site-header.css?v=theme-sync" />
  </head>
  <body>
    <a class="skip-link" href="#main">본문으로 건너뛰기</a>

    <header class="site-header">
      <div class="site-header__inner">
        <a class="site-header__brand" href="../../index.html">코탐록</a>
        <nav id="site-nav" class="site-nav" aria-label="자료 페이지">
          <ul class="site-nav__list">
            <li><a class="site-nav__link" href="../../cover_letter/profile.html">자기소개 / 프로필</a></li>
            <li><a class="site-nav__link" href="../../online_business_card/business-card.html">온라인 명함</a></li>
            <li><a class="site-nav__link" href="../../cv/resume.html">이력서</a></li>
            <li>
              <a
                class="site-nav__link is-active"
                href="index.html"
                aria-current="page"
              >학습록</a>
            </li>
          </ul>
        </nav>
        <div class="site-header__actions">
          <button type="button" class="theme-toggle" data-theme-toggle aria-pressed="false" aria-label="다크 테마로 전
환">
            <span class="theme-toggle__label">고대비</span>
            <span class="theme-toggle__state" data-theme-label>라이트</span>
          </button>
          <button
            type="button"
            class="nav-toggle"
            aria-expanded="false"
            aria-controls="site-nav"
            aria-label="메뉴 열기"
          >
            <span class="nav-toggle__bar" aria-hidden="true"></span>
            <span class="nav-toggle__bar" aria-hidden="true"></span>
          </button>
        </div>
      </div>
    </header>

```

```
    <section class="hero hidden" id="hero" aria-label="사진 갤러리">
      <button type="button" class="hero__stage" id="hero-stage" aria-label="다른 사진 보기">
        <div class="hero__slides">
          <img class="hero__img is-visible" id="hero-img-a" alt="" decoding="async" />
          <img class="hero__img" id="hero-img-b" alt="" decoding="async" />
        </div>
        <span class="hero__hint" aria-hidden="true">클릭하세요</span>
      </button>
    </section>

    <div class="container page">
      <section class="section" id="about" aria-labelledby="about-title">
        <h2 class="section__title" id="about-title">1. 학습·과제 아카이브</h2>
        <p class="section__lead" id="subtitle"></p>
      </section>

      <div class="info-grid" aria-label="요약 정보">
        <div class="info-col">
          <h3 class="info-col__title">구성</h3>
          <p class="info-col__text" id="info-compose"></p>
        </div>
        <div class="info-col">
          <h3 class="info-col__title">이용 방법</h3>
          <p class="info-col__text">
            카드를 클릭하면 요약본을 읽을 수 있습니다. 요약 안의 원본 출처 링크를 누르면 전체 텍스트를 확인할 수 있습니다.
            과제 항목의 교수·학번은 프라이버시를 위해 마스킹되어 있습니다.
          </p>
        </div>
      </div>

      <p class="section__note" id="note"></p>

      <main class="section" id="main">
        <section id="records" aria-labelledby="records-title">
          <h2 class="section__title" id="records-title">2. 기록 목록</h2>
          <p class="section__desc">
            독서 노트·강의·과제 원본을 바탕으로 정리한 요약입니다. 제목이나 요약으로 검색할 수 있습니다.
          </p>

          <div class="toolbar">
            <label class="search-field">
              <span class="visually-hidden">검색</span>
              <input
                type="search"
                id="search"
                class="search-field__input"
                placeholder="제목·요약 검색…"
                autocomplete="off"
              />
            </label>
          </div>

          <div class="card-grid" id="grid"></div>
          <p class="empty hidden" id="empty">검색 결과가 없습니다.</p>
        </section>
      </main>

      <footer class="site-footer">
        <div class="site-footer__inner">
          <p class="site-footer__meta">
            <span id="footer-count"></span>
            · 정적 HTML/CSS/JavaScript ·
            <code>python scripts/build_static_site.py</code>
            · <a href="../../index.html">코탐록 메인</a>
          </p>
          <p class="site-footer__note">
            2026년 5월 25일 제작 · Cursor AI를 사용해 제작했습니다.
```

```
          </p>
        </div>
      </footer>
    </div>

    <div class="modal-overlay hidden" id="modal-overlay" aria-hidden="true">
      <div class="modal" role="dialog" aria-modal="true" aria-labelledby="modal-title">
        <header class="modal__header">
          <div class="modal__meta" id="modal-meta"></div>
          <button type="button" class="modal__close" id="modal-close" aria-label="닫기">×</button>
        </header>
        <div class="modal__body" id="modal-body"></div>
      </div>
    </div>

    <div class="modal-overlay hidden" id="source-overlay" aria-hidden="true">
      <div class="modal modal--wide" role="dialog" aria-modal="true" aria-labelledby="source-title">
        <header class="modal__header">
          <div class="modal__meta">
            <p class="card-label">SOURCE</p>
            <h2 class="modal__title" id="source-title">원본</h2>
          </div>
          <button type="button" class="modal__close" id="source-close" aria-label="닫기">×</button>
        </header>
        <div class="modal__body" id="source-body"></div>
      </div>
    </div>

    <noscript>
      <p class="noscript container">JavaScript를 활성화한 뒤 새로고침하세요.</p>
    </noscript>

    <script src="js/hero-images.js"></script>
    <script src="js/data.js?v=e39029c6e275"></script>
    <script src="js/hero.js"></script>
    <script src="js/app.js?v=e39029c6e275"></script>
    <script src="../../js/main.js?v=theme-sync"></script>
  </body>
</html>
```

study_note_record_organization/dashboard/js/app.js

```
/**
 * 학습 기록 대시보드에서 카드 목록, 검색, 요약 모달, 원본 보기를 초기화한다.
 */
(function () {
  "use strict";

  var CARD_LABEL = { study: "STUDY NOTE", assignment: "ASSIGNMENT" };
  var catalog = [];
  var articles = {};
  var sourceMap = {};
  var sourceCache = {};
  var sourceBuildId = "";
  var searchQuery = "";

  var grid = document.getElementById("grid");
  var empty = document.getElementById("empty");
  var subtitleEl = document.getElementById("subtitle");
  var noteEl = document.getElementById("note");
  var infoCompose = document.getElementById("info-compose");
  var footerCount = document.getElementById("footer-count");
  var searchInput = document.getElementById("search");
  var overlay = document.getElementById("modal-overlay");
  var modalBody = document.getElementById("modal-body");
  var modalMeta = document.getElementById("modal-meta");
  var modalClose = document.getElementById("modal-close");
  var sourceOverlay = document.getElementById("source-overlay");
  var sourceBody = document.getElementById("source-body");
  var sourceTitle = document.getElementById("source-title");
  var sourceClose = document.getElementById("source-close");

  /**
   * 사용자 문자를 HTML로 넣을 때 태그가 실행되지 않도록 이스케이프한다.
   * @param {string} s 원문
   * @returns {string}
   */
  function escapeHtml(s) {
    var d = document.createElement("div");
    d.textContent = s == null ? "" : String(s);
    return d.innerHTML;
  }

  /**
   * 빌드가 넣어 둔 window.SITE_DATA를 돌려준다. 없으면 null이다.
   * @returns {object|null}
   */
  function getData() {
    return window.SITE_DATA || null;
  }

  /**
   * 카탈로그 건수로 소개 문구, 학습·과제 수, 하단 총계를 채운다.
   */
  function renderIntro() {
    var study = 0;
    var assignment = 0;
    catalog.forEach(function (i) {
      if (i.type === "study") study++;
      else if (i.type === "assignment") assignment++;
    });
    var total = catalog.length;

    if (subtitleEl) {
      subtitleEl.textContent =
```

```
64          "독서 노트·강의·과제 기록을 요약해 둔 개인 아카이브입니다. " +
65          "웹·모바일·AI 학습 흐름을 한곳에서 이어 볼 수 있도록 정리했습니다.";
66      }
67
68      if (infoCompose) {
69        infoCompose.innerHTML =
70          "<strong>학습 " +
71          study +
72          "건</strong> — 독서·강의·자격·도구 노트<br>" +
73          "<strong>과제 " +
74          assignment +
75          "건</strong> — 학점은행제 제출 했던 과제";
76      }
77
78      if (noteEl) {
79        noteEl.textContent =
80          "현재 총 " +
81          total +
82          "개 항목을 검색·열람할 수 있습니다. 요약 md는 " +
83          "<code>python scripts/build_output.py</code>로, 정적 페이지 데이터는 " +
84          "<code>python scripts/build_static_site.py</code>로 갱신합니다.";
85      }
86
87      if (footerCount) {
88        footerCount.textContent = "총 " + total + "개 기록";
89      }
90    }
91
92    /**
93     * 검색어가 제목, 요약, 종류, 태그에 포함된 항목만 돌려준다. 검색어가 없으면 전체다.
94     * @returns {object[]}
95     */
96    function filteredItems() {
97      var q = searchQuery.trim().toLowerCase();
98      return catalog.filter(function (item) {
99        if (!q) return true;
100       var hay = [item.title, item.summary, item.type]
101         .concat(item.tags || [])
102         .join(" ")
103         .toLowerCase();
104       return hay.indexOf(q) !== -1;
105     });
106   }
107
108   /**
109    * 검색 결과로 카드 그리드를 다시 그린다. 결과가 없으면 빈 상태 문구를 보여 준다.
110    */
111   function renderGrid() {
112     var items = filteredItems();
113     grid.innerHTML = "";
114
115     if (items.length === 0) {
116       empty.classList.remove("hidden");
117       return;
118     }
119     empty.classList.add("hidden");
120
121     items.forEach(function (item) {
122       var btn = document.createElement("button");
123       btn.type = "button";
124       btn.className = "card card--" + item.type;
125       btn.setAttribute("aria-label", item.title + " 요약 보기");
126
127       btn.innerHTML =
128         '<p class="card-label">' +
129         escapeHtml(CARD_LABEL[item.type] || "RECORD") +
130         "</p>" +
```

```
        '<h3 class="card__title">' +
        escapeHtml(item.title) +
        "</h3>" +
        '<p class="card__summary">' +
        escapeHtml(item.summary || "요약 없음") +
        "</p>";

      btn.addEventListener("click", function () {
        openModal(item);
      });
      grid.appendChild(btn);
    });
  }

  /**
   * 원본 경로가 과제 폴더인지 판별한다. 과제면 열람 전에 개인정보를 가린다.
   * @param {string} key 원본 경로
   * @returns {boolean}
   */
  function isAssignmentSourceKey(key) {
    return key.indexOf("과제분류하기") !== -1 || key.indexOf("data_과제분류하기") !== -1;
  }

  /**
   * 과제 원문에서 지도교수 이름, 학번, 파일명 속 학번 숫자를 가린다.
   * @param {string} text 원문
   * @returns {string}
   */
  function maskAssignmentPrivacy(text) {
    return text
      .replace(/(지도교수\s*[:：]\s*)\S+(?:\s*교수님)?/g, "$1000 교수님")
      .replace(/((?:학\s*번|수\s*강\s*번\s*호)\s*[:：]\s*)\d+/g, "$100000000")
      .replace(/\(\d{7,8}\s+/g, "(00000000 ")
      .replace(/\(\d{7,8}\)/g, "(00000000)")
      .replace(/_\d{7,8}_/g, "_00000000_");
  }

  /**
   * HTML 원본을 태그가 없는 화면 텍스트로 바꾼다.
   * @param {string} html HTML 문자열
   * @returns {string}
   */
  function htmlToPlainText(html) {
    var doc = new DOMParser().parseFromString(html, "text/html");
    return (doc.body && doc.body.innerText) || html;
  }

  /**
   * 데이터베이스 총괄과제 원본에만 ER 다이어그램 이미지를 덧붙일 정보를 돌려준다.
   * @param {string} key 원본 경로
   * @returns {{src: string, alt: string, caption: string}|null}
   */
  function getSourceExtraImage(key) {
    if (!key) return null;
    var k = key;
    try {
      var decoded = decodeURIComponent(key);
      if (decoded) k = decoded;
    } catch (e) {
      /* ignore */
    }
    if (k.indexOf("데이터베이스") !== -1 && k.indexOf("총괄") !== -1) {
      return {
        src: "assets/database-er-diagram.png",
        alt: "게임개발 DB구축 ER 다이어그램",
        caption: "ER 다이어그램 (원본 PDF 도표)",
      };
```

```
    }
    return null;
  }

  /**
   * 원문 텍스트를 모달에 넣을 HTML로 만든다. 해당하면 ER 그림을 함께 넣는다.
   * @param {string} text 원문
   * @param {string} key 원본 경로
   * @returns {string}
   */
  function renderPlainSource(text, key) {
    var extra = getSourceExtraImage(key);
    var html =
      '<article class="md-content"><div class="source-plain">' +
      '<div class="source-plain__text">' +
      escapeHtml(text.replace(/\r\n/g, "\n")) +
      "</div>";
    if (extra) {
      html +=
        '<figure class="source-plain__figure">' +
        '<img src="' +
        escapeHtml(extra.src) +
        '" alt="' +
        escapeHtml(extra.alt) +
        '" loading="lazy" decoding="async" />' +
        '<figcaption>' +
        escapeHtml(extra.caption) +
        "</figcaption></figure>";
    }
    html += "</div></article>";
    return html;
  }

  /**
   * 원본 경로에 대응하는 소스 JSON 파일 이름을 찾는다. 인코딩된 경로도 풀어 본다.
   * @param {string} key 원본 경로
   * @returns {string|null}
   */
  function resolveSourceChunk(key) {
    if (sourceMap[key]) return sourceMap[key];
    try {
      var decoded = decodeURIComponent(key);
      if (decoded !== key && sourceMap[decoded]) return sourceMap[decoded];
    } catch (e) {
      /* ignore */
    }
    return null;
  }

  /**
   * 소스 JSON을 가져와 원문 텍스트를 돌려준다. 한 번 받은 값은 캐시에 둔다.
   * @param {string} key 원본 경로
   * @returns {Promise<string|null>}
   */
  function fetchSourceText(key) {
    if (sourceCache[key]) {
      return Promise.resolve(sourceCache[key]);
    }
    var chunk = resolveSourceChunk(key);
    if (!chunk) {
      return Promise.resolve(null);
    }
    var url = new URL("js/sources/" + chunk, window.location.href);
    if (sourceBuildId) {
      url.searchParams.set("v", sourceBuildId);
    }
    return fetch(url.href, { cache: "no-store" })
```

```
      .then(function (res) {
        if (!res.ok) throw new Error("fetch failed");
        return res.json();
      })
      .then(function (payload) {
        var text = payload && payload.text != null ? payload.text : null;
        sourceCache[key] = text;
        if (payload && payload.key && payload.key !== key) {
          sourceCache[payload.key] = text;
        }
        return text;
      })
      .catch(function () {
        return null;
      });
  }

  /**
   * 원본 보기 제목에서 링크 문구 또는 파일 이름을 쓰고, 추출본 꼬리표를 뺀다.
   * @param {string} label 링크에 보이던 문구
   * @param {string} key 원본 경로
   * @returns {string}
   */
  function sourceDisplayName(label, key) {
    return (
      (label || key.split("/").pop() || key).replace(/\s*\(텍스트 추출본\)\s*$/, "").trim() ||
      "원본"
    );
  }

  /**
   * 원본 보기 오버레이를 열고 텍스트를 불러온다. HTML 과제는 평문으로, 과제 원문은 개인정보를 가린다.
   * @param {string} key 원본 경로
   * @param {string} label 제목에 쓸 문구
   */
  function openSourceViewer(key, label) {
    sourceOverlay.classList.remove("hidden");
    sourceOverlay.setAttribute("aria-hidden", "false");
    document.body.style.overflow = "hidden";
    if (!overlay.classList.contains("hidden")) {
      overlay.style.pointerEvents = "none";
    }

    sourceTitle.textContent = sourceDisplayName(label, key);
    sourceBody.innerHTML = '<p class="loading">원본을 불러오는 중…</p>';

    fetchSourceText(key).then(function (text) {
      if (text == null) {
        sourceBody.innerHTML =
          '<p class="error"><code>build_static_site.py</code> 실행 후 다시 시도하세요.</p>';
        return;
      }

      if (/\.html?$/i.test(key)) {
        text = htmlToPlainText(text);
      }
      if (isAssignmentSourceKey(key)) {
        text = maskAssignmentPrivacy(text);
      }
      sourceBody.innerHTML = renderPlainSource(text, key);
    });
  }

  /**
   * 원본 보기 오버레이를 닫는다. 요약 모달이 없으면 페이지 스크롤을 다시 연다.
   */
  function closeSourceModal() {
```

```
    sourceOverlay.classList.add("hidden");
    sourceOverlay.setAttribute("aria-hidden", "true");
    overlay.style.pointerEvents = "";
    if (overlay.classList.contains("hidden")) {
      document.body.style.overflow = "";
    }
  }

  /**
   * 요약 본문 안의 원본 링크 클릭을 가로채 원본 보기를 연다.
   * @param {Element} container 요약 HTML이 들어간 요소
   */
  function bindSourceLinks(container) {
    container.addEventListener("click", function (e) {
      var a = e.target.closest("a[data-source], a.source-link");
      if (!a) return;
      e.preventDefault();
      e.stopPropagation();
      var key = a.getAttribute("data-source");
      if (key) openSourceViewer(key, a.textContent.trim());
    });
  }

  /**
   * 카드에 해당하는 요약 HTML을 모달에 넣고 원본 링크를 연결한다. 요약이 없으면 오류 문구를 보여 준다.
   * @param {object} item 카탈로그 항목
   */
  function openModal(item) {
    overlay.classList.remove("hidden");
    overlay.setAttribute("aria-hidden", "false");
    document.body.style.overflow = "hidden";

    modalMeta.innerHTML =
      '<p class="card-label">' +
      escapeHtml(CARD_LABEL[item.type] || "RECORD") +
      "</p>" +
      '<h2 id="modal-title" class="modal__title">' +
      escapeHtml(item.title) +
      "</h2>" +
      '<p class="modal__summary">' +
      escapeHtml(item.summary || "") +
      "</p>";

    var html = articles[item.slug];
    if (!html) {
      modalBody.innerHTML = '<p class="error">요약본이 없습니다.</p>';
      return;
    }

    modalBody.innerHTML = '<article class="md-content">' + html + "</article>";
    bindSourceLinks(modalBody);
  }

  /**
   * 요약 모달을 닫는다. 원본 보기가 열려 있지 않으면 페이지 스크롤을 다시 연다.
   */
  function closeModal() {
    overlay.classList.add("hidden");
    overlay.setAttribute("aria-hidden", "true");
    if (sourceOverlay.classList.contains("hidden")) {
      document.body.style.overflow = "";
    }
  }

  /**
   * SITE_DATA로 카탈로그와 원본 지도를 읽고 소개와 카드 그리드를 그린다.
   * 데이터가 없으면 빌드 스크립트를 실행하라는 안내를 보여 준다.
```

```
399     */
400    function init() {
401      var data = getData();
402      if (!data || !data.catalog) {
403        grid.innerHTML = '<p class="error">data.js가 없습니다. build_static_site.py를 실행하세요.</p>';
404        return;
405      }
406
407      catalog = data.catalog;
408      articles = data.articles || {};
409      sourceMap = data.sourceMap || {};
410      sourceBuildId = data.sourceBuildId || "";
411      sourceCache = {};
412      if (data.sources && !data.sourceMap) {
413        sourceCache = data.sources;
414      }
415      renderIntro();
416      renderGrid();
417    }
418
419    modalClose.addEventListener("click", closeModal);
420    sourceClose.addEventListener("click", closeSourceModal);
421    overlay.addEventListener("click", function (e) {
422      if (e.target === overlay) closeModal();
423    });
424    sourceOverlay.addEventListener("click", function (e) {
425      if (e.target === sourceOverlay) closeSourceModal();
426    });
427
428    document.addEventListener("keydown", function (e) {
429      if (e.key !== "Escape") return;
430      if (!sourceOverlay.classList.contains("hidden")) {
431        closeSourceModal();
432        return;
433      }
434      if (!overlay.classList.contains("hidden")) closeModal();
435    });
436
437    searchInput.addEventListener("input", function (e) {
438      searchQuery = e.target.value;
439      renderGrid();
440    });
441
442    if (document.readyState === "loading") {
443      document.addEventListener("DOMContentLoaded", init);
444    } else {
445      init();
446    }
447  })();
```

study_note_record_organization/dashboard/js/hero.js

```
/**
 * 학습 기록 대시보드 상단에서 이미지 두 장을 번갈아 보여 주는 히어로 갤러리를 초기화한다.
 */
(function () {
  "use strict";

  var AUTO_MS = 5500;
  var FADE_MS = 1200;

  /**
   * 상대 경로의 파일 이름만 인코딩해 img src로 쓸 수 있는 주소를 만든다.
   * @param {string} rel data/image 기준 상대 경로
   * @returns {string}
   */
  function heroSrc(rel) {
    var slash = rel.lastIndexOf("/");
    if (slash === -1) {
      return encodeURI(rel);
    }
    return rel.slice(0, slash + 1) + encodeURIComponent(rel.slice(slash + 1));
  }

  /**
   * 히어로에 쓸 이미지 경로 목록을 가져온다.
   * hero-images.js의 HERO_IMAGES를 우선하고, 없으면 SITE_DATA.heroImages를 본다.
   * @returns {string[]}
   */
  function getHeroImages() {
    if (window.HERO_IMAGES && window.HERO_IMAGES.length) {
      return window.HERO_IMAGES;
    }
    var data = window.SITE_DATA;
    if (data && data.heroImages && data.heroImages.length) {
      return data.heroImages;
    }
    return [];
  }

  /**
   * 히어로 영역을 찾고 이미지가 있으면 첫 장을 띄운 뒤, 클릭과 타이머로 다음 장으로 넘긴다.
   * 이미지가 없으면 히어로 구역을 숨긴다.
   */
  function initHero() {
    var section = document.getElementById("hero");
    var stage = document.getElementById("hero-stage");
    var imgA = document.getElementById("hero-img-a");
    var imgB = document.getElementById("hero-img-b");
    if (!section || !stage || !imgA || !imgB) return;

    var images = getHeroImages();
    if (!images.length) {
      section.classList.add("hidden");
      return;
    }

    section.classList.remove("hidden");
    section.removeAttribute("hidden");

    var current = -1;
    var showA = true;
    var busy = false;
    var timer = null;

```

```
    /**
     * 지금 화면에 보이는 히어로 이미지 요소를 돌려준다.
     * @returns {HTMLImageElement}
     */
    function visibleImg() {
      return showA ? imgA : imgB;
    }

    /**
     * 다음에 보여 주기 위해 숨겨 둔 히어로 이미지 요소를 돌려준다.
     * @returns {HTMLImageElement}
     */
    function hiddenImg() {
      return showA ? imgB : imgA;
    }

    /**
     * 현재 장과 다른 이미지 인덱스를 무작위로 고른다. 한 장이면 0을 돌려준다.
     * @returns {number}
     */
    function pickIndex() {
      if (images.length === 1) return 0;
      var idx;
      do {
        idx = Math.floor(Math.random() * images.length);
      } while (idx === current);
      return idx;
    }

    /**
     * 이미지 요소의 src와 대체 텍스트를 목록의 해당 장으로 바꾼다.
     * @param {HTMLImageElement} imgEl 대상 이미지
     * @param {number} index 이미지 목록 인덱스
     */
    function applyImage(imgEl, index) {
      var rel = images[index];
      imgEl.src = heroSrc(rel);
      imgEl.alt = "학습·과제 기록 갤러리 이미지";
    }

    /**
     * 숨겨 둔 이미지에 다음 장을 불러온 뒤, 로드가 끝나면 보이는 장을 교체한다.
     * 이미 전환 중이거나 같은 장이면 아무것도 하지 않는다.
     * @param {number} index 보여줄 이미지 인덱스
     */
    function swapTo(index) {
      if (busy || index === current) return;
      busy = true;

      var next = hiddenImg();
      var prev = visibleImg();
      applyImage(next, index);
      current = index;

      /**
       * 새 이미지에 표시 클래스를 옮기고 전환 중 상태를 해제한다.
       */
      function finish() {
        next.classList.add("is-visible");
        prev.classList.remove("is-visible");
        showA = next === imgA;
        busy = false;
      }

      if (next.complete && next.naturalWidth > 0) {
        finish();
      } else {
```

```
        next.onload = function () {
          next.onload = null;
          finish();
        };
        next.onerror = function () {
          next.onerror = null;
          busy = false;
        };
      }
    }

    /**
     * 현재와 다른 장을 골라 교체한다.
     */
    function nextRandom() {
      swapTo(pickIndex());
    }

    /**
     * 자동 전환 타이머를 다시 시작한다.
     * 동작 줄이기가 켜져 있거나 이미지가 한 장이면 타이머를 두지 않는다.
     */
    function startTimer() {
      if (timer) clearInterval(timer);
      var reduced =
        window.matchMedia &&
        window.matchMedia("(prefers-reduced-motion: reduce)").matches;
      if (!reduced && images.length > 1) {
        timer = setInterval(nextRandom, AUTO_MS);
      }
    }

    current = pickIndex();
    applyImage(imgA, current);
    imgA.classList.add("is-visible");

    stage.addEventListener("click", function () {
      nextRandom();
      startTimer();
    });

    startTimer();
  }

  if (document.readyState === "loading") {
    document.addEventListener("DOMContentLoaded", initHero);
  } else {
    initHero();
  }
})();
```

study_note_record_organization/scripts/build_output.py

```python
# -*- coding: utf-8 -*-
"""학습 노트와 과제 원본에서 텍스트를 뽑고, 카탈로그와 요약 마크다운을 만든다."""
from __future__ import annotations

import json
import os
import re
import textwrap
from urllib.parse import quote
from dataclasses import dataclass, field
from html.parser import HTMLParser
from pathlib import Path
from typing import Any

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"
ASSIGN = DATA / "과제분류하기"
OUT = ROOT / "output"
EXTRACTED = OUT / "_extracted"
STUDY_OUT = OUT / "study"
ASSIGN_OUT = OUT / "assignment"

UNIT_RE = re.compile(
    r"(\d+[~\-]?\d*\s*)?(장|일차|강|주차)|서론|프롤로그|부록|코딩면허",
    re.I,
)
NOTE_SUFFIX_RE = re.compile(
    r"\s*(독서\s*노트|독서노트|수강노트|영상노트|독서 노트)\s*$",
    re.I,
)


class MLStripper(HTMLParser):
    """HTML 태그 사이의 글자만 모아 순수 텍스트로 만드는 파서."""
    def __init__(self):
        """태그 사이에 모을 문자열 목록을 비워 둔다."""
        super().__init__()
        self.parts: list[str] = []

    def handle_data(self, data: str) -> None:
        """파서가 만난 텍스트 조각을 목록에 추가한다."""
        self.parts.append(data)

    def get_text(self) -> str:
        """모은 텍스트 조각을 줄바꿈으로 이어 한 문자열로 돌려준다."""
        return "\n".join(self.parts)


def read_text_file(path: Path) -> str:
    """utf-8, utf-8-sig, cp949 순으로 텍스트 파일을 읽는다. 모두 실패하면 빈 문자열을 돌려준다."""
    for enc in ("utf-8", "utf-8-sig", "cp949"):
        try:
            return path.read_text(encoding=enc)
        except (UnicodeDecodeError, OSError):
            continue
    return ""


def extract_docx(path: Path) -> str:
    """워드 문서의 비어 있지 않은 문단을 줄로 이어 뽑는다. python-docx가 없으면 안내 문구를 돌려준다."""
    try:
        from docx import Document
    except ImportError:
```

```
        return "[python-docx not installed]"
    doc = Document(str(path))
    return "\n".join(p.text for p in doc.paragraphs if p.text.strip())


def extract_pdf(path: Path) -> str:
    """PDF 각 페이지의 텍스트를 이어 붙인다. PyMuPDF가 없으면 안내 문구를 돌려준다."""
    try:
        import fitz
    except ImportError:
        return "[pymupdf not installed]"
    text_parts = []
    with fitz.open(str(path)) as pdf:
        for page in pdf:
            text_parts.append(page.get_text())
    return "\n".join(text_parts)


def extract_html(path: Path) -> str:
    """HTML 파일에서 태그를 제거하고 본문 글자만 돌려준다. 파싱에 실패하면 원문을 그대로 둔다."""
    raw = read_text_file(path)
    stripper = MLStripper()
    try:
        stripper.feed(raw)
        return stripper.get_text()
    except Exception:
        return raw


KEY_MASK = "142857"

_SECRET_PATTERNS = [
    # Google API Key
    re.compile(r"AIza[0-9A-Za-z\-_]{35}"),
    # OpenAI API Key
    re.compile(r"sk-[a-zA-Z0-9_\-]{20,}"),
    # Anthropic API Key
    re.compile(r"sk-ant-[a-zA-Z0-9_\-]{20,}"),
    # GitHub Token
    re.compile(r"(?:ghp|gho|ghu|ghs|ghr)_[a-zA-Z0-9]{36}|github_pat_[a-zA-Z0-9_]{50,}"),
    # AWS Access Key
    re.compile(r"(?:AKIA|ABIA|ACCA|ASIA)[0-9A-Z]{16}"),
    # Slack Token
    re.compile(r"xox[baprs]-[0-9a-zA-Z]{10,}"),
    # Firebase FCM device registration token
    re.compile(r"[a-zA-Z0-9_\-]{15,}:APA91b[a-zA-Z0-9_\-]{50,}"),
]

_KEY_ASSIGNMENT_PATTERNS = [
    re.compile(
        r'(?i)((?:api[_-]?key|client[_-]?secret|secret[_-]?key|access[_-]?token|service[_-]?key)\s*[:=]\s*["\'])([a-z
A-Z0-9_\-\.\%\+\=]{16,})(["\'])'
    ),
    re.compile(
        r'(?i)((?:android:name=["\']com\.google\.android\.maps\.v2\.API_KEY["\']\s+android:value=["\']))([^"\']+)
(["\'])'
    ),
]


def mask_secrets(text: str, mask: str = KEY_MASK) -> str:
    """본문에서 API 키·토큰 패턴을 찾아 고정 마스크 문자열로 바꾼다."""
    if not text:
        return text
    for pat in _SECRET_PATTERNS:
        text = pat.sub(mask, text)
    for pat in _KEY_ASSIGNMENT_PATTERNS:
```

```
        text = pat.sub(rf"\g<1>{mask}\g<3>", text)
    return text


def extract_source(path: Path) -> str:
    """확장자에 맞춰 텍스트, 워드, PDF, HTML, VB에서 본문을 뽑고 비밀 값을 가린다."""
    ext = path.suffix.lower()
    raw = ""
    if ext == ".txt":
        raw = read_text_file(path)
    elif ext == ".docx":
        raw = extract_docx(path)
    elif ext == ".pdf":
        raw = extract_pdf(path)
    elif ext in (".html", ".htm"):
        raw = extract_html(path)
    elif ext == ".vb":
        raw = read_text_file(path)
    return mask_secrets(raw)


def source_href(path: str) -> str:
    """data 기준 상대 경로를 사이트에서 여는 /data/ 주소로 바꾼다. 경로 조각은 URL 인코딩한다."""
    rel = path.replace("\\", "/")
    if rel.startswith("data/"):
        rel = rel[5:]
    return "/data/" + "/".join(quote(seg, safe="/") for seg in rel.split("/"))


def extracted_href(path: str) -> str:
    """원본 경로를 안전한 파일 이름으로 바꿔 추출 텍스트 주소 /_extracted/...txt 를 만든다."""
    safe = re.sub(r"[^\w\-]", "_", path)[:120]
    return f"/_extracted/{safe}.txt"


def source_link_target(s: SourceRef) -> tuple[str, str]:
    """원본 종류에 따라 링크에 보일 이름과 클릭 주소를 정한다.

    워드와 PDF는 추출 텍스트로, 텍스트·HTML·VB는 원본 경로로 연결한다.
    """
    p = ROOT / s.path
    if p.is_dir():
        vb = p / "Form1.vb"
        if vb.is_file():
            rel = f"{s.path.replace(chr(92), '/')}/Form1.vb"
            return "Form1.vb", source_href(rel)
        return os.path.basename(s.path), source_href(s.path)

    name = os.path.basename(s.path)
    ext = p.suffix.lower()
    if ext in (".docx", ".pdf"):
        return f"{name} (텍스트 추출본)", extracted_href(s.path)
    if ext in (".html", ".htm", ".txt", ".vb"):
        return name, source_href(s.path)
    return name, source_href(s.path)


PROFESSOR_MASK = "OOO 교수님"
STUDENT_ID_MASK = "OOOOOOOO"
_ASSIGNMENT_DIR = "과제분류하기"
_ID_META_KEYS = frozenset({"학번", "수강번호"})


def is_assignment_path(rel_path: str) -> bool:
    """경로에 과제 폴더 이름이 있으면 참을 돌려준다."""
    return _ASSIGNMENT_DIR in rel_path.replace("\\", "/")

```

```

def mask_assignment_privacy(text: str) -> str:
    """과제 본문의 지도교수 이름, 학번, 파일명 속 학번 숫자를 가린다."""
    if not text:
        return text
    text = re.sub(
        r"(지도교수\s*[:：]\s*)\S+(?:\s*교수님)?",
        rf"\1{PROFESSOR_MASK}",
        text,
    )
    text = re.sub(
        r"((?:학\s*번|수\s*강\s*번\s*호)\s*[:：]\s*)\d+",
        rf"\1{STUDENT_ID_MASK}",
        text,
    )
    text = re.sub(r"\(\d{7,8}\s+", f"({STUDENT_ID_MASK} ", text)
    text = re.sub(r"\(\d{7,8}\)", f"({STUDENT_ID_MASK})", text)
    text = re.sub(r"_\d{7,8}_", f"_{STUDENT_ID_MASK}_", text)
    return text


def cache_extract(rel_path: str, content: str) -> Path:
    """뽑은 본문을 output/_extracted 아래 텍스트 파일로 저장하고 그 경로를 돌려준다."""
    safe = re.sub(r"[^\w\-]", "_", rel_path)[:120]
    out = EXTRACTED / f"{safe}.txt"
    out.parent.mkdir(parents=True, exist_ok=True)
    clean_content = mask_secrets(content)
    out.write_text(clean_content or "[empty]", encoding="utf-8")
    return out


def parse_unit(filename: str) -> str:
    """파일 이름에서 장, 일차, 강, 주차, 서론, 부록, 코딩면허 같은 학습 단위를 찾는다. 없으면 전체다."""
    name = filename.replace(".txt", "")
    for pat in [
        r"(\d+[~\-]?\d*)\s*장",
        r"(\d+[~\-]?\d*)\s*일차",
        r"(\d+[~\-]?\d*)\s*강",
        r"(서론|프롤로그|부록)",
        r"코딩면허",
        r"(\d+[~\-]?\d*)\s*주차",
    ]:
        m = re.search(pat, name, re.I)
        if m:
            return m.group(0)
    return "전체"


def sort_key_unit(unit: str) -> tuple:
    """학습 단위를 서론, 구간, 본문, 부록 순으로 정렬하기 위한 키를 만든다."""
    if unit == "전체":
        return (0, 0)
    if "서론" in unit or "프롤로그" in unit:
        return (1, 0)
    nums = re.findall(r"\d+", unit)
    n = int(nums[0]) if nums else 99
    if "~" in unit or "-" in unit:
        return (2, n)
    if "부록" in unit:
        return (4, n)
    return (3, n)


@dataclass
class SourceRef:
    """카탈로그 항목이 가리키는 원본 파일 하나. 경로, 학습 단위, 추출한 본문을 담는다."""
    path: str
```

```
    unit: str = "전체"
    text: str = ""


@dataclass
class CatalogItem:
    """학습 또는 과제 카드 하나. 제목, 슬러그, 태그, 원본 목록을 갖고 마크다운으로 렌더된다."""
    id: str
    type: str
    category: str
    title: str
    slug: str
    summary: str
    tags: list[str]
    sources: list[SourceRef] = field(default_factory=list)

    def load_sources(self) -> None:
        """각 원본 파일의 본문을 읽어 text에 채우고, 가린 내용을 추출 캐시에 저장한다. 폴더면 Form1.vb를 합친다."""
        for s in self.sources:
            p = ROOT / s.path
            if p.is_file():
                s.text = extract_source(p)
                if is_assignment_path(s.path):
                    cache_extract(s.path, mask_assignment_privacy(s.text))
                else:
                    cache_extract(s.path, s.text)
            elif p.is_dir():
                for vb in sorted(p.rglob("*.vb")):
                    if "obj" in vb.parts or "bin" in vb.parts:
                        continue
                    if vb.name in ("Form1.vb", "Form1.Designer.vb"):
                        t = extract_source(vb)
                        s.text += f"\n\n--- {vb.name} ---\n{t}"
                if is_assignment_path(s.path):
                    cache_extract(s.path, mask_assignment_privacy(s.text))
                else:
                    cache_extract(s.path, s.text)


STUDY_MERGE_ALIASES: dict[str, str] = {
    "UI디자인&프로그래밍 서론": "UI디자인&프로그래밍",
    "커서AI 트랜드 &활용백과 프롤로그": "커서AI 트랜드 &활용백과",
    "커서AI 트랜드 &활용백과 부록": "커서AI 트랜드 &활용백과",
    "커서AI 트랜드 &활용백과 3~4장": "커서AI 트랜드 &활용백과",
    "된다! 블로그 10분 작성법 3~": "된다! 블로그 10분 작성법",
    "피지컬 컴퓨팅 데이터 수집 및 처리 - 노시훈": "피지컬 컴퓨팅 데이터 수집 및 처리",
    "페스트캠퍼스 일스트레이터 입문 팁": "페스트캠퍼스 일스트레이터 입문",
}

# slug, summary, tags, category — title은 그룹 키(과목명)
STUDY_META: dict[str, tuple[str, str, list[str], str]] = {
    "Do it! Node.js 프로그래밍 입문": ("doit-nodejs", "Node.js·Express·MongoDB 백엔드 입문", ["nodejs", "javascript"],
"book"),
    "콜로소 바이브 코딩": ("coloso-vibe-coding", "Cursor AI·바이브 코딩·웹 배포", ["cursor", "ai", "web"], "course"),
    "UI디자인&프로그래밍": ("ui-design-programming", "UI·앱 아이디어·디자인·개발 프로세스", ["ui", "design"], "book"),
    "Do it 5일만에 끝내는 깃&깃허브 입문": ("doit-git-github", "Git·GitHub 협업·Pages", ["git", "github"], "book"),
    "Do it 점프 투 파이썬": ("doit-jump-python", "파이썬 기초·코딩면허", ["python"], "book"),
    "커서AI 트랜드 &활용백과": ("cursor-ai-encyclopedia", "Cursor AI 트렌드·활용", ["cursor", "ai"], "book"),
    "Do it 커서로 시작하는 AI 코딩 입문": ("doit-cursor-ai-intro", "AI 코딩·Cursor 입문", ["cursor", "ai"], "book"),
    "인프런 취준생이 반드시 일겅봐야 할 중고신입 전략": ("inflearn-job-strategy", "취업·면접·이력서 전략", ["career", "jo
b"], "course"),
    "Do it 첫 알고리즘": ("doit-first-algorithm", "알고리즘·자료구조·정렬", ["algorithm"], "book"),
    "된다! 블로그 10분 작성법": ("blog-10min", "블로그·콘텐츠 작성", ["blog", "writing"], "book"),
    "코딩펙토리 flutter Dart": ("codingfactory-flutter", "Flutter·Dart 앱 개발", ["flutter", "dart"], "course"),
    "Do it! HTML + CSS + 자바스크립트 웹 표준의 정석": ("doit-html-css-js", "웹 표준·HTML/CSS/JS", ["html", "css", "java
script"], "book"),
    "Do it! 모던 자바스크립트 프로그래밍의 정석": ("doit-modern-javascript", "모던 JavaScript", ["javascript"], "book"),
```

```
    "가장 쉬운 독학 노션 첫걸음": ("notion-first-steps", "노션 워크스페이스·DB", ["notion", "productivity"], "book"),
    "과학동아 323호 AI 공정한가": ("science-donga-323", "AI 윤리·공정성", ["ai", "ethics"], "book"),
    "과학동아 40주년 AI 특이점, 그후 40년": ("science-donga-40", "AI 특이점·미래 전망", ["ai", "future"], "book"),
    "그래도 나아간다는 믿음": ("faith-forward", "자기계발·믿음", ["self-help"], "book"),
    "꿈은 이루어진다": ("dreams-come-true", "자기계발·목표", ["self-help"], "book"),
    "대기업을 이긴 한국의 스타트업": ("korean-startups", "스타트업·비즈니스", ["startup", "business"], "book"),
    "맛있는 디자인 에프터 이펙트": ("tasty-after-effects", "AE 실습", ["after-effects", "design"], "course"),
    "맛있는 디자인 일러스트레이터": ("tasty-illustrator", "일러스트 실습", ["illustrator", "design"], "course"),
    "맛있는 디자인 포토샵": ("tasty-photoshop", "포토샵 실습", ["photoshop", "design"], "course"),
    "맛있는 디자인 프리미어 프로": ("tasty-premiere", "프리미어 실습", ["premiere", "video"], "course"),
    "소스 코드, 더 비기닝": ("source-code-beginning", "IT 역사·스타트업", ["history", "startup"], "book"),
    "스마트폰으로 AI영상 제작": ("smartphone-ai-video", "모바일 AI 영상", ["ai", "video"], "book"),
    "스토리가 스펙을 이긴다": ("story-beats-spec", "스토리·커리어", ["career", "story"], "book"),
    "요즘 바이브 코딩 커서 AI 30가지 프로그램만들기": ("vibe-coding-30-apps", "바이브 코딩 30 프로젝트", ["cursor", "ai"],
"book"),
    "정치가 왜이래": ("politics-why", "정치·시사", ["politics"], "book"),
    "트렌드 코리아 2026": ("trend-korea-2026", "2026 트렌드", ["trend", "business"], "book"),
    "패스트캠퍼스 안드로이드 코틀린": ("fastcampus-android-kotlin", "안드로이드·Kotlin", ["android", "kotlin"], "cours
e"),
    "퍼플렉시티 완벽가이드": ("perplexity-guide", "Perplexity AI 활용", ["ai", "search"], "book"),
    "페스트캠퍼스 일스트레이터 입문": ("fastcampus-illustrator", "일러스트레이터 입문·팁", ["illustrator", "design"], "cou
rse"),
    "페스트캠퍼스 포토샵": ("fastcampus-photoshop", "포토샵 입문", ["photoshop", "design"], "course"),
    "프롬프트 텔링": ("prompt-telling", "프롬프트·스토리텔링", ["ai", "prompt"], "book"),
    "피지컬 컴퓨팅 데이터 수집 및 처리": ("physical-computing", "피지컬 컴퓨팅·센서", ["arduino", "iot"], "book"),
    "Do it android kotlin": ("doit-android-kotlin", "Android·Kotlin (Do it)", ["android", "kotlin"], "book"),
    "AI 에전트 트렌드 & 활용백과 독서노트": (
        "ai-agent-encyclopedia",
        "AI 에이전트 트렌드·활용 통합 노트",
        ["ai", "agent"],
        "book",
    ),
}

# 과목별 서술형 구어체 요약 (원문 메모를 그대로 나열하지 않음)
STUDY_NARRATIVES: dict[str, str] = {
    "doit-nodejs": (
        "백엔드가 어떻게 돌아가는지 처음부터 손으로 익혀 본 과목이에요. "
        "자바스크립트·Node.js·npm 개념을 잡고, Express로 서버를 만들고, "
        "데이터베이스와 CRUD, 인증까지 단계를 올려 가며 실습했어요. "
        "프론트와 서버가 어떻게 맞물리는지 감이 잡히기 시작한 시점이었어요."
    ),
    "coloso-vibe-coding": (
        "Cursor AI로 말하듯 코딩하고, 만든 결과를 웹에 올려 보는 흐름을 익힌 강의예요. "
        "자연어로 요청해 코드를 받고, HTML·CSS·JS로 페이지를 다듬은 뒤 "
        "FTP·호스팅으로 배포까지 연결해 봤어요. "
        "바이브 코딩이 실제 작업 속도를 어떻게 바꾸는지 체감한 과목이에요."
    ),
    "ui-design-programming": (
        "앱·서비스를 만들 때 아이디어부터 디자인, 개발까지 이어지는 과정을 정리한 책이에요. "
        "무엇을 만들지 구상하고, 사용자 관점에서 화면을 설계한 다음 "
        "구현과 피드백을 반복하는 흐름을 배웠어요. "
        "기술만이 아니라 기획·디자인 사고방식을 같이 챙긴 학습이었어요."
    ),
    "doit-git-github": (
        "Git과 GitHub를 처음 제대로 써 본 기록이에요. "
        "저장소·브랜치·커밋·병합 같은 기본기부터 협업·원격·Pages 배포까지 "
        "차근차근 따라 가며 익혔어요. "
        "버전 관리가 왜 필요한지, 실무에서 어떻게 쓰이는지 감이 잡힌 과목이에요."
    ),
    "doit-jump-python": (
        "파이썬 문법과 기본기를 처음부터 쌓은 과목이에요. "
        "설치·자료형·제어문·함수·클래스, 파일·인코딩, 예외 처리·실습 예제를 거치고 "
        "마지막에는 코딩면허 스타일 문제도 풀어 봤어요. "
        "데이터·AI·웹까지 파이썬이 어디에 쓰이는지 큰 그림도 같이 잡았어요."
    ),
    "cursor-ai-encyclopedia": (
```

```
        "Cursor AI가 어떤 도구인지, 어떻게 쓰면 생산성이 오르는지 훑어 본 책이에요. "
        "설치·기본 사용법부터 프롬프트·프로젝트 활용, 트렌드·부록까지 "
        "AI 코딩 환경을 한 바퀴 돌아본 느낌이에요. "
        "에디터와 AI가 합쳐진 작업 방식을 익히는 데 초점을 둔 학습이었어요."
    ),
    "doit-cursor-ai-intro": (
        "Cursor로 AI 코딩을 시작하는 입문 코스예요. "
        "환경 설정·프롬프트 작성·간단한 프로젝트 실습을 하루 단위로 진행했고, "
        "말로 요청하고 코드를 고치는 반복에 익숙해졌어요. "
        "처음 AI 코딩 툴을 손에 쥔 때의 로드맵을 정리한 기록이에요."
    ),
    "inflearn-job-strategy": (
        "취업·이직을 준비할 때 서류·면접·전략을 점검한 강의예요. "
        "이력서·자기소개·면접 대비를 단계별로 정리하고, "
        "중고신입으로서 어떤 포인트를 어필해야 하는지 배웠어요. "
        "기술만이 아니라 커리어 설계 관점에서 돌아본 학습이었어요."
    ),
    "doit-first-algorithm": (
        "알고리즘과 자료구조를 입문자 눈높이로 익힌 책이에요. "
        "시간 복잡도·기본 자료구조·정렬·탐색을 차례로 다루며 "
        "왜 이런 구조와 알고리즘이 쓰이는지 이해하려 했어요. "
        "코딩 테스트·면접 준비의 밑바닥을 다지는 과목이었어요."
    ),
    "blog-10min": (
        "블로그 글을 짧은 시간 안에 쓰는 루틴을 익힌 책이에요. "
        "주제 잡기·초안·발행까지 하루 단위로 연습하며 "
        "꾸준히 콘텐츠를 낼 수 있는 습관을 만들려 했어요. "
        "기록과 글쓰기를 병행하는 데 도움이 된 학습이었어요."
    ),
    "codingfactory-flutter": (
        "Flutter와 Dart로 앱 개발 입문을 시도한 강의예요. "
        "위젯·화면 구성·기본 문법을 강 단위로 따라 가며 "
        "크로스플랫폼 앱이 어떻게 짜이는지 훑어 봤어요. "
        "모바일 UI를 코드로 만드는 첫 경험에 가까웠어요."
    ),
    "doit-html-css-js": (
        "웹 표준에 맞춰 HTML·CSS·자바스크립트 기초를 정리한 책이에요. "
        "문서 구조·스타일·동작을 나눠 이해하고, "
        "브라우저에서 페이지가 어떻게 그려지는지 기초를 다졌어요. "
        "프론트엔드와 퍼블리싱의 출발점을 만든 학습이었어요."
    ),
    "doit-modern-javascript": (
        "모던 JavaScript 문법과 실습 위주로 필기한 기록이에요. "
        "ES6 이후 자주 쓰는 문법·패턴을 정리하고, "
        "예제를 따라 하며 최신 JS 스타일에 익숙해지려 했어요. "
        "Node·프론트로 가기 전 JS 기초를 보강한 과목이에요."
    ),
    "notion-first-steps": (
        "노션으로 워크스페이스와 데이터베이스를 처음 구성해 본 책이에요. "
        "요금제·페이지·DB·템플릿 개념을 익히고, "
        "개인 기록·할 일·자료를 한곳에 모으는 방법을 배웠어요. "
        "생산성 도구로 노션을 쓰기 시작한 계기가 된 학습이었어요."
    ),
    "science-donga-323": (
        "AI가 공정한지, 편향·윤리 문제는 없는지 다룬 과학동아 기사 학습이에요. "
        "기술만이 아니라 사회·윤리 관점에서 AI를 바라보는 "
        "짧지만 인상 깊은 독서 노트였어요."
    ),
    "science-donga-40": (
        "AI 특이점 이후 40년을 상상해 본 과학동아 특집을 읽고 정리했어요. "
        "기술·사회 변화를 장기 시계로 바라보는 관점을 "
        "가볍게 훑어 본 학습이었어요."
    ),
    "faith-forward": (
        "힘들 때도 나아간다는 믿음·마음가짐을 다룬 자기계발 독서예요. "
        "기술 서적과는 결이 다르게, 삶의 태도와 "
        "멘탈을 돌아보는 시간을 가졌어요."
```

```
    ),
    "dreams-come-true": (
        "꿈과 목표를 이루는 마음가짐을 다룬 자기계발 책이에요. "
        "무엇을 원하는지, 어떻게 밀고 나갈지에 대한 "
        "짧은 메모형 독서였어요."
    ),
    "korean-startups": (
        "대기업과 맞서 성장한 한국 스타트업 사례를 읽고 정리했어요. "
        "창업·비즈니스·기술이 어떻게 맞물리는지 "
        "이야기 위주로 훑어 본 책이에요."
    ),
    "tasty-after-effects": (
        "애프터 이펙트로 모션·합성을 실습 위주로 익힌 강의예요. "
        "효과·타임라인·레이어 작업을 따라 하며 "
        "영상 후반 작업이 어떻게 이뤄지는지 감을 잡았어요."
    ),
    "tasty-illustrator": (
        "일러스트레이터로 벡터 작업을 실습한 기록이에요. "
        "도구·패스·편집 기본을 익히고 "
        "그래픽 작업의 기초를 쌓는 데 집중했어요."
    ),
    "tasty-photoshop": (
        "포토샵으로 합성·보정을 실습 위주로 배운 강의예요. "
        "레이어·선택·보정 도구를 쓰며 "
        "이미지 편집 워크플로를 익혔어요."
    ),
    "tasty-premiere": (
        "프리미어 프로로 영상 편집 실습을 진행한 강의예요. "
        "컷 편집·타임라인·보내기까지 "
        "영상 제작 기초를 따라 해 본 학습이었어요."
    ),
    "source-code-beginning": (
        "IT·스타트업 역사를 인물과 사건 중심으로 읽은 책이에요. "
        "기술이 어떻게 비즈니스와 맞물려 왔는지 "
        "이야기로 풀어 보며 큰 그림을 챙긴 독서였어요."
    ),
    "smartphone-ai-video": (
        "스마트폰만으로 AI 영상을 만드는 방법을 익힌 책이에요. "
        "촬영·편집·AI 도구를 활용해 "
        "가벼운 영상 제작 파이프라인을 정리했어요."
    ),
    "story-beats-spec": (
        "스토리와 경험이 스펙만큼 중요하다는 관점의 책이에요. "
        "이력·자기소개·커리어를 이야기로 풀어야 한다는 "
        "메시지를 중심으로 읽고 정리했어요."
    ),
    "vibe-coding-30-apps": (
        "Cursor AI로 30가지 가벼운 프로그램을 만들어 보는 흐름을 담은 책이에요. "
        "아이디어·프롬프트·빠른 구현을 반복하며 "
        "바이브 코딩으로 무엇을 만들 수 있는지 넓혀 본 학습이었어요."
    ),
    "politics-why": (
        "정치·시사 이슈를 가볍게 읽고 생각해 본 독서예요. "
        "기술 학습과는 다른 맥락에서 "
        "사회 흐름을 이해하려 한 기록이에요."
    ),
    "trend-korea-2026": (
        "2026년 소비·기술·사회 트렌드를 한권에 정리한 책이에요. "
        "앞으로 어떤 키워드가 뜰지 "
        "미리 훑어 보며 시야를 넓힌 독서였어요."
    ),
    "fastcampus-android-kotlin": (
        "안드로이드 앱 개발을 Kotlin으로 입문한 강의예요. "
        "앱 구조·Kotlin 문법·기본 화면을 익히며 "
        "모바일 네이티브 개발의 문을 열어 본 학습이었어요."
    ),
    "perplexity-guide": (
```

```
        "Perplexity AI 검색·질의 도구를 활용하는 방법을 익힌 책이에요. "
        "출처와 함께 답을 받는 검색 경험과 "
        "리서치·학습에 쓰는 팁을 정리했어요."
    ),
    "fastcampus-illustrator": (
        "일러스트레이터 입문과 실무 팁을 함께 정리한 강의예요. "
        "기본 도구 사용법부터 단축키·작업 요령까지 "
        "벡터 작업 입문을 마친 과목이에요."
    ),
    "fastcampus-photoshop": (
        "포토샵 기초 도구와 합성 입문을 배운 강의예요. "
        "레이어·마스크·보정을 익히며 "
        "이미지 편집의 첫 단계를 밟았어요."
    ),
    "prompt-telling": (
        "프롬프트를 스토리처럼 설계하는 방법을 다룬 책이에요. "
        "AI에게 무엇을 어떻게 말해야 원하는 결과가 나오는지 "
        "구조·표현 관점에서 정리했어요."
    ),
    "physical-computing": (
        "아두이노·센서로 데이터를 수집하고 처리하는 피지컬 컴퓨팅을 공부했어요. "
        "하드웨어와 소프트웨어가 만나는 지점을 익히고, "
        "실습·보충 노트를 통해 IoT 기초 감각을 쌓았어요."
    ),
    "doit-android-kotlin": (
        "Do it 시리즈로 Android·Kotlin 기초를 정리한 학습이에요. "
        "앱 개발 환경·문법·화면 구성을 익히며 "
        "모바일 개발 학습을 이어 간 과목이에요."
    ),
    "ai-agent-encyclopedia": (
        "AI 에이전트 트렌드와 활용을 한권에 모아 읽은 통합 노트예요. "
        "에이전트 개념·도구·활용 방향을 훑으며 "
        "Cursor·AI 코딩과 연결해 본 학습이었어요."
    ),
}


def normalize_study_title(filename: str) -> str:
    """노트 파일 이름에서 장·일차 번호와 독서노트·수강노트 꼬리를 떼고 과목 이름만 남긴다."""
    n = filename.replace(".txt", "").replace(".docx", "")
    n = re.sub(
        r"\s+(\d+[~\-]?\d*\s*)?(장|일차|강|주차)\s*(독서\s*노트|독서노트|수강노트|영상노트|독서 노트)\s*$",
        "",
        n,
        flags=re.I,
    )
    n = re.sub(r"\s+(독서노트|수강노트|영상노트|독서 노트)\s*$", "", n, flags=re.I)
    n = re.sub(r"\s+코딩면허.*$", "", n, flags=re.I)
    n = re.sub(r"\s+노트는\s*실습.*$", "", n, flags=re.I)
    n = re.sub(r"\s+실습\s*필기.*$", "", n, flags=re.I)
    n = re.sub(r"\s+실습위주.*$", "", n, flags=re.I)
    n = re.sub(r"\s+통합\(단순합치기\).*$", "", n)
    return n.strip()


def collect_study_groups() -> dict[str, list[str]]:
    """data 폴더의 텍스트·워드 노트를 과목 이름으로 묶는다.

    hwpx는 빼고, 별칭은 같은 과목으로 합친 뒤 장·강 순으로 정렬한다.
    """
    raw: dict[str, list[str]] = {}
    skipped_hwpx: list[str] = []

    for f in sorted(DATA.iterdir()):
        if not f.is_file():
            continue
        ext = f.suffix.lower()
```

```
        if ext == ".hwpx":
            skipped_hwpx.append(f.name)
            continue
        if ext not in (".txt", ".docx"):
            continue
        base = normalize_study_title(f.name)
        raw.setdefault(base, []).append(f"data/{f.name}")

    merged: dict[str, list[str]] = {}
    for base, paths in raw.items():
        canonical = STUDY_MERGE_ALIASES.get(base, base)
        merged.setdefault(canonical, []).extend(paths)

    for paths in merged.values():
        paths.sort(key=lambda p: sort_key_unit(parse_unit(os.path.basename(p))))

    return merged, skipped_hwpx


def build_study_catalog() -> list[CatalogItem]:
    """과목 그룹이 37개가 아니면 감사 파일을 남기고 중단한다. 맞으면 학습 CatalogItem 목록을 만든다."""
    groups, skipped_hwpx = collect_study_groups()

    if len(groups) != 37:
        report = OUT / "study-audit.txt"
        lines = [
            f"학습 그룹 수: {len(groups)} (기대: 37)",
            f"제외 hwpx: {len(skipped_hwpx)}",
            "",
            "=== 그룹 목록 ===",
        ]
        for k in sorted(groups):
            lines.append(f"  [{len(groups[k]):2d}] {k}")
        if skipped_hwpx:
            lines.extend(["", "=== 제외 hwpx ===", *[f"  {x}" for x in skipped_hwpx]])
        report.write_text("\n".join(lines), encoding="utf-8")
        raise SystemExit(f"학습 과목 수가 37개가 아닙니다 ({len(groups)}개). output/study-audit.txt 참고")

    items: list[CatalogItem] = []
    for title in sorted(groups.keys()):
        paths = groups[title]
        meta = STUDY_META.get(title)
        if meta:
            slug, summary, tags, category = meta
        else:
            slug = re.sub(r"[^\w]+", "-", title.lower())[:48].strip("-") or "study-unknown"
            summary = f"{title} 학습 노트"
            tags = ["study"]
            category = "book" if "Do it" in title else "course"

        items.append(
            CatalogItem(
                id=f"study-{slug}",
                type="study",
                category=category,
                title=title,
                slug=slug,
                summary=summary,
                tags=tags,
                sources=[
                    SourceRef(path=p, unit=parse_unit(os.path.basename(p))) for p in paths
                ],
            )
        )

    return items

```

```
659
660 def find_assign_file(files: list[str], keywords: list[str], used: set[str], exclude_contains: list[str] | None = Non
e) -> str | None:
661     """아직 쓰지 않은 과제 파일 중 키워드가 들어 있고 제외어가 없는 첫 파일을 찾는다."""
662     excludes = exclude_contains or []
663     for kw in keywords:
664         for f in files:
665             if f in used:
666                 continue
667             if any(ex in f for ex in excludes):
668                 continue
669             if kw in f:
670                 return f
671     return None
672
673
674 def build_assignment_catalog() -> list[CatalogItem]:
675     """과제 폴더의 파일 이름과 과목 키워드를 맞춰 과제 카탈로그를 만든다.
676
677     윈도우 프로그래밍1은 텍스트 문서가 없어 출처 목록을 비워 둔다.
678     """
679     files = [f for f in os.listdir(ASSIGN) if (ASSIGN / f).is_file()]
680     used: set[str] = set()
681     items: list[CatalogItem] = []
682
683     def add(slug: str, title: str, keywords: list[str], tags: list[str], exclude_contains: list[str] | None = None, e
xtra_sources: list[SourceRef] | None = None):
684         """키워드로 과제 파일을 하나 묶어 과제 CatalogItem을 목록에 추가한다. 추가 출처가 있으면 같이 넣는다."""
685         matched = find_assign_file(files, keywords, used, exclude_contains)
686         srcs: list[SourceRef] = []
687         if matched:
688             used.add(matched)
689             srcs.append(SourceRef(path=f"data/과제분류하기/{matched}", unit="총괄과제"))
690         if extra_sources:
691             srcs.extend(extra_sources)
692         items.append(
693             CatalogItem(
694                 id=f"assignment-{slug}",
695                 type="assignment",
696                 category="university",
697                 title=title,
698                 slug=slug,
699                 summary=f"{title} 총괄/최종 과제 수행 기록",
700                 tags=tags,
701                 sources=srcs,
702             )
703         )
704
705     add("c-language-1", "C언어1", ["C언어1"], ["c", "pointer"])
706     add("pc-skills-1", "PC활용1", ["PC활용1"], ["office", "pc"])
707     add("database", "데이터베이스", ["데이터베이스"], ["database", "sql"])
708     add("multimedia-intro", "멀티미디어 개론", ["멀티미디어 개론", "멀티미디어개론"], ["multimedia"])
709     add("multimedia-telecom", "멀티미디어 통신", ["멀티미디어통신"], ["multimedia", "telecom"])
710     add("software-engineering", "소프트웨어공학", ["소프트웨어공학"], ["software-engineering"])
711     add("system-analysis-design", "시스템분석설계", ["시스템분석설계"], ["system", "analysis"])
712     add("discrete-math", "이산수학", ["이산수학"], ["math"])
713     add("internet-marketing", "인터넷 마케팅", ["인터넷 마케팅"], ["marketing", "internet"])
714     add("internet-programming", "인터넷프로그래밍", ["인터넷프로그래밍_"], ["html", "web"], exclude_contains=["프로그래밍
2"])
715     add("internet-programming-2", "인터넷프로그래밍2", ["인터넷프로그래밍2"], ["html", "web"])
716     add("data-structures", "자료구조", ["자료구조"], ["algorithm", "data-structure"])
717     add("information-processing", "정보처리", ["정보처리"], ["information"])
718     add("computer-intro", "컴퓨터개론", ["컴퓨터개론"], ["computer-science"])
719     add("computer-graphics-1", "컴퓨터그래픽1", ["컴퓨터그래픽1"], ["graphics"])
720     add("computer-network", "컴퓨터네트워크", ["컴퓨터네트워크"], ["network", "protocol"])
721     items.append(
722         CatalogItem(
```

```
            id="assignment-windows-programming-1",
            type="assignment",
            category="university",
            title="윈도우 프로그래밍1",
            slug="windows-programming-1",
            summary="WinForms 실행 프로그램 제출 (텍스트 문서 없음)",
            tags=["vbnet", "winforms", "program"],
            sources=[],
        )
    )
    add("windows-programming", "윈도우프로그래밍2", ["윈도우프로그래밍2"], ["document"])

    return items


def extract_meta_assignment(text: str) -> dict[str, str]:
    """과제 본문에서 학번, 수강번호, 이름, 지도교수, 제출일 값을 정규식으로 골라 딕셔너리로 돌려준다."""
    meta = {}
    patterns = [
        ("학번", r"학\s*번\s*[:：]\s*(\d+)"),
        ("수강번호", r"수\s*강\s*번\s*호\s*[:：]\s*(\d+)"),
        ("이름", r"이\s*름\s*[:：]\s*(\S+)"),
        ("지도교수", r"지도교수\s*[:：]\s*(\S+)"),
        ("제출일", r"제출일\s*[:：]\s*([\d\-]+)"),
    ]
    for key, pat in patterns:
        m = re.search(pat, text)
        if m:
            meta[key] = m.group(1)
    return meta


_ASSIGNMENT_META_FIELDS = re.compile(
    r"(?:"
    r"과\s*제\s*주\s*제|과목\s*명|지\s*도\s*교\s*수|담\s*당\s*교\s*수|"
    r"학\s*번|수\s*강\s*번\s*호|이\s*름|제\s*출\s*일(?:\s*자)?"
    r")\s*[:：]",
    re.I,
)
_ASSIGNMENT_META_LINE = re.compile(
    r"^(?:[•\-\*·]\s*)?"
    r"(?:학\s*번|수\s*강\s*번\s*호|이\s*름|지도\s*교수|담당\s*교수|"
    r"제출\s*일(?:\s*자)?|과목\s*명|과\s*제\s*주\s*제|총\s*괄\s*과\s*제)"
    r"\s*[:：]",
    re.I,
)
_ASSIGNMENT_SECTION_HEADER = re.compile(
    r"^(?:과제\s*주제|과제에\s*대한\s*답변|출력\s*결과|과제\s*정보|총\s*괄\s*과\s*제)\s*$",
    re.I,
)
_ASSIGNMENT_DEPT_LINE = re.compile(r"^[-\s•*·]*[-\s].*[-\s•*·]$")


def _assignment_line_inner(line: str) -> str:
    """줄 앞의 글머리표와 중복 공백을 제거한 속 내용을 돌려준다."""
    return re.sub(r"\s+", " ", line.strip().lstrip("•-*· ").strip())


def _is_assignment_meta_line(line: str) -> bool:
    """학번, 제출일, 과목명, 학과 줄처럼 과제 표지 메타인지 판별한다."""
    s = line.strip()
    if not s:
        return False
    inner = _assignment_line_inner(s)
    if _ASSIGNMENT_META_LINE.match(s) or _ASSIGNMENT_META_LINE.match(inner):
        return True
    if _ASSIGNMENT_META_FIELDS.search(inner):
```

```
        return True
    if re.match(r"^총\s*괄\s*과\s*제\s*$", inner, re.I):
        return True
    if _ASSIGNMENT_DEPT_LINE.match(s) and re.search(r"[가-힣]", s):
        return True
    return False


def strip_assignment_cover_block(text: str) -> str:
    """표지·과제주제 라벨·학번/제출일 등 메타 구간을 본문(서론·목차 등) 전까지 제거."""
    lines = text.splitlines()
    kept: list[str] = []
    in_cover = True
    skip_topic_lines = False

    for ln in lines:
        s = ln.strip()
        if in_cover:
            if not s:
                continue
            if re.fullmatch(r"={3,}", s):
                continue
            if _is_assignment_meta_line(ln):
                skip_topic_lines = False
                continue
            if re.search(r"과\s*제\s*주\s*제\s*[:：]", s, re.I):
                skip_topic_lines = True
                continue
            if skip_topic_lines:
                if re.match(r"^[•\-\*·]", s):
                    if _is_assignment_meta_line(ln):
                        continue
                    skip_topic_lines = False
                    continue
                continue
            if re.match(r"^(?:서론|목차|본론|결론)\b", s):
                in_cover = False
                kept.append(ln)
                continue
            if s.startswith("  ") or len(s) > 55:
                in_cover = False
                kept.append(ln)
                continue
            if len(s) <= 40:
                continue
            in_cover = False
            kept.append(ln)
        else:
            kept.append(ln)
    return "\n".join(kept)


def strip_assignment_meta_text(text: str) -> str:
    """과제 원본에서 학번·제출일·과제주제 표지 등 메타 블록을 제거."""
    if not text:
        return text
    text = re.sub(
        r"^[ \t]*={3,}[ \t]*\n(?:.*?\n)*?[ \t]*={3,}[ \t]*\n+",
        "",
        text,
        count=1,
        flags=re.M,
    )
    text = re.sub(
        r"^[ \t]*[•\-\*·][ \t]*"
        r"(?:과목\s*명|지\s*도\s*교\s*수|담\s*당\s*교\s*수|이\s*름|학\s*번|"
        r"수\s*강\s*번\s*호|제\s*출\s*일(?:\s*자)?)\s*[:：][^\n]*\n?",
```

```
        "",
        text,
        flags=re.M | re.I,
    )
    text = strip_assignment_cover_block(text)
    kept: list[str] = []
    for ln in text.splitlines():
        s = ln.strip()
        if not s:
            kept.append("")
            continue
        if re.fullmatch(r"={3,}", s):
            continue
        if _is_assignment_meta_line(ln):
            continue
        if _ASSIGNMENT_SECTION_HEADER.match(s):
            continue
        kept.append(ln)
    return "\n".join(kept)


def is_assignment_meta_bullet(bullet: str) -> bool:
    """요약 목록에 넣으면 안 되는 표지 정보, 경로, 코드 잡음인지 판별한다."""
    s = bullet.strip()
    if not s:
        return True
    inner = _assignment_line_inner(s)
    if _ASSIGNMENT_META_FIELDS.search(inner):
        return True
    if re.match(r"^총\s*괄\s*과\s*제\s*$", inner, re.I):
        return True
    if _ASSIGNMENT_DEPT_LINE.match(s) and re.search(r"[가-힣]", s):
        return True
    if re.match(r"^C:[/\\]", s):
        return True
    if re.match(r"^(?:File |Traceback|using |import |#include|Private Sub )", s, re.I):
        return True
    return False


def summarize_assignment_bullets(text: str, max_items: int = 10) -> list[str]:
    """과제 본문에서 표지 메타를 뺀 뒤, 요약에 쓸 불릿을 최대 개수만큼 고른다."""
    cleaned = strip_assignment_meta_text(mask_assignment_privacy(text))
    raw = summarize_bullets(cleaned, max_items * 2)
    out: list[str] = []
    for b in raw:
        if is_assignment_meta_bullet(b):
            continue
        out.append(b)
        if len(out) >= max_items:
            break
    return out


def _clip_prose(s: str, max_len: int = 200) -> str:
    """문장을 최대 길이로 자르고 말끝을 부드럽게 다듬는다."""
    s = re.sub(r"\s+", " ", s.strip())
    s = re.sub(r"^[•\-\*·]\s*", "", s)
    if not s:
        return ""
    if len(s) <= max_len:
        return soften_sentence(s)
    clipped = s[: max_len - 1].rsplit(" ", 1)[0]
    return soften_sentence(clipped + "…")


def _is_readable_assignment_prose(line: str) -> bool:
```

```
    """과제 줄이 사람이 읽을 산문인지 본다. 공백 없는 긴 토큰은 제외한다."""
    s = line.strip()
    if not is_prose_line(s):
        return False
    if len(s) > 48 and " " not in s:
        return False
    return True


def extract_assignment_points(text: str, max_items: int = 15) -> list[str]:
    """과제 본문에서 서로 다른 읽을 수 있는 줄을 앞에서부터 최대 개수만큼 모은다."""
    points: list[str] = []
    seen: set[str] = set()
    for ln in text.splitlines():
        if not ln.strip():
            continue
        ln = re.sub(r"\s+", " ", ln.strip())
        if not _is_readable_assignment_prose(ln):
            continue
        key = ln[:50]
        if key in seen:
            continue
        seen.add(key)
        points.append(ln)
        if len(points) >= max_items:
            break
    return points


def _split_assignment_sections(text: str) -> dict[str, str]:
    """서론, 본론, 결론 제목을 기준으로 과제 본문을 세 구간 문자열로 나눈다."""
    buffers: dict[str, list[str]] = {"intro": [], "body": [], "outro": []}
    active = "intro"
    for ln in text.splitlines():
        s = ln.strip()
        if re.match(r"^서론\s*$", s):
            active = "intro"
            continue
        if re.match(r"^본론\s*$", s):
            active = "body"
            continue
        if re.match(r"^결론\s*$", s):
            active = "outro"
            continue
        buffers[active].append(ln)
    return {k: "\n".join(v) for k, v in buffers.items() if any(x.strip() for x in v)}


def _has_assignment_section_markers(text: str) -> bool:
    """본문에 서론과 본론 제목이 둘 다 있는지 확인한다."""
    return bool(re.search(r"^서론\s*$", text, re.M)) and bool(
        re.search(r"^본론\s*$", text, re.M)
    )


def _extract_assignment_topics(text: str) -> list[str]:
    """목차나 짧은 제목 줄에서 과제 주제 후보를 최대 여섯 개 고른다."""
    topics: list[str] = []
    prefer = False
    for ln in text.splitlines():
        s = ln.strip()
        if re.match(r"^목차\s*$", s):
            prefer = True
            continue
        if not s or len(s) < 6 or len(s) > 48:
            continue
        if _is_assignment_meta_line(ln):
```

```
            continue
        if re.match(r"^\d+$", s):
            continue
        if re.match(r"^(?:서론|본론|결론|목차|참고|출력)", s):
            continue
        if not re.search(r"[가-힣]", s):
            continue
        if s in topics:
            continue
        if prefer or len(topics) < 4:
            topics.append(s)
        if len(topics) >= 6:
            break
    if len(topics) < 3:
        topics = []
        for ln in text.splitlines():
            s = ln.strip()
            if not s or len(s) < 8 or len(s) > 48:
                continue
            if _is_assignment_meta_line(ln) or re.match(r"^\d+$", s):
                continue
            if re.match(r"^(?:서론|본론|결론|목차|참고)", s):
                continue
            if re.search(r"[가-힣]", s) and s not in topics:
                topics.append(s)
            if len(topics) >= 6:
                break
    return topics


def _assignment_summary_filler(item: CatalogItem, index: int) -> str:
    """요약 문장이 부족할 때 순번에 맞춰 넣는 기본 문장을 돌려준다."""
    fillers = [
        f"{item.title} 총괄과제에서 요구한 주제랑 형식에 맞춰 답안·산출물을 구성했어요.",
        "본문·코드·도표·발표 자료 같은 제출 형식에 맞게 핵심 내용을 담았어요.",
        "과제 요구사항에 맞춰 조사·정리한 내용을 마무리하고 제출했어요.",
    ]
    return fillers[min(index, len(fillers) - 1)]


_SECTION_VERBS = {
    "서론": "소개했어요",
    "본론": "설명했어요",
    "결론": "마무리했어요",
    "마무리": "마무리했어요",
}


def _sentence_from_section(points: list[str], item: CatalogItem, section_name: str) -> str:
    """서론·본론·결론 구간의 첫 문장을 그 구간에 맞는 말투의 한 문장으로 만든다."""
    verb = _SECTION_VERBS.get(section_name, "정리했어요")
    if not points:
        return f"{section_name}에서는 {item.summary.rstrip('.')} 관련 내용을 {verb}"
    core = _clip_prose(points[0], 52).rstrip(".")
    return f"{section_name}에서는 {core}를 중심으로 {verb}"


def _compose_summary_from_topics(item: CatalogItem, topics: list[str]) -> list[str]:
    """주제 세 개로 개요, 본문, 마무리 형식의 요약 문장 세 개를 만든다."""
    picks = topics[:3]
    while len(picks) < 3:
        picks.append(item.summary)
    t0, t1, t2 = picks[0], picks[1], picks[2]
    return [
        f"과제는 「{t0}」 주제로 개요와 배경부터 소개했어요.",
        f"「{t1}」 파트에서 개념·사례·조사 자료를 표와 도표로 정리했어요.",
        f"「{t2}」 내용으로 결론과 참고 문헌까지 마무리했어요.",
```

```
    ]


def _sentence_from_points(points: list[str], opener: str, fallback: str = "") -> str:
    """고른 문장의 앞부분을 시작 말과 이어 요약 한 문장으로 만든다."""
    if not points:
        base = fallback or "과제 원문에서 핵심만 골라"
        return f"{opener.rstrip('.')} {base} 정리했어요."
    core = _clip_prose(points[0], 52).rstrip(".")
    return f"{opener.rstrip('.')} {core}를 다루며 정리했어요."


def build_assignment_content_summary(item: CatalogItem, all_text: str) -> list[str]:
    """과제 본문에서 구어체 한 문장 요약 3개 생성."""
    cleaned = strip_assignment_meta_text(mask_assignment_privacy(all_text))
    sections = _split_assignment_sections(cleaned)
    section_keys = [("intro", "서론"), ("body", "본론"), ("outro", "결론")]
    section_lines: list[str] = []

    if _has_assignment_section_markers(cleaned):
        for key, name in section_keys:
            sec = sections.get(key, "").strip()
            if not sec:
                continue
            pts = extract_assignment_points(sec, 6)
            line = _sentence_from_section(pts, item, name)
            if line:
                section_lines.append(line)

        if len(section_lines) >= 2:
            if len(section_lines) < 3:
                tail = sections.get("outro", "") or sections.get("body", "")
                pts = extract_assignment_points(tail, 4)
                if pts:
                    section_lines.append(
                        _sentence_from_section([pts[-1]], item, "마무리")
                    )
            while len(section_lines) < 3:
                section_lines.append(_assignment_summary_filler(item, len(section_lines)))
            return section_lines[:3]

    topics = _extract_assignment_topics(cleaned)
    is_slide_pdf = any(s.path.lower().endswith(".pdf") for s in item.sources)
    if len(topics) >= 3 and (is_slide_pdf or len(extract_assignment_points(cleaned, 15)) < 4):
        return _compose_summary_from_topics(item, topics)

    points = extract_assignment_points(cleaned, 15)
    if len(points) < 3:
        if len(topics) >= 3:
            return _compose_summary_from_topics(item, topics)
        if points:
            openers = ["먼저", "이어서", "마무리로는"]
            lines = [
                _sentence_from_points([p], openers[i], f"{item.title} 과제에서")
                for i, p in enumerate(points[:3])
            ]
            while len(lines) < 3:
                lines.append(_assignment_summary_filler(item, len(lines)))
            return lines[:3]
        return [
            _assignment_summary_filler(item, 0),
            _assignment_summary_filler(item, 1),
            _assignment_summary_filler(item, 2),
        ]

    third = max(1, len(points) // 3)
    chunks = [
```

```
        points[:third],
        points[third : third * 2],
        points[third * 2 :],
    ]
    openers = ["앞부분에서는", "중간에서는", "마무리에서는"]
    lines = [
        _sentence_from_points(chunk, openers[i], f"{item.title} 과제에서")
        for i, chunk in enumerate(chunks)
    ]
    for i, line in enumerate(lines):
        if not line.strip():
            lines[i] = _assignment_summary_filler(item, i)
    return lines[:3]


_HANGUL = re.compile(r"[가-힣]")
_CODE_NOISE = re.compile(
    r"^(>>>|\.\.\.|File \"|Traceback|import |from |def |class |#include|"
    r"using namespace|return |if |for |while |else|\{|\}|</|http://|https://)",
    re.I,
)


def is_prose_line(line: str) -> bool:
    """한글이 있고 코드.로그.너무 짧거나 긴 줄이 아닌 서술 줄인지 판별한다."""
    s = line.strip()
    if len(s) < 12 or len(s) > 280:
        return False
    if not _HANGUL.search(s):
        return False
    if _CODE_NOISE.match(s):
        return False
    if s.count("=") > 4 or s.startswith("---"):
        return False
    ascii_ratio = sum(1 for c in s if ord(c) < 128) / len(s)
    if ascii_ratio > 0.65 and "파이썬" not in s and "프로그래" not in s:
        return False
    return True


def extract_study_points(text: str, max_items: int = 8) -> list[str]:
    """노트에서 서술에 쓸 만한 한국어 문장만 추출."""
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    points: list[str] = []
    seen: set[str] = set()
    for ln in lines:
        if not is_prose_line(ln):
            continue
        ln = re.sub(r"\s+", " ", ln)
        key = ln[:50]
        if key in seen:
            continue
        seen.add(key)
        points.append(ln)
        if len(points) >= max_items:
            break
    return points


def soften_sentence(s: str) -> str:
    """문장 끝의 불필요한 마침표와 '등의'를 정리하고, 끝이 없으면 마침표를 붙인다."""
    s = s.rstrip(".")
    s = re.sub(r"\s*등의\s*$", "", s)
    if s and s[-1] not in ".!?":
        s += "."
    return s

```

```python

def weave_narrative(opener: str, points: list[str], fallback: str = "") -> str:
    """고른 문장을 구어체 서술 한 덩어리로 이어 붙인다. 쓸 문장이 없으면 fallback으로 채운다."""
    cleaned = [soften_sentence(p).rstrip(".") for p in points if p.strip()]
    if not cleaned:
        base = fallback or "교재·강의 흐름을 따라가며 전체 개념을 익히는 데 집중"
        return f"{opener.rstrip('.')} {base}했어요."

    if len(cleaned) == 1:
        return f"{opener.rstrip('.')} {cleaned[0]} 내용을 중심으로 공부했어요."

    first = " ".join(cleaned[:3])
    rest = " ".join(cleaned[3:6])
    if rest:
        return (
            f"{opener.rstrip('.')} {first} 같은 내용을 먼저 잡았고, "
            f"이어서 {rest} 부분까지 넓혀 가며 정리했어요."
        )
    return f"{opener.rstrip('.')} {first} 흐름으로 전반적인 내용을 익혔어요."


def _phase_unit_narratives(item: CatalogItem) -> list[tuple[str, str]]:
    """장.강.일차가 많을 때 앞.중.뒤 구간만 구어체로 요약."""
    n = len(item.sources)
    if n < 4:
        return []

    sources = item.sources
    if n <= 6:
        phases = [(0, 0, "처음"), (n // 2, n // 2, "중간"), (n - 1, n - 1, "마지막")]
    else:
        third = max(1, n // 3)
        phases = [
            (0, third - 1, "앞부분"),
            (third, 2 * third - 1, "중반"),
            (2 * third, n - 1, "후반"),
        ]

    blocks: list[tuple[str, str]] = []
    for start, end, label in phases:
        seg = sources[start : end + 1]
        u_from = seg[0].unit
        u_to = seg[-1].unit
        unit_label = u_from if u_from == u_to else f"{u_from}~{u_to}"
        if label in ("처음", "앞부분"):
            para = (
                f"{unit_label} 구간에서는 {item.summary}의 기초를 다지며 "
                f"개념과 환경 설정에 집중했어요."
            )
        elif label in ("마지막", "후반"):
            para = (
                f"{unit_label} 구간에서는 지금까지 배운 내용을 연결·정리하고 "
                f"실전·응용 쪽으로 마무리했어요."
            )
        else:
            para = (
                f"{unit_label} 구간에서는 예제·실습을 따라 하며 "
                f"손에 익히는 단계를 밟았어요."
            )
        blocks.append((unit_label, para))
    return blocks


def build_study_narrative(item: CatalogItem) -> dict[str, Any]:
    """과목 전체를 서술형 구어체로 요약 (원문 나열 없음)."""
    unit_hint = ""
    if len(item.sources) >= 3:
```

```
        units = [s.unit for s in item.sources if s.unit != "전체"]
        if units:
            unit_hint = f" ({units[0]}부터 {units[-1]}까지)"

    overview = (
        f"「{item.title}」을(를) 공부하면서{unit_hint} "
        f"총 {len(item.sources)}개의 노트를 남겼어요. "
        f"한 줄로 말하면 {item.summary} 쪽을 중심으로 학습했다고 보면 됩니다."
    )

    main = STUDY_NARRATIVES.get(item.slug)
    if not main:
        main = (
            f"이 과목에서는 {item.summary}을(를) 목표로 "
            f"{len(item.sources)}번에 걸쳐 내용을 쌓아 갔어요. "
            f"노트는 키워드 위주라서, 여기서는 그때의 학습 흐름과 목표를 "
            f"이야기하듯 풀어 썼어요."
        )

    reflection = (
        f"돌이켜 보면 {item.title}에서 {item.summary}와 관련된 내용을 "
        f"차근차근 쌓아 갔어요. 원본 문장을 그대로 옮기지 않고, "
        f"그때 무엇을 배우려 했는지 전체 흐름으로 정리한 거예요."
    )

    return {
        "overview": overview,
        "main": main,
        "reflection": reflection,
        "units": _phase_unit_narratives(item),
    }


def summarize_bullets(text: str, max_items: int = 10) -> list[str]:
    """글머리표, 번호 목록, 긴 줄을 중복 없이 최대 개수만큼 모아 요약 불릿으로 만든다."""
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    bullets = []
    for ln in lines:
        if len(ln) < 4:
            continue
        if ln.startswith(("-", "*", "•")):
            bullets.append(ln.lstrip("-*• ").strip())
        elif re.match(r"^\d+[\.\)]", ln):
            bullets.append(re.sub(r"^\d+[\.\)]\s*", "", ln))
        elif len(ln) > 15 and not ln.startswith("="):
            bullets.append(ln)
    seen = set()
    out = []
    for b in bullets:
        key = b[:60]
        if key not in seen:
            seen.add(key)
            out.append(b)
        if len(out) >= max_items:
            break
    return out


def make_overview(item: CatalogItem, bullets: list[str]) -> str:
    """학습이면 노트 수와 키워드로, 과제이면 수행 기록과 원본 파일 수로 한눈에 보기 문장을 만든다."""
    if item.type == "assignment":
        return (
            f"{item.title} 과목의 총괄/최종 과제를 수행하며 "
            f"과제 요구사항에 맞춰 답안·코드·문서를 작성했습니다. "
            f"원본 자료는 {len(item.sources)}개 파일에서 확인할 수 있습니다."
        )
    if bullets:
```

```
        topics = "·".join(b[:20] for b in bullets[:3])
        return (
            f"「{item.title}」 학습을 진행하며 {len(item.sources)}개 단위의 노트를 남겼습니다. "
            f"주요 키워드: {topics} 등."
        )
    return f"「{item.title}」 관련 학습 노트를 정리했습니다. 원본 메모가 짧거나 키워드 수준일 수 있습니다."


WINDOWS_PROGRAMMING1_SCREENSHOT = "../data/image/윈도우프로그래밍1-총괄과제.png"


def render_windows_programming1_markdown(item: CatalogItem) -> str:
    """실행 파일 제출 과제 — 텍스트 원문 없이 프로그램 화면으로 설명."""
    lines = [
        "---",
        f"id: {item.id}",
        f"type: {item.type}",
        f"category: {item.category}",
        f'title: "{item.title}"',
        f"slug: {item.slug}",
        f'summary: "{item.summary}"',
        f"tags: {json.dumps(item.tags, ensure_ascii=False)}",
        "progress:",
        "  units: 0",
        "  completed: 0",
        "sources: []",
        "created_from: study_note_record_organization",
        "---",
        "",
        f"# {item.title}",
        "",
        "## 한눈에 보기",
        "",
        "윈도우 프로그래밍1 총괄과제는 Word·한글·PDF 같은 **텍스트 문서가 아니라**, "
        "Visual Basic .NET WinForms로 만든 **실행 프로그램(.exe) 자체**를 제출한 과제입니다. "
        "그래서 다른 과제 카드처럼 독서·과제 노트 원문을 붙여 두지 않았고, "
        "아래 실행 화면 캡처와 동작 설명으로 대신 정리했습니다.",
        "",
        "## 제출 형태",
        "",
        "- **과목**: 윈도우 프로그래밍1",
        "- **제출물**: Windows 데스크톱 앱 (WinForms `.exe`)",
        "- **텍스트 문서**: 없음 — 보고서·답안지 파일은 따로 작성·제출하지 않음",
        "",
        "![윈도우 프로그래밍1 총괄과제 실행 화면](" + WINDOWS_PROGRAMMING1_SCREENSHOT + ")",
        "",
        "*캡처: 총괄과제 WinForms 프로그램 실행 화면. 상단 레이블에 입력 내용이 실시간으로 표시됩니다.*",
        "",
        "## 동작 요약",
        "",
        "- 창 제목 **「총괄과제」** 의 WinForms 폼 하나로 구성",
        "- **상단** `Label`(`lblOutput`): 테두리가 있는 출력 영역",
        "- **하단** `TextBox`(`txtInput`): 여러 줄 입력 가능",
        "- 입력할 때마다 `TextChanged` 이벤트로 하단 글자가 **즉시** 상단 레이블에 같은 내용으로 반영",
        "- 폼이 열릴 때 `lblOutput`에 포커스를 두도록 설정",
        "",
        "## 배운 점 / 적용",
        "",
        "- WinForms 컨트롤 배치·속성 설정과 이벤트 핸들러 연결",
        "- 사용자 입력을 UI에 바로 보여 주는 기본 GUI 프로그래밍",
        "- 문서가 아닌 **동작하는 프로그램**으로 과제 요구를 충족하는 제출 방식",
        "",
    ]
    return "\n".join(lines)


def render_markdown(item: CatalogItem) -> str:
```

```
    """카탈로그 항목 하나의 프론트매터와 본문 마크다운을 만든다. 윈도우 프로그래밍1은 별도 형식을 쓴다."""
    if item.slug == "windows-programming-1":
        return render_windows_programming1_markdown(item)

    all_text = "\n\n".join(s.text for s in item.sources if s.text)
    bullets = summarize_bullets(all_text)
    study_narr = build_study_narrative(item) if item.type == "study" else None
    overview = study_narr["overview"] if study_narr else make_overview(item, bullets)
    fm_summary = item.summary

    progress_units = len(item.sources)
    fm_sources = [{"path": s.path, "unit": s.unit} for s in item.sources]

    lines = [
        "---",
        f"id: {item.id}",
        f"type: {item.type}",
        f"category: {item.category}",
        f'title: "{item.title}"',
        f"slug: {item.slug}",
        f'summary: "{fm_summary}"',
        f"tags: {json.dumps(item.tags, ensure_ascii=False)}",
        "progress:",
        f"  units: {progress_units}",
        f"  completed: {progress_units}",
        "sources:",
    ]
    for s in fm_sources:
        path = mask_assignment_privacy(s["path"]) if item.type == "assignment" else s["path"]
        lines.append(f"  - path: {path}")
        lines.append(f"    unit: \"{s['unit']}\"")
    lines.extend(["created_from: study_note_record_organization", "---", ""])

    lines.append(f"# {item.title}\n")
    lines.append("## 한눈에 보기\n")
    lines.append(overview + "\n")

    if item.type == "study" and study_narr:
        lines.append("## 무엇을 배웠는지\n")
        lines.append(study_narr["main"] + "\n\n")
        lines.append(study_narr["reflection"] + "\n\n")
        if study_narr["units"]:
            lines.append("## 단계별로 살보면\n")
            for unit, para in study_narr["units"]:
                lines.append(f"### {unit}\n")
                lines.append(para + "\n\n")
    elif item.type != "assignment":
        lines.append("## 핵심 요약\n")
        if bullets:
            for b in bullets:
                lines.append(f"- {b}")
        else:
            lines.append("- 원본 메모가 키워드 수준이거나 추출 텍스트가 비어 있습니다.")
        lines.append("")

    if item.type == "assignment":
        lines.append("## 내용 요약\n")
        summary_lines = build_assignment_content_summary(item, all_text)
        for para in summary_lines:
            lines.append(para + "\n")
        lines.append("")

    if item.type == "assignment":
        lines.append("## 배운 점 / 적용\n")
        lines.append(f"- {item.title} 과목 핵심 개념을 과제 답안·구현으로 정리")
        lines.append("- 제출 형식(문서/HTML/PDF/코드)에 맞춘 산출물 작성 경험")
        lines.append("")
```

```
    lines.append("## 원본 출처\n")
    lines.append("아래 링크를 클릭하면 원본.추출 텍스트 전체를 볼 수 있습니다.\n")
    for s in item.sources:
        label, href = source_link_target(s)
        if item.type == "assignment":
            label = mask_assignment_privacy(label)
            disp_path = mask_assignment_privacy(s.path)
            lines.append(f"- [{label}]({href}) ({s.unit}) — `{disp_path}`")
        else:
            lines.append(f"- [{label}]({href}) ({s.unit}) — `{s.path}`")
    lines.append("")

    return mask_secrets("\n".join(lines))


def main() -> None:
    """학습.과제 요약을 output에 쓰고 catalog.json과 index.md를 갱신한 뒤 정적 사이트 빌드를 호출한다."""
    STUDY_OUT.mkdir(parents=True, exist_ok=True)
    ASSIGN_OUT.mkdir(parents=True, exist_ok=True)
    EXTRACTED.mkdir(parents=True, exist_ok=True)

    study_items = build_study_catalog()
    assign_items = build_assignment_catalog()

    all_items = study_items + assign_items
    print(f"Study: {len(study_items)}, Assignment: {len(assign_items)}, Total: {len(all_items)}")

    for item in all_items:
        item.load_sources()

    expected_study = {f"{it.slug}.md" for it in study_items}
    expected_assign = {f"{it.slug}.md" for it in assign_items}
    for d, expected in ((STUDY_OUT, expected_study), (ASSIGN_OUT, expected_assign)):
        for old in d.glob("*.md"):
            if old.name not in expected:
                old.unlink()

    catalog_data = []
    for item in all_items:
        md = render_markdown(item)
        out_dir = STUDY_OUT if item.type == "study" else ASSIGN_OUT
        (out_dir / f"{item.slug}.md").write_text(md, encoding="utf-8")
        catalog_data.append(
            {
                "id": item.id,
                "type": item.type,
                "category": item.category,
                "title": item.title,
                "slug": item.slug,
                "summary": item.summary,
                "tags": item.tags,
                "output": f"output/{item.type}/{item.slug}.md",
                "sources": [{"path": s.path, "unit": s.unit} for s in item.sources],
            }
        )

    (OUT / "catalog.json").write_text(
        json.dumps({"items": catalog_data, "counts": {"study": len(study_items), "assignment": len(assign_items), "to
tal": len(all_items)}}, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

    # index.md
    idx = ["# 학습.과제 기록 인덱스\n", f"총 **{len(all_items)}**개 항목 (학습 {len(study_items)} + 과제 {len(assign_ite
ms)})\n"]
    idx.append("## 학습 (study)\n")
```

```
    for it in study_items:
        idx.append(f"- [{it.title}](study/{it.slug}.md) — {it.summary}")
    idx.append("\n## 과제 (assignment)\n")
    for it in assign_items:
        idx.append(f"- [{it.title}](assignment/{it.slug}.md) — {it.summary}")
    (OUT / "index.md").write_text("\n".join(idx), encoding="utf-8")
    print("Done.")
    try:
        import subprocess
        import sys

        script = ROOT / "scripts" / "build_static_site.py"
        if script.is_file():
            subprocess.run([sys.executable, str(script)], check=False)
    except OSError:
        pass


if __name__ == "__main__":
    main()
```

study_note_record_organization/scripts/build_static_site.py

```
# -*- coding: utf-8 -*-
"""요약 마크다운과 원본 텍스트로 대시보드용 data.js, 소스 JSON, 히어로 목록을 만든다."""
from __future__ import annotations

import hashlib
import json
import re
import shutil
from pathlib import Path
from urllib.parse import unquote

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "output"
SITE = ROOT / "dashboard"
HERO_IMAGE_DIR = ROOT / "data" / "image"
HERO_IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".gif", ".webp", ".bmp"}
CATALOG_PATH = OUT / "catalog.json"
GITHUB_MAX_BYTES = 100 * 1024 * 1024
_ASSIGNMENT_DIR = "과제분류하기"
PROFESSOR_MASK = "OOO 교수님"
STUDENT_ID_MASK = "OOOOOOOO"
KEY_MASK = "142857"

_SECRET_PATTERNS = [
    # Google API Key
    re.compile(r"AIza[0-9A-Za-z\-_]{35}"),
    # OpenAI API Key
    re.compile(r"sk-[a-zA-Z0-9_\-]{20,}"),
    # Anthropic API Key
    re.compile(r"sk-ant-[a-zA-Z0-9_\-]{20,}"),
    # GitHub Token
    re.compile(r"(?:ghp|gho|ghu|ghs|ghr)_[a-zA-Z0-9]{36}|github_pat_[a-zA-Z0-9_]{50,}"),
    # AWS Access Key
    re.compile(r"(?:AKIA|ABIA|ACCA|ASIA)[0-9A-Z]{16}"),
    # Slack Token
    re.compile(r"xox[baprs]-[0-9a-zA-Z]{10,}"),
    # Firebase FCM device registration token
    re.compile(r"[a-zA-Z0-9_\-]{15,}:APA91b[a-zA-Z0-9_\-]{50,}"),
]

_KEY_ASSIGNMENT_PATTERNS = [
    re.compile(
        r'(?i)((?:api[_-]?key|client[_-]?secret|secret[_-]?key|access[_-]?token|service[_-]?key)\s*[:=]\s*["\'])([a-zA
-Z0-9_\-\.\%\+\=]{16,})(["\'])'
    ),
    re.compile(
        r'(?i)((?:android:name=["\']com\.google\.android\.maps\.v2\.API_KEY["\']\s+android:value=["\']))([^"\']+)
(["\'])'
    ),
]


def mask_secrets(text: str, mask: str = KEY_MASK) -> str:
    """본문에서 API 키·토큰 패턴을 찾아 고정 마스크 문자열로 바꾼다."""
    if not text:
        return text
    for pat in _SECRET_PATTERNS:
        text = pat.sub(mask, text)
    for pat in _KEY_ASSIGNMENT_PATTERNS:
        text = pat.sub(rf"\g<1>{mask}\g<3>", text)
    return text

_SOURCE_HREF_RE = re.compile(
```

```
    r'<a href="(/(?:data|_extracted)/[^"]+)"([^>]*)>',
    re.I,
)


def is_assignment_path(rel_path: str) -> bool:
    """경로에 과제 폴더 이름이 있으면 참을 돌려준다."""
    return _ASSIGNMENT_DIR in rel_path.replace("\\", "/")


def mask_assignment_privacy(text: str) -> str:
    """과제 본문의 지도교수 이름, 학번, 파일명 속 학번 숫자를 가린다."""
    if not text:
        return text
    text = re.sub(
        r"(지도교수\s*[:：]\s*)\S+(?:\s*교수님)?",
        rf"\1{PROFESSOR_MASK}",
        text,
    )
    text = re.sub(
        r"((?:학\s*번|수\s*강\s*번\s*호)\s*[:：]\s*)\d+",
        rf"\1{STUDENT_ID_MASK}",
        text,
    )
    text = re.sub(r"\(\d{7,8}\s+", f"({STUDENT_ID_MASK} ", text)
    text = re.sub(r"\(\d{7,8}\)", f"({STUDENT_ID_MASK})", text)
    text = re.sub(r"_\d{7,8}_", f"_{STUDENT_ID_MASK}_", text)
    return text


def parse_frontmatter(raw: str) -> tuple[dict, str]:
    """마크다운 앞의 YAML 프론트매터를 딕셔너리와 본문으로 나눈다. 태그 목록과 sources도 읽는다."""
    match = re.match(r"^---\r?\n([\s\S]*?)\r?\n---\r?\n([\s\S]*)$", raw)
    if not match:
        return {}, raw
    yaml, body = match.group(1), match.group(2)
    meta: dict = {}
    current_key = None
    in_sources = False
    source_item = None

    for line in yaml.split("\n"):
        if line.strip().startswith("sources:"):
            in_sources = True
            meta["sources"] = []
            continue
        if in_sources:
            if re.match(r"^\s+-\s+path:", line):
                source_item = {"path": line.split("path:", 1)[1].strip()}
                meta["sources"].append(source_item)
            elif source_item and re.match(r"^\s+unit:", line):
                source_item["unit"] = (
                    line.split("unit:", 1)[1].strip().strip("\"'")
                )
            elif line.strip() and not line.startswith(" "):
                in_sources = False
        if not in_sources:
            m = re.match(r"^(\w+):\s*(.*)$", line)
            if m:
                current_key = m.group(1)
                val = m.group(2).strip().strip("\"'")
                meta[current_key] = val
            elif current_key == "tags" and re.match(r"^\s+-\s+", line):
                meta.setdefault("tags", [])
                meta["tags"].append(line.split("-", 1)[1].strip())
    if isinstance(meta.get("tags"), str) and meta["tags"].startswith("["):
        try:
```

```
            meta["tags"] = json.loads(meta["tags"].replace("'", '"'))
        except json.JSONDecodeError:
            meta["tags"] = []
    return meta, body


def md_to_html(body: str) -> str:
    """요약 마크다운 본문을 HTML로 바꾼다. markdown 패키지가 없으면 제목·목록·문단만 직접 변환한다."""
    try:
        import markdown

        return markdown.markdown(
            body,
            extensions=["extra", "nl2br", "sane_lists"],
            output_format="html5",
        )
    except ImportError:
        html = []
        for block in re.split(r"\n\n+", body.strip()):
            block = block.strip()
            if not block:
                continue
            if block.startswith("### "):
                html.append(f"<h3>{_esc(block[4:])}</h3>")
            elif block.startswith("## "):
                html.append(f"<h2>{_esc(block[3:])}</h2>")
            elif block.startswith("# "):
                html.append(f"<h1>{_esc(block[2:])}</h1>")
            elif block.startswith("- "):
                items = "".join(
                    f"<li>{_inline_md(line[2:])}</li>"
                    for line in block.split("\n")
                    if line.startswith("- ")
                )
                html.append(f"<ul>{items}</ul>")
            else:
                html.append(f"<p>{_inline_md(block.replace(chr(10), ' '))}</p>")
        return "\n".join(html)


def _esc(s: str) -> str:
    """HTML에 넣을 문자열의 &, <, >, 따옴표를 이스케이프한다."""
    return (
        s.replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace('"', "&quot;")
    )


def _inline_md(s: str) -> str:
    """한 줄 안의 굵게, 코드, 사이트 내부 링크를 HTML로 바꾼다."""
    s = _esc(s)
    s = re.sub(r"\*\*(.+?)\*\*", r"<strong>\1</strong>", s)
    s = re.sub(r"`([^`]+)`", r"<code>\1</code>", s)
    s = re.sub(
        r"\[([^\]]+)\]\((/[^)]+)\)",
        r'<a href="\2">\1</a>',
        s,
    )
    return s


def normalize_source_key(key: str) -> str:
    """마크다운 링크의 퍼센트 인코딩을 풀어 디스크의 원본 경로와 같게 만든다."""
    k = key.replace("\\", "/").lstrip("/")
    prev = None
```

```
    while prev != k:
        prev = k
        k = unquote(k)
    return k


def rewrite_source_links(html: str, pending: set[str]) -> str:
    """본문 HTML의 /data, /_extracted 링크를 원본 보기용 data-source 앵커로 바꾸고, 필요한 원본 키를 모은다."""
    def repl(m: re.Match) -> str:
        """링크 하나에서 원본 키를 꺼내 pending에 넣고, source-link 클래스가 있는 앵커로 바꾼다."""
        href = m.group(1)
        key = normalize_source_key(href)
        pending.add(key)
        attrs = m.group(2) or ""
        if "class=" in attrs:
            attrs = re.sub(
                r'class="([^"]*)"',
                lambda cm: f'class="{cm.group(1)} source-link"',
                attrs,
                count=1,
            )
        else:
            attrs += ' class="source-link"'
        return f'<a href="#" data-source="{key}"{attrs}>'

    return _SOURCE_HREF_RE.sub(repl, html)


def resolve_source_path(key: str) -> Path | None:
    """원본 키를 output/_extracted 또는 저장소 data 아래의 실제 파일 경로로 바꾼다."""
    key = normalize_source_key(key)
    if key.startswith("_extracted/"):
        return OUT / key
    if key.startswith("data/"):
        return ROOT / key
    return None


def read_text_file(path: Path) -> str:
    """utf-8, utf-8-sig, cp949 순으로 읽고, 실패하면 깨진 글자를 대체해 utf-8로 읽는다."""
    for enc in ("utf-8", "utf-8-sig", "cp949"):
        try:
            return path.read_text(encoding=enc)
        except (UnicodeDecodeError, OSError):
            continue
    return path.read_text(encoding="utf-8", errors="replace")


def read_source_text(key: str) -> str | None:
    """원본 키의 파일을 읽어 비밀 값을 가리고, 과제면 개인정보도 가린 뒤 돌려준다. 파일이 없으면 None이다."""
    path = resolve_source_path(key)
    if not path or not path.is_file():
        return None
    try:
        text = read_text_file(path)
    except OSError:
        return None
    text = mask_secrets(text)
    if is_assignment_path(key):
        text = mask_assignment_privacy(text)
    return text


def collect_hero_images() -> list[str]:
    """data/image의 이미지 파일을 정렬해 대시보드에서 쓰는 상대 경로 목록으로 만든다."""
    if not HERO_IMAGE_DIR.is_dir():
        return []
```

```
    paths: list[str] = []
    for path in sorted(HERO_IMAGE_DIR.iterdir(), key=lambda p: p.name.lower()):
        if not path.is_file():
            continue
        if path.suffix.lower() not in HERO_IMAGE_EXTS:
            continue
        paths.append(f"../data/image/{path.name}")
    return paths


def write_hero_images_js(js_dir: Path) -> int:
    """히어로 이미지 경로를 window.HERO_IMAGES 대입문으로 hero-images.js에 쓰고 개수를 돌려준다."""
    images = collect_hero_images()
    content = "window.HERO_IMAGES = " + json.dumps(images, ensure_ascii=False) + ";\n"
    (js_dir / "hero-images.js").write_text(content, encoding="utf-8")
    return len(images)


def source_chunk_filename(key: str, text: str) -> str:
    """원본 키와 본문 해시로 sources 폴더에 저장할 JSON 파일 이름을 만든다."""
    key_digest = hashlib.sha256(key.encode("utf-8")).hexdigest()[:8]
    content_digest = hashlib.sha256(text.encode("utf-8")).hexdigest()[:8]
    return f"{key_digest}-{content_digest}.json"


def source_build_id(source_map: dict[str, str]) -> str:
    """원본 지도 전체를 해시해 캐시 무효화에 쓰는 짧은 빌드 아이디를 만든다."""
    payload = "|".join(f"{k}:{v}" for k, v in sorted(source_map.items()))
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()[:12]


def stamp_index_scripts(build_id: str) -> None:
    """대시보드 index.html의 data.js, app.js 주소에 빌드 아이디 쿼리를 붙인다."""
    index_path = SITE / "index.html"
    html = index_path.read_text(encoding="utf-8")
    html = re.sub(
        r'src="js/(data|app)\.js(?:\?[^"]*)?"',
        lambda m: f'src="js/{m.group(1)}.js?v={build_id}"',
        html,
    )
    index_path.write_text(html, encoding="utf-8")


def assert_under_github_limit(path: Path, label: str) -> None:
    """파일 크기가 GitHub 한 파일 한도를 넘으면 빌드를 중단한다."""
    size = path.stat().st_size
    if size > GITHUB_MAX_BYTES:
        mb = size / (1024 * 1024)
        raise SystemExit(
            f"{label} ({path}) is {mb:.1f} MB — exceeds GitHub 100 MB file limit."
        )


def collect_catalog(catalog_data: dict) -> list[dict]:
    """catalog.json 항목에서 카드에 필요한 id, 종류, 제목, 슬러그, 요약, 태그만 고른다."""
    items = []
    for it in catalog_data.get("items", []):
        items.append(
            {
                "id": it["id"],
                "type": it["type"],
                "title": it["title"],
                "slug": it["slug"],
                "summary": it.get("summary", ""),
                "tags": it.get("tags") or [],
            }
        )
```

```
    return items


def build_site() -> None:
    """요약 HTML, 원본 JSON, data.js, 히어로 목록을 dashboard에 쓴다. catalog.json이 없으면 중단한다."""
    if not CATALOG_PATH.is_file():
        raise SystemExit("catalog.json 없음. 먼저 python scripts/build_output.py 실행")

    catalog_data = json.loads(CATALOG_PATH.read_text(encoding="utf-8"))
    catalog = collect_catalog(catalog_data)
    articles: dict[str, str] = {}
    pending_sources: set[str] = set()

    for item in catalog_data.get("items", []):
        md_path = ROOT / item["output"]
        if not md_path.is_file():
            continue
        raw = md_path.read_text(encoding="utf-8")
        _, body = parse_frontmatter(raw)
        html = md_to_html(mask_secrets(body))
        html = rewrite_source_links(html, pending_sources)
        articles[item["slug"]] = html

    source_map: dict[str, str] = {}
    sources_dir = SITE / "js" / "sources"
    if sources_dir.is_dir():
        shutil.rmtree(sources_dir)
    sources_dir.mkdir(parents=True, exist_ok=True)

    written_sources = 0
    sources_total_bytes = 0
    for key in sorted(pending_sources):
        text = read_source_text(key)
        if text is None:
            continue
        chunk_name = source_chunk_filename(key, text)
        chunk_path = sources_dir / chunk_name
        chunk_payload = {"key": key, "text": text}
        chunk_path.write_text(
            json.dumps(chunk_payload, ensure_ascii=False),
            encoding="utf-8",
        )
        assert_under_github_limit(chunk_path, "Source chunk")
        source_map[key] = chunk_name
        written_sources += 1
        sources_total_bytes += chunk_path.stat().st_size

    build_id = source_build_id(source_map)
    site_data = {
        "catalog": catalog,
        "articles": articles,
        "sourceMap": source_map,
        "sourceBuildId": build_id,
        "counts": catalog_data.get("counts", {}),
    }

    js_dir = SITE / "js"
    css_dir = SITE / "css"
    js_dir.mkdir(parents=True, exist_ok=True)
    css_dir.mkdir(parents=True, exist_ok=True)

    data_js_path = js_dir / "data.js"
    data_js = "window.SITE_DATA = " + json.dumps(site_data, ensure_ascii=False) + ";\n"
    data_js_path.write_text(data_js, encoding="utf-8")
    assert_under_github_limit(data_js_path, "data.js")
    stamp_index_scripts(build_id)
    hero_count = write_hero_images_js(js_dir)
```

```
397
398     print(f"Static data: {len(catalog)} items, {written_sources} source chunks")
399     print(f"Wrote {data_js_path} ({data_js_path.stat().st_size // 1024} KB)")
400     print(
401         f"Wrote {sources_dir} ({written_sources} files, "
402         f"{sources_total_bytes // 1024} KB total)"
403     )
404     print(f"Wrote {js_dir / 'hero-images.js'} ({hero_count} hero images)")
405
406
407 def main() -> None:
408     """build_site를 실행하고, 만든 대시보드를 브라우저에서 열 수 있다고 안내한다."""
409     build_site()
410     print("Done. Open dashboard/index.html in a browser (or use a local static server).")
411
412
413 if __name__ == "__main__":
414     main()
```

study_note_record_organization/scripts/extract_sources.py

```python
# -*- coding: utf-8 -*-
"""build_output.main을 호출해 원본 추출, 요약 마크다운, 정적 사이트 데이터 갱신을 한 번에 실행한다."""
from build_output import main

if __name__ == "__main__":
    main()
```